RUNGTA INTERNATIONAL SKILLS UNIVERSITY

SCHOOL OF INFORMATION TECHNOLOGY

BACHELOR OF COMPUTER APPLICATION

BCA / FIRST SEM / SECTION - 1A

Programming Fundamentals Using Python (RU-100-05-00058)

Date of Allocation:

Date of Submission:

Rubrics:

Rubrics Name	***Internal Marks***	***Evaluation Criteria***	***External Marks***
Logic Building & Overall Implementation	7	Problem understanding, logic building, correctness, functionality and overall implementation	7
Presentation & Viva	6	Project explanation, presentation skills, understanding and viva responses	6
Documentation & Report Submission on GitHub	3	Documentation quality, report completeness and GitHub submission	3
Technology Relevance / Code Quality	4	Appropriate technology, coding practices, readability, structure and code quality	4
Total	20		20
Total 40			

RUNGTA INTERNATIONAL SKILLS UNIVERSITY
SCHOOL OF INFORMATION TECHNOLOGY

BACHELOR OF COMPUTER APPLICATION

BCA / FIRST SEM / SECTION · 1A

Programming Fundamentals Using Python (RU-100-05-00058)

PROJECT DESCRIPTION

<table>
<tr><td>PS No.</td><td>1</td></tr>
<tr><td>Project Title</td><td>Number Guessing Game</td></tr>
<tr><td>Problem Statement</td><td>Create a game in which the computer picks a random number between 1 and 100 and the player guesses it within limited attempts, receiving 'Too high' or 'Too low' hints.
Language: Python 3 Concepts: Loops, Conditions, Functions, Random
Note: Player gets 5 attempts. For testing, the secret number is fixed to 42.</td></tr>
<tr><td>Test Cases / Case Statements (4 cases)</td><td>
<table>
<tr><th>INPUT</th><th>EXPECTED OUTPUT</th></tr>
<tr><td colspan="2">Test Case 1: Normal Input</td></tr>
<tr><td>Secret number: 42
Guess: 42</td><td>Congratulations! You guessed it in 1 attempt.</td></tr>
<tr><td colspan="2">Test Case 2: Boundary / Minimum Input</td></tr>
<tr><td>Secret number: 42
Guesses: 1, then 100 (range limits)</td><td>Too low! Attempts left: 4
Too high! Attempts left: 3</td></tr>
<tr><td colspan="2">Test Case 3: Invalid Input</td></tr>
<tr><td>Secret number: 42
Guesses: abc, then 150</td><td>Invalid input. Enter a whole number.
Out of range. Enter a number between 1 and 100.
Attempts left: 5 (no attempt lost)</td></tr>
<tr><td colspan="2">Test Case 4: Different Valid Scenario</td></tr>
<tr><td>Secret number: 42
Guesses: 10, 90, 50, 30, 20</td><td>Too low | Too high | Too high | Too low | Too low
Game Over! The number was 42.</td></tr>
</table>
</td></tr>
<tr><td>Bare Minimum Requirements</td><td>1. Write the solution in Python 3 (one .py file) and run it from the console.
2. Generate the secret number using the random module.
3. Use a loop to control the 5 attempts and give Too high / Too low hints.
4. Validate the input (numbers only, within range).
5. Show attempts used and ask 'Play again? (y/n)' at the end.</td></tr>
<tr><td>Instruction</td><td>1. Understand the problem statement and define the problem clearly.
2. Develop the solution using proper logic and programming concepts.
3. Implement all required features and pass the given test cases.
4. Test and debug the project to ensure correct output.
5. Prepare the documentation/report and upload the code and report to GitHub.
6. Complete and submit the project within the given deadline.</td></tr>
</table>

BACHELOR OF COMPUTER APPLICATION

<table>
<tr><td>PS No.</td><td>2</td></tr>
<tr><td>Project Title</td><td>Personal Expense Tracker</td></tr>
<tr><td>Problem Statement</td><td>Build a tool to record daily expenses by category and date, store them in a CSV file and generate summaries such as category-wise and monthly spending.
Language: Python 3 Concepts: Lists, Dictionaries, CSV, Pandas
Note: Expenses are saved in expenses.csv with columns Date, Category, Description, Amount.</td></tr>
<tr><td>Test Cases / Case Statements (4 cases)</td><td>
<table>
<tr><th>INPUT</th><th>EXPECTED OUTPUT</th></tr>
<tr><td colspan="2">Test Case 1: Normal Input</td></tr>
<tr><td>Menu option 1 (Add)
2026-08-10, Food, Lunch, 250</td><td>Expense saved. Total records: 1</td></tr>
<tr><td colspan="2">Test Case 2: Boundary / Minimum Input</td></tr>
<tr><td>Menu option 1 (Add)
2026-08-10, Travel, Bus, 1 (minimum amount)</td><td>Expense saved. Total records: 1</td></tr>
<tr><td colspan="2">Test Case 3: Invalid Input</td></tr>
<tr><td>Menu option 1 (Add)
2026-08-10, Food, Tea, -50</td><td>Invalid amount. Amount must be a positive number. Expense not saved.</td></tr>
<tr><td colspan="2">Test Case 4: Different Valid Scenario</td></tr>
<tr><td>Records: Food 250, Travel 100, Food 150
Menu option 3 (Category summary)</td><td>Food: 400 | Travel: 100
Total: 500</td></tr>
</table>
</td></tr>
<tr><td>Bare Minimum Requirements</td><td>1. Write the solution in Python 3 (one .py file) and run it from the console.
2. Add expenses with date, category, description and amount.
3. Store and read data using a CSV file.
4. Show a category-wise summary using Pandas groupby().
5. Validate the date format and the amount before saving.</td></tr>
<tr><td>Instruction</td><td>1. Understand the problem statement and define the problem clearly.
2. Develop the solution using proper logic and programming concepts.
3. Implement all required features and pass the given test cases.
4. Test and debug the project to ensure correct output.
5. Prepare the documentation/report and upload the code and report to GitHub.
6. Complete and submit the project within the given deadline.</td></tr>
</table>

BACHELOR OF COMPUTER APPLICATION

<table>
<tr><td>PS No.</td><td>3</td></tr>
<tr><td>Project Title</td><td>Electricity Bill Calculator</td></tr>
<tr><td>Problem Statement</td><td>Write a program that calculates an electricity bill from the units consumed using slab-wise rates, a fixed charge and tax, and prints a formatted bill.
Language: Python 3 Concepts: Conditions, Operators, Functions
Note: Slabs: first 100 units Rs.3, next 100 units Rs.5, above 200 units Rs.8. Fixed charge Rs.50. Tax 5% on (energy + fixed charge).</td></tr>
<tr><td>Test Cases / Case Statements (4 cases)</td><td>
<table>
<tr><th>INPUT</th><th>EXPECTED OUTPUT</th></tr>
<tr><td colspan="2">Test Case 1: Normal Input</td></tr>
<tr><td>Units consumed: 80</td><td>Energy: 240.00 | Fixed: 50.00 | Tax: 14.50
Total Bill: 304.50</td></tr>
<tr><td colspan="2">Test Case 2: Boundary / Minimum Input</td></tr>
<tr><td>Units consumed: 100 (slab limit)</td><td>Energy: 300.00 | Fixed: 50.00 | Tax: 17.50
Total Bill: 367.50</td></tr>
<tr><td colspan="2">Test Case 3: Invalid Input</td></tr>
<tr><td>Units consumed: -20, then abc</td><td>Invalid units. Enter a non-negative number.</td></tr>
<tr><td colspan="2">Test Case 4: Different Valid Scenario</td></tr>
<tr><td>Units consumed: 250</td><td>Energy: 1200.00 | Fixed: 50.00 | Tax: 62.50
Total Bill: 1312.50</td></tr>
</table>
</td></tr>
<tr><td>Bare Minimum Requirements</td><td>1. Write the solution in Python 3 (one .py file) and run it from the console.
2. Take consumer name and units as input.
3. Use if-elif-else for slab-wise calculation.
4. Create functions such as calculate_energy_charge() and generate_bill().
5. Print the bill in a neat, formatted layout.</td></tr>
<tr><td>Instruction</td><td>1. Understand the problem statement and define the problem clearly.
2. Develop the solution using proper logic and programming concepts.
3. Implement all required features and pass the given test cases.
4. Test and debug the project to ensure correct output.
5. Prepare the documentation/report and upload the code and report to GitHub.
6. Complete and submit the project within the given deadline.</td></tr>
</table>

BACHELOR OF COMPUTER APPLICATION

<table>
<tr><td>PS No.</td><td>4</td></tr>
<tr><td>Project Title</td><td>Quiz Application</td></tr>
<tr><td>Problem Statement</td><td>Develop a console quiz that asks multiple-choice questions randomly from a question bank, checks the answers, keeps score and displays the result.
Language: Python 3 Concepts: Lists, Dictionaries, Loops, Random
Note: For testing, the questions are asked in this order: Q1 'Keyword to define a function?' (A func, B def, C function, D lambda), Q2 'len("Hi") = ?' (A 1, B 2, C 3, D 4), Q3 'Mutable type?' (A tuple, B str, C list, D int). Pass mark is 50%.</td></tr>
<tr><td>Test Cases / Case Statements (4 cases)</td><td>
<table>
<tr><th>INPUT</th><th>EXPECTED OUTPUT</th></tr>
<tr><td colspan="2">Test Case 1: Normal Input</td></tr>
<tr><td>Answers: B, B, C</td><td>Score: 3/3 | Percentage: 100.00%
Result: Pass</td></tr>
<tr><td colspan="2">Test Case 2: Boundary / Minimum Input</td></tr>
<tr><td>Answers: A, A, A</td><td>Score: 0/3 | Percentage: 0.00%
Result: Fail
Correct answers: B, B, C</td></tr>
<tr><td colspan="2">Test Case 3: Invalid Input</td></tr>
<tr><td>Q1 answer: E, then B</td><td>Invalid option. Enter A, B, C or D.
(Q1 is asked again) Correct!</td></tr>
<tr><td colspan="2">Test Case 4: Different Valid Scenario</td></tr>
<tr><td>Answers: B, A, C</td><td>Q2 Wrong. Correct answer: B
Score: 2/3 | Percentage: 66.67% | Result: Pass</td></tr>
</table>
</td></tr>
<tr><td>Bare Minimum Requirements</td><td>1. Write the solution in Python 3 (one .py file) and run it from the console.
2. Keep at least 10 questions in a list of dictionaries (question, options, answer).
3. Select questions randomly using the random module.
4. Check every answer and keep the score.
5. Show the final score, percentage and a pass/fail message.</td></tr>
<tr><td>Instruction</td><td>1. Understand the problem statement and define the problem clearly.
2. Develop the solution using proper logic and programming concepts.
3. Implement all required features and pass the given test cases.
4. Test and debug the project to ensure correct output.
5. Prepare the documentation/report and upload the code and report to GitHub.
6. Complete and submit the project within the given deadline.</td></tr>
</table>

BACHELOR OF COMPUTER APPLICATION

<table>
<tr><td>PS No.</td><td>5</td></tr>
<tr><td>Project Title</td><td>Sales Data Analyzer</td></tr>
<tr><td>Problem Statement</td><td>Analyse a sales CSV file to find total revenue, best-selling products and region-wise performance using grouping, and visualise the results.
Language: Python 3 Concepts: Pandas, CSV, Matplotlib, Grouping
Note: CSV columns: Date, Product, Region, Quantity, Amount.</td></tr>
<tr><td>Test Cases / Case Statements (4 cases)</td><td>
<table>
<tr><th>INPUT</th><th>EXPECTED OUTPUT</th></tr>
<tr><td colspan="2">Test Case 1: Normal Input</td></tr>
<tr><td>sales.csv:
2026-08-01,Laptop,North,2,100000
2026-08-02,Mouse,South,10,5000
2026-08-03,Laptop,South,1,50000
Menu option 1 (Summary)</td><td>Total Revenue: 155000 | Transactions: 3 | Units sold: 13</td></tr>
<tr><td colspan="2">Test Case 2: Boundary / Minimum Input</td></tr>
<tr><td>sales.csv: 1 row only
2026-08-01,Mouse,North,1,500
Menu option 1 (Summary)</td><td>Total Revenue: 500 | Transactions: 1 | Top product: Mouse</td></tr>
<tr><td colspan="2">Test Case 3: Invalid Input</td></tr>
<tr><td>sales.csv:
2026-08-01,Laptop,North,2,100000
2026-08-02,Mouse,South,10,abc
2026-08-03,Laptop,South,1,50000</td><td>Row 2 skipped: invalid Amount 'abc'
Total Revenue: 150000 | Transactions: 2</td></tr>
<tr><td colspan="2">Test Case 4: Different Valid Scenario</td></tr>
<tr><td>Same CSV as Test Case 1
Menu option 2 (Group by Product & Region)</td><td>Laptop: 150000 | Mouse: 5000
North: 100000 | South: 55000
Bar chart 'Region-wise Sales' displayed</td></tr>
</table>
</td></tr>
<tr><td>Bare Minimum Requirements</td><td>1. Write the solution in Python 3 (one .py file) and run it from the console.
2. Read the CSV file using Pandas and clean invalid or duplicate rows.
3. Use groupby() to find product-wise and region-wise sales.
4. Show the top 5 selling products.
5. Plot at least two Matplotlib charts (bar and line).</td></tr>
<tr><td>Instruction</td><td>1. Understand the problem statement and define the problem clearly.
2. Develop the solution using proper logic and programming concepts.
3. Implement all required features and pass the given test cases.
4. Test and debug the project to ensure correct output.
5. Prepare the documentation/report and upload the code and report to GitHub.
6. Complete and submit the project within the given deadline.</td></tr>
</table>

BACHELOR OF COMPUTER APPLICATION

<table>
<tr><td>PS No.</td><td>6</td></tr>
<tr><td>Project Title</td><td>Contact Management System</td></tr>
<tr><td>Problem Statement</td><td>Create a contact book application that supports Create, Read, Update and Delete (CRUD) operations on contacts with search by name.
Language: Python 3 Concepts: Lists, Dictionaries, Functions, CRUD
Note: Phone number must be exactly 10 digits and unique. Test cases run in order on the same contact list.</td></tr>
<tr><td>Test Cases / Case Statements (4 cases)</td><td>
<table>
<tr><th>INPUT</th><th>EXPECTED OUTPUT</th></tr>
<tr><td colspan="2">Test Case 1: Normal Input</td></tr>
<tr><td>Menu option 1 (Add)
Ravi Sharma, 9876543210, ravi@mail.com</td><td>Contact added: Ravi Sharma | 9876543210</td></tr>
<tr><td colspan="2">Test Case 2: Boundary / Minimum Input</td></tr>
<tr><td>Menu option 1 (Add)
Neha Rao, 9999999999 (exactly 10 digits)</td><td>Contact added: Neha Rao | 9999999999</td></tr>
<tr><td colspan="2">Test Case 3: Invalid Input</td></tr>
<tr><td>(a) Add: Amit, 98765
(b) Add: Amit, 9876543210</td><td>(a) Invalid phone number. Enter exactly 10 digits.
(b) Contact with this number already exists.</td></tr>
<tr><td colspan="2">Test Case 4: Different Valid Scenario</td></tr>
<tr><td>(a) Option 3 (Search): Ravi
(b) Option 4 (Update): Ravi Sharma, new phone 9123456780
(c) Option 5 (Delete): Neha Rao</td><td>(a) Ravi Sharma | 9876543210 | ravi@mail.com
(b) Phone updated: 9123456780
(c) Contact 'Neha Rao' deleted. Total contacts: 1</td></tr>
</table>
</td></tr>
<tr><td>Bare Minimum Requirements</td><td>1. Write the solution in Python 3 (one .py file) and run it from the console.
2. Store contacts as a list of dictionaries (name, phone, email).
3. Write one function for each CRUD operation.
4. Validate the phone number and prevent duplicates.
5. Provide a loop-driven menu with an Exit option.</td></tr>
<tr><td>Instruction</td><td>1. Understand the problem statement and define the problem clearly.
2. Develop the solution using proper logic and programming concepts.
3. Implement all required features and pass the given test cases.
4. Test and debug the project to ensure correct output.
5. Prepare the documentation/report and upload the code and report to GitHub.
6. Complete and submit the project within the given deadline.</td></tr>
</table>

BACHELOR OF COMPUTER APPLICATION

<table>
<tr><td>PS No.</td><td>7</td></tr>
<tr><td>Project Title</td><td>College Attendance Analyzer</td></tr>
<tr><td>Problem Statement</td><td>Analyse student attendance from a CSV file to calculate attendance percentage, find students below the 75% requirement and show attendance patterns.
Language: Python 3 Concepts: Pandas, CSV, Matplotlib, Filtering
Note: CSV columns: Name, Subject, Classes_Held, Classes_Attended. Minimum required attendance is 75%.</td></tr>
<tr><td>Test Cases / Case Statements (4 cases)</td><td>
<table>
<tr><th>INPUT</th><th>EXPECTED OUTPUT</th></tr>
<tr><td colspan="2">Test Case 1: Normal Input</td></tr>
<tr><td>attendance.csv:
Aman,Python,40,36
Riya,Python,40,28
Karan,Python,40,30
Menu option 1 (Percentage)</td><td>Aman: 90.00% | Riya: 70.00% | Karan: 75.00%</td></tr>
<tr><td colspan="2">Test Case 2: Boundary / Minimum Input</td></tr>
<tr><td>attendance.csv:
Karan,Python,40,30
Menu option 2 (Shortage list)</td><td>No student below 75%.
Karan (75.00%) is eligible.</td></tr>
<tr><td colspan="2">Test Case 3: Invalid Input</td></tr>
<tr><td>attendance.csv:
Aman,Python,40,45</td><td>Row 1 invalid: attended (45) cannot exceed held (40). Row skipped.</td></tr>
<tr><td colspan="2">Test Case 4: Different Valid Scenario</td></tr>
<tr><td>Test Case 1 rows plus: Aman,Maths,50,45
Menu option 3 (Subject-wise average)</td><td>Python: 78.33% | Maths: 90.00%
Bar chart displayed</td></tr>
</table>
</td></tr>
<tr><td>Bare Minimum Requirements</td><td>1. Write the solution in Python 3 (one .py file) and run it from the console.
2. Load the attendance CSV file using Pandas.
3. Calculate the attendance percentage of every student.
4. Filter and display students below 75% using boolean filtering.
5. Plot at least two Matplotlib charts (bar and pie).</td></tr>
<tr><td>Instruction</td><td>1. Understand the problem statement and define the problem clearly.
2. Develop the solution using proper logic and programming concepts.
3. Implement all required features and pass the given test cases.
4. Test and debug the project to ensure correct output.
5. Prepare the documentation/report and upload the code and report to GitHub.
6. Complete and submit the project within the given deadline.</td></tr>
</table>

BACHELOR OF COMPUTER APPLICATION

<table>
<tr><td>PS No.</td><td>8</td></tr>
<tr><td>Project Title</td><td>Password Strength Checker</td></tr>
<tr><td>Problem Statement</td><td>Build a program that checks a password against security rules and classifies it as Weak, Medium or Strong, giving suggestions to improve it.
Language: Python 3 Concepts: Strings, Sets, Conditions, Regex
Note: One point per rule: length >= 8, uppercase, lowercase, digit, special character (@#$%&*!). Score 0-2 Weak, 3-4 Medium, 5 Strong.</td></tr>
<tr><td>Test Cases / Case Statements (4 cases)</td><td>
<table>
<tr><th>INPUT</th><th>EXPECTED OUTPUT</th></tr>
<tr><td colspan="2">Test Case 1: Normal Input</td></tr>
<tr><td>Password: abc123</td><td>Strength: Weak (2/5)
Suggestions: use at least 8 characters, add an uppercase letter, add a special character</td></tr>
<tr><td colspan="2">Test Case 2: Boundary / Minimum Input</td></tr>
<tr><td>Password: Hello123</td><td>Strength: Medium (4/5)
Suggestion: add a special character</td></tr>
<tr><td colspan="2">Test Case 3: Invalid Input</td></tr>
<tr><td>(a) Password: (empty)
(b) Password: Hello 123</td><td>(a) Password cannot be empty.
(b) Password must not contain spaces.</td></tr>
<tr><td colspan="2">Test Case 4: Different Valid Scenario</td></tr>
<tr><td>Password: Hello@123Xy</td><td>Strength: Strong (5/5)
No suggestions. Your password is secure.</td></tr>
</table>
</td></tr>
<tr><td>Bare Minimum Requirements</td><td>1. Write the solution in Python 3 (one .py file) and run it from the console.
2. Check length, uppercase, lowercase, digit and special character.
3. Use the re module for at least one pattern check.
4. Classify the password as Weak, Medium or Strong using conditions.
5. Show which rules failed, with suggestions.</td></tr>
<tr><td>Instruction</td><td>1. Understand the problem statement and define the problem clearly.
2. Develop the solution using proper logic and programming concepts.
3. Implement all required features and pass the given test cases.
4. Test and debug the project to ensure correct output.
5. Prepare the documentation/report and upload the code and report to GitHub.
6. Complete and submit the project within the given deadline.</td></tr>
</table>

BACHELOR OF COMPUTER APPLICATION

<table>
<tr><td>PS No.</td><td>9</td></tr>
<tr><td>Project Title</td><td>Employee Salary Calculator</td></tr>
<tr><td>Problem Statement</td><td>Create a program that calculates an employee's net salary from basic pay, allowances (HRA, DA) and deductions (PF, tax) and prints a salary slip.
Language: Python 3 Concepts: Functions, Operators, Conditions
Note: HRA 20% and DA 10% of basic; PF 12% of basic. Tax on gross: up to 25000 none, 25001-50000 5%, above 50000 10%.</td></tr>
<tr><td>Test Cases / Case Statements (4 cases)</td><td>
<table>
<tr><th>INPUT</th><th>EXPECTED OUTPUT</th></tr>
<tr><td colspan="2">Test Case 1: Normal Input</td></tr>
<tr><td>Basic salary: 30000</td><td>HRA: 6000 | DA: 3000 | Gross: 39000
PF: 3600 | Tax: 1950 | Net Salary: 33450</td></tr>
<tr><td colspan="2">Test Case 2: Boundary / Minimum Input</td></tr>
<tr><td>Basic salary: 10000 (no tax slab)</td><td>HRA: 2000 | DA: 1000 | Gross: 13000
PF: 1200 | Tax: 0 | Net Salary: 11800</td></tr>
<tr><td colspan="2">Test Case 3: Invalid Input</td></tr>
<tr><td>Basic salary: -5000, then abc</td><td>Invalid salary. Enter a positive number.</td></tr>
<tr><td colspan="2">Test Case 4: Different Valid Scenario</td></tr>
<tr><td>Basic salary: 50000</td><td>HRA: 10000 | DA: 5000 | Gross: 65000
PF: 6000 | Tax: 6500 | Net Salary: 52500</td></tr>
</table>
</td></tr>
<tr><td>Bare Minimum Requirements</td><td>1. Write the solution in Python 3 (one .py file) and run it from the console.
2. Take employee name, ID and basic salary as input.
3. Calculate HRA, DA, PF and tax using operators and conditions.
4. Create functions such as calculate_gross() and print_slip().
5. Print a neat salary slip and allow more employees using a loop.</td></tr>
<tr><td>Instruction</td><td>1. Understand the problem statement and define the problem clearly.
2. Develop the solution using proper logic and programming concepts.
3. Implement all required features and pass the given test cases.
4. Test and debug the project to ensure correct output.
5. Prepare the documentation/report and upload the code and report to GitHub.
6. Complete and submit the project within the given deadline.</td></tr>
</table>

BACHELOR OF COMPUTER APPLICATION

<table>
<tr><td>PS No.</td><td>10</td></tr>
<tr><td>Project Title</td><td>Shopping Cart System</td></tr>
<tr><td>Problem Statement</td><td>Develop a console shopping cart where the user can view products, add or remove items and generate a final bill with discount and GST.
Language: Python 3 Concepts: Lists, Dictionaries, Functions, Billing
Note: Products: Pen Rs.10, Notebook Rs.50, Bag Rs.500. GST 18%. 10% discount if subtotal is more than 500.</td></tr>
<tr><td>Test Cases / Case Statements (4 cases)</td><td>
<table>
<tr><th>INPUT</th><th>EXPECTED OUTPUT</th></tr>
<tr><td colspan="2">Test Case 1: Normal Input</td></tr>
<tr><td>Add to cart: Notebook, Quantity: 2</td><td>Notebook x2 added | Cart subtotal: 100</td></tr>
<tr><td colspan="2">Test Case 2: Boundary / Minimum Input</td></tr>
<tr><td>Add to cart: Bag, Quantity: 1
Checkout</td><td>Subtotal: 500 | Discount: 0 | GST: 90
Grand Total: 590</td></tr>
<tr><td colspan="2">Test Case 3: Invalid Input</td></tr>
<tr><td>(a) Add to cart: Laptop
(b) Add to cart: Pen, Quantity: -1</td><td>(a) Invalid product. Available: Pen, Notebook, Bag
(b) Quantity must be at least 1.</td></tr>
<tr><td colspan="2">Test Case 4: Different Valid Scenario</td></tr>
<tr><td>(a) Add Bag x2
(b) Add Pen x5
(c) Remove Pen
(d) Checkout</td><td>(c) Pen removed. Cart: Bag x2 = 1000
(d) Subtotal: 1000 | Discount: 100 | GST: 162
Grand Total: 1062</td></tr>
</table>
</td></tr>
<tr><td>Bare Minimum Requirements</td><td>1. Write the solution in Python 3 (one .py file) and run it from the console.
2. Keep the product catalogue in a dictionary.
3. Add, remove and view cart items using functions.
4. Calculate subtotal, discount, GST and grand total.
5. Validate the product name and quantity.</td></tr>
<tr><td>Instruction</td><td>1. Understand the problem statement and define the problem clearly.
2. Develop the solution using proper logic and programming concepts.
3. Implement all required features and pass the given test cases.
4. Test and debug the project to ensure correct output.
5. Prepare the documentation/report and upload the code and report to GitHub.
6. Complete and submit the project within the given deadline.</td></tr>
</table>

BACHELOR OF COMPUTER APPLICATION

<table>
<tr><td>PS No.</td><td>11</td></tr>
<tr><td>Project Title</td><td>Temperature Data Analyzer</td></tr>
<tr><td>Problem Statement</td><td>Analyse temperature readings over a period using NumPy and Pandas to find statistics, detect extreme values and plot trends.
Language: Python 3 Concepts: NumPy, Pandas, Matplotlib, Statistics
Note: CSV columns: Date, Temp_C. A hot day is a day with temperature above 35 C.</td></tr>
<tr><td>Test Cases / Case Statements (4 cases)</td><td>
<table>
<tr><th>INPUT</th><th>EXPECTED OUTPUT</th></tr>
<tr><td colspan="2">Test Case 1: Normal Input</td></tr>
<tr><td>temps.csv:
2026-08-01,30
2026-08-02,36
2026-08-03,34
Menu option 1 (Statistics)</td><td>Mean: 33.33 | Median: 34.00 | Max: 36 | Min: 30 | Std Dev: 2.49
Hot days: 2026-08-02</td></tr>
<tr><td colspan="2">Test Case 2: Boundary / Minimum Input</td></tr>
<tr><td>temps.csv: 35, 35, 35 (3 days)
Menu option 1 (Statistics)</td><td>Mean: 35.00 | Std Dev: 0.00
No days above 35 C</td></tr>
<tr><td colspan="2">Test Case 3: Invalid Input</td></tr>
<tr><td>temps.csv:
2026-08-02,hot</td><td>Invalid reading 'hot' on 2026-08-02. Value ignored (NaN).</td></tr>
<tr><td colspan="2">Test Case 4: Different Valid Scenario</td></tr>
<tr><td>Same CSV as Test Case 1
Menu option 3 (Convert to Fahrenheit and plot)</td><td>86.00 F | 96.80 F | 93.20 F
Line chart displayed</td></tr>
</table>
</td></tr>
<tr><td>Bare Minimum Requirements</td><td>1. Write the solution in Python 3 (one .py file) and run it from the console.
2. Load the data with Pandas and handle missing or invalid readings.
3. Calculate mean, median, max, min and standard deviation using NumPy.
4. Detect hot days and outliers.
5. Plot a line chart and a histogram with labels.</td></tr>
<tr><td>Instruction</td><td>1. Understand the problem statement and define the problem clearly.
2. Develop the solution using proper logic and programming concepts.
3. Implement all required features and pass the given test cases.
4. Test and debug the project to ensure correct output.
5. Prepare the documentation/report and upload the code and report to GitHub.
6. Complete and submit the project within the given deadline.</td></tr>
</table>

BACHELOR OF COMPUTER APPLICATION

<table>
<tr><td>PS No.</td><td>12</td></tr>
<tr><td>Project Title</td><td>Simple Banking System</td></tr>
<tr><td>Problem Statement</td><td>Create a banking application that lets customers open an account, deposit, withdraw, transfer money and check their balance, with all data stored in a file.
Language: Python 3 Concepts: Loops, Functions, Conditions, File Handling
Note: Account numbers start from 1001. A minimum balance of Rs.500 must always remain. Data is stored in bank.txt.</td></tr>
<tr><td>Test Cases / Case Statements (4 cases)</td><td>
<table>
<tr><th>INPUT</th><th>EXPECTED OUTPUT</th></tr>
<tr><td colspan="2">Test Case 1: Normal Input</td></tr>
<tr><td>Menu option 1 (Create account)
Name: Anil, Opening deposit: 2000</td><td>Account created. A/C No: 1001 | Balance: 2000</td></tr>
<tr><td colspan="2">Test Case 2: Boundary / Minimum Input</td></tr>
<tr><td>Account 1001 has balance 2000
Withdraw 1500</td><td>Withdrawal successful. Balance: 500</td></tr>
<tr><td colspan="2">Test Case 3: Invalid Input</td></tr>
<tr><td>Account 1001 has balance 2000
Transfer 100 to account 9999</td><td>Account 9999 not found. Transfer cancelled.</td></tr>
<tr><td colspan="2">Test Case 4: Different Valid Scenario</td></tr>
<tr><td>Accounts: 1001 (2000), 1002 (3000)
Transfer 1000 from 1002 to 1001
Balance enquiry: 1001</td><td>Transfer successful.
1002 balance: 2000 | 1001 balance: 3000</td></tr>
</table>
</td></tr>
<tr><td>Bare Minimum Requirements</td><td>1. Write the solution in Python 3 (one .py file) and run it from the console.
2. Create accounts with a unique account number.
3. Implement deposit, withdraw, transfer and balance enquiry using functions.
4. Enforce the minimum balance rule and validate all inputs.
5. Store and load account data using a file.</td></tr>
<tr><td>Instruction</td><td>1. Understand the problem statement and define the problem clearly.
2. Develop the solution using proper logic and programming concepts.
3. Implement all required features and pass the given test cases.
4. Test and debug the project to ensure correct output.
5. Prepare the documentation/report and upload the code and report to GitHub.
6. Complete and submit the project within the given deadline.</td></tr>
</table>

BACHELOR OF COMPUTER APPLICATION

<table>
<tr><td>PS No.</td><td>13</td></tr>
<tr><td>Project Title</td><td>Word Counter & Text Analyzer</td></tr>
<tr><td>Problem Statement</td><td>Build a tool that analyses a piece of text or a text file and reports the number of words, characters, sentences and the frequency of each word.
Language: Python 3 Concepts: Strings, Dictionaries, Sets, Frequency Analysis
Note: Counting is case-insensitive and punctuation is removed before counting words.</td></tr>
<tr><td>Test Cases / Case Statements (4 cases)</td><td>
<table>
<tr><th>INPUT</th><th>EXPECTED OUTPUT</th></tr>
<tr><td colspan="2">Test Case 1: Normal Input</td></tr>
<tr><td>Text: Python is easy. Python is fun.</td><td>Words: 6 | Characters (with spaces): 30 | Sentences: 2</td></tr>
<tr><td colspan="2">Test Case 2: Boundary / Minimum Input</td></tr>
<tr><td>Text: Hello</td><td>Words: 1 | Characters: 5 | Sentences: 1 | Unique words: 1</td></tr>
<tr><td colspan="2">Test Case 3: Invalid Input</td></tr>
<tr><td>(a) Text: (empty)
(b) File name: notes.txt (does not exist)</td><td>(a) No text to analyse.
(b) File 'notes.txt' not found.</td></tr>
<tr><td colspan="2">Test Case 4: Different Valid Scenario</td></tr>
<tr><td>Text: The cat saw the dog. The dog ran.
Menu option 2 (Frequency)</td><td>Top words: the: 3 | dog: 2
Unique words: 5 (cat, dog, ran, saw, the)</td></tr>
</table>
</td></tr>
<tr><td>Bare Minimum Requirements</td><td>1. Write the solution in Python 3 (one .py file) and run it from the console.
2. Read text from the user or from a file.
3. Clean the text using string methods (lower, split, strip, replace).
4. Count word frequency using a dictionary.
5. Find unique words using a set and show results sorted by frequency.</td></tr>
<tr><td>Instruction</td><td>1. Understand the problem statement and define the problem clearly.
2. Develop the solution using proper logic and programming concepts.
3. Implement all required features and pass the given test cases.
4. Test and debug the project to ensure correct output.
5. Prepare the documentation/report and upload the code and report to GitHub.
6. Complete and submit the project within the given deadline.</td></tr>
</table>

BACHELOR OF COMPUTER APPLICATION

<table>
<tr><td>PS No.</td><td>14</td></tr>
<tr><td>Project Title</td><td>Student Record Management System</td></tr>
<tr><td>Problem Statement</td><td>Develop a system to maintain student records (roll number, name, course, marks) with the ability to add, view, search, update and delete records.
Language: Python 3 Concepts: Lists, Tuples, Dictionaries, CRUD
Note: Roll numbers must be unique and marks must be between 0 and 100. Test cases run in order on the same records.</td></tr>
<tr><td>Test Cases / Case Statements (4 cases)</td><td>
<table>
<tr><th>INPUT</th><th>EXPECTED OUTPUT</th></tr>
<tr><td colspan="2">Test Case 1: Normal Input</td></tr>
<tr><td>Menu option 1 (Add)
101, Aman, BTech, 78</td><td>Record added: 101 | Aman | BTech | 78</td></tr>
<tr><td colspan="2">Test Case 2: Boundary / Minimum Input</td></tr>
<tr><td>Menu option 1 (Add)
102, Riya, BTech, 100 (maximum marks)</td><td>Record added: 102 | Riya | BTech | 100</td></tr>
<tr><td colspan="2">Test Case 3: Invalid Input</td></tr>
<tr><td>(a) Add: 101, Neha, BTech, 70
(b) Add: 103, Ravi, BTech, 150</td><td>(a) Roll number 101 already exists.
(b) Marks must be between 0 and 100.</td></tr>
<tr><td colspan="2">Test Case 4: Different Valid Scenario</td></tr>
<tr><td>(a) Search: 101
(b) Update marks of 101 to 85
(c) Delete: 999</td><td>(a) 101 | Aman | BTech | 78
(b) Marks updated: 85
(c) Roll number 999 not found.</td></tr>
</table>
</td></tr>
<tr><td>Bare Minimum Requirements</td><td>1. Write the solution in Python 3 (one .py file) and run it from the console.
2. Store records using lists, tuples and dictionaries.
3. Write a separate function for each CRUD operation.
4. Ensure unique roll numbers and valid marks.
5. Display records in a neat table format.</td></tr>
<tr><td>Instruction</td><td>1. Understand the problem statement and define the problem clearly.
2. Develop the solution using proper logic and programming concepts.
3. Implement all required features and pass the given test cases.
4. Test and debug the project to ensure correct output.
5. Prepare the documentation/report and upload the code and report to GitHub.
6. Complete and submit the project within the given deadline.</td></tr>
</table>

BACHELOR OF COMPUTER APPLICATION

<table>
<tr><td>PS No.</td><td>15</td></tr>
<tr><td>Project Title</td><td>Unit Conversion System</td></tr>
<tr><td>Problem Statement</td><td>Create a converter that converts values between units of length, weight, temperature and time through a simple menu.
Language: Python 3 Concepts: Functions, Operators, Conditions
Note: Menu: 1 Length, 2 Weight, 3 Temperature, 4 Time. Results are rounded to 2 decimal places. 1 km = 0.621371 miles.</td></tr>
<tr><td>Test Cases / Case Statements (4 cases)</td><td>
<table>
<tr><th>INPUT</th><th>EXPECTED OUTPUT</th></tr>
<tr><td colspan="2">Test Case 1: Normal Input</td></tr>
<tr><td>Option 1 (Length): 5 km to miles</td><td>5 km = 3.11 miles</td></tr>
<tr><td colspan="2">Test Case 2: Boundary / Minimum Input</td></tr>
<tr><td>Option 3 (Temperature): 0 C to F</td><td>0 C = 32.00 F</td></tr>
<tr><td colspan="2">Test Case 3: Invalid Input</td></tr>
<tr><td>(a) Menu choice: 7
(b) Value: abc</td><td>(a) Invalid choice. Select 1 to 4.
(b) Invalid value. Enter a number.</td></tr>
<tr><td colspan="2">Test Case 4: Different Valid Scenario</td></tr>
<tr><td>(a) Weight: 2 kg to g
(b) Temperature: 100 C to F
(c) Time: 2 hours to minutes</td><td>(a) 2000.00 g
(b) 212.00 F
(c) 120.00 minutes</td></tr>
</table>
</td></tr>
<tr><td>Bare Minimum Requirements</td><td>1. Write the solution in Python 3 (one .py file) and run it from the console.
2. Provide a menu of categories and units.
3. Write one function for each conversion type.
4. Use operators and if-elif-else for the formulas.
5. Validate the menu choice and the value entered.</td></tr>
<tr><td>Instruction</td><td>1. Understand the problem statement and define the problem clearly.
2. Develop the solution using proper logic and programming concepts.
3. Implement all required features and pass the given test cases.
4. Test and debug the project to ensure correct output.
5. Prepare the documentation/report and upload the code and report to GitHub.
6. Complete and submit the project within the given deadline.</td></tr>
</table>

BACHELOR OF COMPUTER APPLICATION

<table>
<tr><td>PS No.</td><td>16</td></tr>
<tr><td>Project Title</td><td>Hospital Patient Data Analyzer</td></tr>
<tr><td>Problem Statement</td><td>Analyse hospital patient data from a CSV file to clean the data and find insights such as department-wise patient load, average length of stay and age-group distribution.
Language: Python 3 Concepts: Pandas, CSV, Data Cleaning, Grouping
Note: CSV columns: Patient_ID, Age, Gender, Department, Admit_Date, Discharge_Date. Use dummy data only.</td></tr>
<tr><td>Test Cases / Case Statements (4 cases)</td><td>
<table>
<tr><th>INPUT</th><th>EXPECTED OUTPUT</th></tr>
<tr><td colspan="2">Test Case 1: Normal Input</td></tr>
<tr><td>patients.csv:
P1,45,M,Cardiology,2026-08-01,2026-08-05
P2,30,F,Ortho,2026-08-02,2026-08-04
P3,60,M,Cardiology,2026-08-03,2026-08-08
Menu option 1 (Department count)</td><td>Cardiology: 2 | Ortho: 1
Total patients: 3</td></tr>
<tr><td colspan="2">Test Case 2: Boundary / Minimum Input</td></tr>
<tr><td>patients.csv:
P4,25,F,ENT,2026-08-10,2026-08-10 (same-day discharge)
Menu option 2 (Length of stay)</td><td>P4: 0 days
Average stay: 0.00 days</td></tr>
<tr><td colspan="2">Test Case 3: Invalid Input</td></tr>
<tr><td>Test Case 1 rows plus:
P1 repeated (duplicate)
P6,,M,ENT,2026-08-06,2026-08-07 (age missing)</td><td>Duplicates removed: 1
Missing Age filled with median: 45</td></tr>
<tr><td colspan="2">Test Case 4: Different Valid Scenario</td></tr>
<tr><td>Same CSV as Test Case 1
Menu option 3 (Stay and age groups)</td><td>Average stay: 3.67 days
Age groups: 0-40: 1 | 41-60: 2 | 61+: 0
Bar chart displayed</td></tr>
</table>
</td></tr>
<tr><td>Bare Minimum Requirements</td><td>1. Write the solution in Python 3 (one .py file) and run it from the console.
2. Load the CSV file using Pandas.
3. Clean missing, duplicate and invalid values.
4. Use groupby() for department-wise and gender-wise analysis.
5. Calculate the length of stay and plot at least one Matplotlib chart.</td></tr>
<tr><td>Instruction</td><td>1. Understand the problem statement and define the problem clearly.
2. Develop the solution using proper logic and programming concepts.
3. Implement all required features and pass the given test cases.
4. Test and debug the project to ensure correct output.
5. Prepare the documentation/report and upload the code and report to GitHub.
6. Complete and submit the project within the given deadline.</td></tr>
</table>

BACHELOR OF COMPUTER APPLICATION

<table>
<tr><td>PS No.</td><td>17</td></tr>
<tr><td>Project Title</td><td>Inventory Management System</td></tr>
<tr><td>Problem Statement</td><td>Build a console inventory system to add, view, update, search and delete items with quantity and price, and to show the total stock value and a low-stock report.
Language: Python 3 Concepts: Dictionaries, Lists, Functions, CRUD
Note: Each item is a dictionary (id, name, quantity, price). The first item ID is I101. Low stock means quantity below 5. Test cases run in order on the same inventory.</td></tr>
<tr><td>Test Cases / Case Statements (4 cases)</td><td>
<table>
<tr><th>INPUT</th><th>EXPECTED OUTPUT</th></tr>
<tr><td colspan="2">Test Case 1: Normal Input</td></tr>
<tr><td>Menu option 1 (Add)
Name: Pen
Quantity: 50
Price: 10</td><td>Item added.
ID: I101 │ Pen │ Qty: 50 │ Price: 10</td></tr>
<tr><td colspan="2">Test Case 2: Boundary / Minimum Input</td></tr>
<tr><td>Menu option 1 (Add)
Name: Eraser
Quantity: 0 (minimum)
Price: 5</td><td>Item added. ID: I102 │ Eraser │ Qty: 0 │ Price: 5
Low stock alert: Eraser (0)</td></tr>
<tr><td colspan="2">Test Case 3: Invalid Input</td></tr>
<tr><td>(a) Add: Pencil, Quantity: -5, Price: 4
(b) Menu option 3 (Update): ID I999</td><td>(a) Invalid quantity. Quantity must be a non-negative whole number. Item not added.
(b) Item ID I999 not found.</td></tr>
<tr><td colspan="2">Test Case 4: Different Valid Scenario</td></tr>
<tr><td>Inventory: I101 Pen (50 × 10), I102 Notebook (3 × 40)
(a) Option 5 (Total stock value)
(b) Option 6 (Low stock report)
(c) Option 4 (Delete): ID I101</td><td>(a) Pen: 500 │ Notebook: 120
Total stock value: 620
(b) Low stock (below 5): I102 │ Notebook │ Qty: 3
(c) Item 'Pen' deleted. Items left: 1</td></tr>
</table>
</td></tr>
<tr><td>Bare Minimum Requirements</td><td>1. Write the solution in Python 3 (one .py file) and run it from the console.
2. Store each item as a dictionary inside a list, with a unique item ID.
3. Write one function each for add, view, update, search and delete.
4. Calculate the total stock value and list items with quantity below 5.
5. Validate the quantity and price, and show records in a neat table format.</td></tr>
<tr><td>Instruction</td><td>1. Understand the problem statement and define the problem clearly.
2. Develop the solution using proper logic and programming concepts.
3. Implement all required features and pass the given test cases.
4. Test and debug the project to ensure correct output.
5. Prepare the documentation/report and upload the code and report to GitHub.
6. Complete and submit the project within the given deadline.</td></tr>
</table>

BACHELOR OF COMPUTER APPLICATION

<table>
<tr><td>PS No.</td><td>18</td></tr>
<tr><td>Project Title</td><td>Student Grade Calculator</td></tr>
<tr><td>Problem Statement</td><td>Create a program that takes a student's marks in five subjects, calculates the total, percentage, grade and pass/fail result, and allows more students to be entered through a loop.
Language: Python 3 Concepts: Variables, Conditions, Functions
Note: Marks are out of 100 in five subjects. Grades: >=85 A, >=70 B, >=55 C, >=40 D, below 40 F. A student passes with 40% or more.</td></tr>
<tr><td>Test Cases / Case Statements (4 cases)</td><td>
<table>
<tr><th>INPUT</th><th>EXPECTED OUTPUT</th></tr>
<tr><td colspan="2">Test Case 1: Normal Input</td></tr>
<tr><td>Name: Aman
Marks: 80, 70, 90, 60, 50</td><td>Aman | Total: 350 | Percentage: 70.00% | Grade: B | Result: Pass</td></tr>
<tr><td colspan="2">Test Case 2: Boundary / Minimum Input</td></tr>
<tr><td>Name: Neha
Marks: 40, 40, 40, 40, 40 (pass limit)</td><td>Neha | Total: 200 | Percentage: 40.00% | Grade: D | Result: Pass</td></tr>
<tr><td colspan="2">Test Case 3: Invalid Input</td></tr>
<tr><td>Name: Ravi
Marks for subject 2: 105, then abc, then 75</td><td>Invalid marks. Enter a number between 0 and 100.
Invalid marks. Enter a number between 0 and 100.
(Second subject is accepted as 75)</td></tr>
<tr><td colspan="2">Test Case 4: Different Valid Scenario</td></tr>
<tr><td>(a) Name: Riya; Marks: 95, 88, 92, 90, 85
Another student? y
(b) Name: Karan; Marks: 30, 35, 40, 25, 20</td><td>(a) Riya | Total: 450 | Percentage: 90.00% | Grade: A | Result: Pass
(b) Karan | Total: 150 | Percentage: 30.00% | Grade: F | Result: Fail</td></tr>
</table>
</td></tr>
<tr><td>Bare Minimum Requirements</td><td>1. Write the solution in Python 3 (one .py file) and run it from the console.
2. Take the student name and the marks of five subjects as input.
3. Calculate the total and percentage using functions such as calculate_total() and calculate_percentage().
4. Use if-elif-else to find the grade and the pass/fail result.
5. Validate the marks (numbers from 0 to 100) and allow more students using a loop.</td></tr>
<tr><td>Instruction</td><td>1. Understand the problem statement and define the problem clearly.
2. Develop the solution using proper logic and programming concepts.
3. Implement all required features and pass the given test cases.
4. Test and debug the project to ensure correct output.
5. Prepare the documentation/report and upload the code and report to GitHub.
6. Complete and submit the project within the given deadline.</td></tr>
</table>

BACHELOR OF COMPUTER APPLICATION

<table>
<tr><td>PS No.</td><td>19</td></tr>
<tr><td>Project Title</td><td>Sports Performance Analyzer</td></tr>
<tr><td>Problem Statement</td><td>Build a program that loads players' match scores from a CSV file, cleans the data and analyses it to find each player's average and best score, the top performer and overall statistics, with charts.
Language: Python 3 Concepts: NumPy, Pandas, Matplotlib, Statistics
Note: CSV columns: Player, Match, Score. Standard deviation is the population value (np.std default). Test cases use the small CSV rows given in each INPUT.</td></tr>
<tr><td>Test Cases / Case Statements (4 cases)</td><td>
<table>
<tr><th>INPUT</th><th>EXPECTED OUTPUT</th></tr>
<tr><td colspan="2">Test Case 1: Normal Input</td></tr>
<tr><td>scores.csv:
Aman,1,40
Aman,2,60
Riya,1,30
Riya,2,50
Menu option 1 (Player stats)</td><td>Aman: Avg 50.00 | Best 60 | Total 100
Riya: Avg 40.00 | Best 50 | Total 80
Top performer: Aman (Avg 50.00)</td></tr>
<tr><td colspan="2">Test Case 2: Boundary / Minimum Input</td></tr>
<tr><td>scores.csv: 1 row only
Karan,1,25
Menu option 1 (Player stats)</td><td>Karan: Avg 25.00 | Best 25 | Std Dev: 0.00
Top performer: Karan</td></tr>
<tr><td colspan="2">Test Case 3: Invalid Input</td></tr>
<tr><td>scores.csv:
Aman,1,abc
Riya,1,-10</td><td>Row 1 skipped: invalid Score 'abc'.
Row 2 skipped: Score cannot be negative.
No valid data to analyse.</td></tr>
<tr><td colspan="2">Test Case 4: Different Valid Scenario</td></tr>
<tr><td>Same CSV as Test Case 1
Menu option 2 (Statistics and trend)</td><td>Mean: 45.00 | Median: 45.00 | Std Dev: 11.18 | Max: 60 (Aman, Match 2)
Match-wise average: Match 1: 35.00 | Match 2: 55.00
Line chart 'Score Trend' displayed</td></tr>
</table>
</td></tr>
<tr><td>Bare Minimum Requirements</td><td>1. Write the solution in Python 3 (one .py file) and run it from the console.
2. Load the CSV file using Pandas and clean invalid or missing scores.
3. Use NumPy to calculate mean, median, maximum, minimum and standard deviation.
4. Use groupby() to find each player's average and best score, and show the top performer.
5. Plot at least two Matplotlib charts (line and bar) with title and labels.</td></tr>
<tr><td>Instruction</td><td>1. Understand the problem statement and define the problem clearly.
2. Develop the solution using proper logic and programming concepts.
3. Implement all required features and pass the given test cases.
4. Test and debug the project to ensure correct output.
5. Prepare the documentation/report and upload the code and report to GitHub.
6. Complete and submit the project within the given deadline.</td></tr>
</table>

BACHELOR OF COMPUTER APPLICATION

<table>
<tr><td>PS No.</td><td>20</td></tr>
<tr><td>Project Title</td><td>To-Do List Application</td></tr>
<tr><td>Problem Statement</td><td>Develop a menu-driven to-do list where a user can add tasks, view all tasks, mark a task as completed and delete a task. Tasks are saved to a file after every change.
Language: Python 3 Concepts: Lists, Functions, Loops, File Handling
Note: Tasks are stored in todo.txt. Task numbers start from 1 and every new task starts as Pending. Test cases run in order on the same task list.</td></tr>
<tr><td>Test Cases / Case Statements (4 cases)</td><td>
<table>
<tr><th>INPUT</th><th>EXPECTED OUTPUT</th></tr>
<tr><td colspan="2">Test Case 1: Normal Input</td></tr>
<tr><td>Menu option 1 (Add)
Task: Buy books</td><td>Task added. 1 │ Buy books │ Pending</td></tr>
<tr><td colspan="2">Test Case 2: Boundary / Minimum Input</td></tr>
<tr><td>Empty list (todo.txt has no tasks)
Menu option 2 (View)</td><td>No tasks found. Your to-do list is empty.</td></tr>
<tr><td colspan="2">Test Case 3: Invalid Input</td></tr>
<tr><td>(a) Menu option 3 (Complete): task number 5 (only 1 task exists)
(b) Menu option 1 (Add): Task: (empty)</td><td>(a) Task number 5 not found.
(b) Task title cannot be empty.</td></tr>
<tr><td colspan="2">Test Case 4: Different Valid Scenario</td></tr>
<tr><td>Tasks: 1 Buy books, 2 Pay bill
(a) Option 3 (Complete): 1
(b) Option 2 (View)
(c) Option 4 (Delete): 2</td><td>(a) Task 1 marked as completed.
(b) 1 │ Buy books │ Done
2 │ Pay bill │ Pending
(c) Task 2 deleted. Tasks left: 1</td></tr>
</table>
</td></tr>
<tr><td>Bare Minimum Requirements</td><td>1. Write the solution in Python 3 (one .py file) and run it from the console.
2. Store the tasks in a list and use a loop-driven menu with an Exit option.
3. Write one function each for add, view, complete and delete.
4. Load the tasks from todo.txt at start and save them after every change.
5. Validate the task title and the task number, and handle a missing file safely.</td></tr>
<tr><td>Instruction</td><td>1. Understand the problem statement and define the problem clearly.
2. Develop the solution using proper logic and programming concepts.
3. Implement all required features and pass the given test cases.
4. Test and debug the project to ensure correct output.
5. Prepare the documentation/report and upload the code and report to GitHub.
6. Complete and submit the project within the given deadline.</td></tr>
</table>

BACHELOR OF COMPUTER APPLICATION

<table>
<tr><td>PS No.</td><td>21</td></tr>
<tr><td>Project Title</td><td>Electricity Consumption Analyzer</td></tr>
<tr><td>Problem Statement</td><td>Analyse monthly electricity consumption from a CSV file using NumPy and Pandas to find total and average usage, the highest and lowest months and months above average, and visualise the trend.
Language: Python 3 Concepts: NumPy, Pandas, Visualization, Statistics
Note: CSV columns: Month, Units. Standard deviation is the population value (np.std default). Test cases use the small CSV rows given in each INPUT.</td></tr>
<tr><td>Test Cases / Case Statements (4 cases)</td><td>
<table>
<tr><th>INPUT</th><th>EXPECTED OUTPUT</th></tr>
<tr><td colspan="2">Test Case 1: Normal Input</td></tr>
<tr><td>usage.csv:
Jan,200
Feb,250
Mar,300
Menu option 1 (Summary)</td><td>Total: 750 units │ Average: 250.00
Highest: Mar (300) │ Lowest: Jan (200)
Std Dev: 40.82</td></tr>
<tr><td colspan="2">Test Case 2: Boundary / Minimum Input</td></tr>
<tr><td>usage.csv: 1 row only
Jan,150
Menu option 1 (Summary)</td><td>Total: 150 units │ Average: 150.00
Highest: Jan (150) │ Lowest: Jan (150)
Std Dev: 0.00</td></tr>
<tr><td colspan="2">Test Case 3: Invalid Input</td></tr>
<tr><td>usage.csv:
Jan,200
Feb,abc
Mar,-30</td><td>Row 2 skipped: invalid Units 'abc'.
Row 3 skipped: Units cannot be negative.
Valid rows: 1 │ Total: 200 units</td></tr>
<tr><td colspan="2">Test Case 4: Different Valid Scenario</td></tr>
<tr><td>Same CSV as Test Case 1
Menu option 2 (Above-average months and chart)</td><td>Months above average (250.00): Mar (300)
Bar chart 'Monthly Consumption' displayed</td></tr>
</table>
</td></tr>
<tr><td>Bare Minimum Requirements</td><td>1. Write the solution in Python 3 (one .py file) and run it from the console.
2. Load the CSV file using Pandas and clean invalid or missing readings.
3. Use NumPy to calculate total, mean, maximum, minimum and standard deviation.
4. Find the highest, lowest and above-average months using boolean filtering.
5. Plot at least two Matplotlib charts (bar and line) with title and labels.</td></tr>
<tr><td>Instruction</td><td>1. Understand the problem statement and define the problem clearly.
2. Develop the solution using proper logic and programming concepts.
3. Implement all required features and pass the given test cases.
4. Test and debug the project to ensure correct output.
5. Prepare the documentation/report and upload the code and report to GitHub.
6. Complete and submit the project within the given deadline.</td></tr>
</table>

BACHELOR OF COMPUTER APPLICATION

<table>
<tr><td>PS No.</td><td>22</td></tr>
<tr><td>Project Title</td><td>Placement Data Analyzer</td></tr>
<tr><td>Problem Statement</td><td>Analyse campus placement data from a CSV file to find the number of placed students, average and highest package, branch-wise placements and students above a package limit, and present the results with charts.
Language: Python 3 Concepts: Pandas, CSV, Matplotlib, Filtering
Note: CSV columns: Student, Branch, Company, Package_LPA (placed students only, use dummy data). Packages are in lakhs per annum (LPA).</td></tr>
<tr><td>Test Cases / Case Statements (4 cases)</td><td>
<table>
<tr><th>INPUT</th><th>EXPECTED OUTPUT</th></tr>
<tr><td colspan="2">Test Case 1: Normal Input</td></tr>
<tr><td>placement.csv:
Aman,CSE,TCS,6
Riya,CSE,Infosys,8
Karan,ECE,Wipro,4
Menu option 1 (Summary)</td><td>Total placed: 3 │ Average package: 6.00 LPA
Highest: 8.00 LPA (Riya, Infosys)</td></tr>
<tr><td colspan="2">Test Case 2: Boundary / Minimum Input</td></tr>
<tr><td>placement.csv: 1 row only
Neha,CSE,TCS,3.5
Menu option 1 (Summary)</td><td>Total placed: 1 │ Average package: 3.50 LPA
Highest: 3.50 LPA (Neha, TCS)</td></tr>
<tr><td colspan="2">Test Case 3: Invalid Input</td></tr>
<tr><td>placement.csv:
Aman,CSE,TCS,abc
Riya,CSE,Infosys,8
Karan,ECE,Wipro,-4</td><td>Row 1 skipped: invalid Package 'abc'.
Row 3 skipped: Package must be positive.
Valid records: 1 │ Total placed: 1
Average package: 8.00 LPA</td></tr>
<tr><td colspan="2">Test Case 4: Different Valid Scenario</td></tr>
<tr><td>Test Case 1 rows plus:
Ravi,ECE,Accenture,12
Menu option 2 (Branch-wise and filter above 7 LPA)</td><td>Placements: CSE: 2 │ ECE: 2
Average package: CSE 7.00 │ ECE 8.00
Above 7 LPA: Riya (8.00) │ Ravi (12.00)
Pie chart 'Branch-wise Placements' displayed</td></tr>
</table>
</td></tr>
<tr><td>Bare Minimum Requirements</td><td>1. Write the solution in Python 3 (one .py file) and run it from the console.
2. Load the placement CSV file using Pandas and clean invalid or duplicate rows.
3. Calculate the total placed students, average package and highest package.
4. Use boolean filtering to list students above a given package, and groupby() for branch-wise analysis.
5. Plot at least two Matplotlib charts (bar and pie) with title and labels.</td></tr>
<tr><td>Instruction</td><td>1. Understand the problem statement and define the problem clearly.
2. Develop the solution using proper logic and programming concepts.
3. Implement all required features and pass the given test cases.
4. Test and debug the project to ensure correct output.
5. Prepare the documentation/report and upload the code and report to GitHub.
6. Complete and submit the project within the given deadline.</td></tr>
</table>

BACHELOR OF COMPUTER APPLICATION

<table>
<tr><td>PS No.</td><td>23</td></tr>
<tr><td>Project Title</td><td>Voting Eligibility System</td></tr>
<tr><td>Problem Statement</td><td>Write a program that takes a person's name, age and citizenship, checks whether the person is eligible to vote, gives the reason when not eligible, and allows more voters to be checked in a loop.
Language: Python 3 Concepts: Conditions, Functions, Exceptions
Note: A person is eligible if age is 18 or more and the person is an Indian citizen (y/n). A valid age is a whole number from 0 to 120.</td></tr>
<tr><td>Test Cases / Case Statements (4 cases)</td><td>
<table>
<tr><th>INPUT</th><th>EXPECTED OUTPUT</th></tr>
<tr><td colspan="2">Test Case 1: Normal Input</td></tr>
<tr><td>Name: Aman
Age: 25
Indian citizen (y/n): y</td><td>Aman is eligible to vote.</td></tr>
<tr><td colspan="2">Test Case 2: Boundary / Minimum Input</td></tr>
<tr><td>(a) Name: Neha; Age: 18; Citizen: y
Check another voter? y
(b) Name: Ravi; Age: 17; Citizen: y</td><td>(a) Neha is eligible to vote.
(b) Ravi is not eligible to vote.
Years left to become eligible: 1</td></tr>
<tr><td colspan="2">Test Case 3: Invalid Input</td></tr>
<tr><td>Name: Amit
Age: abc, then -5, then 30
Citizen: y</td><td>Invalid input. Enter a whole number.
Invalid age. Age must be between 0 and 120.
Amit is eligible to vote.</td></tr>
<tr><td colspan="2">Test Case 4: Different Valid Scenario</td></tr>
<tr><td>Name: Riya
Age: 30
Indian citizen (y/n): n</td><td>Riya is not eligible to vote.
Reason: Not an Indian citizen.
Voters checked: 1 | Eligible: 0 | Not eligible: 1</td></tr>
</table>
</td></tr>
<tr><td>Bare Minimum Requirements</td><td>1. Write the solution in Python 3 (one .py file) and run it from the console.
2. Take the name, age and citizenship of the person as input.
3. Use if-elif-else conditions inside a function such as check_eligibility().
4. Handle invalid age input using try-except and ask again until a valid age is entered.
5. Show the reason when the person is not eligible and the final count of voters checked.</td></tr>
<tr><td>Instruction</td><td>1. Understand the problem statement and define the problem clearly.
2. Develop the solution using proper logic and programming concepts.
3. Implement all required features and pass the given test cases.
4. Test and debug the project to ensure correct output.
5. Prepare the documentation/report and upload the code and report to GitHub.
6. Complete and submit the project within the given deadline.</td></tr>
</table>

BACHELOR OF COMPUTER APPLICATION

<table>
<tr><td>PS No.</td><td>24</td></tr>
<tr><td>Project Title</td><td>Employee Data Analyzer</td></tr>
<tr><td>Problem Statement</td><td>Analyse employee data from a CSV file to clean it and find insights such as total employees, average and highest salary, department-wise salary and employees above a salary limit, with a chart.
Language: Python 3 Concepts: Pandas, Filtering, Grouping, Aggregation
Note: CSV columns: Emp_ID, Name, Department, Salary, Experience. Use dummy data only. Missing salaries are filled with the mean of the valid salaries.</td></tr>
<tr><td>Test Cases / Case Statements (4 cases)</td><td>
<table>
<tr><th>INPUT</th><th>EXPECTED OUTPUT</th></tr>
<tr><td colspan="2">Test Case 1: Normal Input</td></tr>
<tr><td>employees.csv:
E1,Aman,IT,50000,3
E2,Riya,HR,40000,2
E3,Karan,IT,70000,5
Menu option 1 (Summary)</td><td>Employees: 3 | Average salary: 53333.33
Highest paid: Karan (70000)
Total payroll: 160000</td></tr>
<tr><td colspan="2">Test Case 2: Boundary / Minimum Input</td></tr>
<tr><td>employees.csv: 1 row only
E1,Aman,IT,50000,3
Menu option 1 (Summary)</td><td>Employees: 1 | Average salary: 50000.00
Highest paid: Aman (50000)</td></tr>
<tr><td colspan="2">Test Case 3: Invalid Input</td></tr>
<tr><td>Test Case 1 rows plus:
E1,Aman,IT,50000,3 (duplicate)
E4,Neha,HR,,4 (salary missing)
Menu option 1 (Summary)</td><td>Duplicates removed: 1
Missing Salary filled with mean: 53333.33
Cleaned rows: 4</td></tr>
<tr><td colspan="2">Test Case 4: Different Valid Scenario</td></tr>
<tr><td>Same CSV as Test Case 1
Menu option 2 (Department-wise and filter above 45000)</td><td>IT: 2 employees | Avg salary: 60000.00
HR: 1 employee | Avg salary: 40000.00
Above 45000: Aman (50000) | Karan (70000)
Bar chart 'Department-wise Average Salary' displayed</td></tr>
</table>
</td></tr>
<tr><td>Bare Minimum Requirements</td><td>1. Write the solution in Python 3 (one .py file) and run it from the console.
2. Load the CSV file using Pandas and clean missing and duplicate values.
3. Use groupby() to find department-wise employee count and average salary.
4. Use boolean filtering to list employees above a given salary.
5. Calculate the highest paid employee and plot at least one Matplotlib bar chart with labels.</td></tr>
<tr><td>Instruction</td><td>1. Understand the problem statement and define the problem clearly.
2. Develop the solution using proper logic and programming concepts.
3. Implement all required features and pass the given test cases.
4. Test and debug the project to ensure correct output.
5. Prepare the documentation/report and upload the code and report to GitHub.
6. Complete and submit the project within the given deadline.</td></tr>
</table>

BACHELOR OF COMPUTER APPLICATION

<table>
<tr><td>PS No.</td><td>25</td></tr>
<tr><td>Project Title</td><td>Duplicate Data Finder</td></tr>
<tr><td>Problem Statement</td><td>Build a tool that reads a text file with one entry per line, finds the duplicate entries, reports how many times each repeats and saves the unique entries to a new file.
Language: Python 3 Concepts: Lists, Sets, Loops, File Handling
Note: Each line of data.txt is one entry. Unique entries are saved to unique.txt. Menu option 2 ignores case and extra spaces.</td></tr>
<tr><td>Test Cases / Case Statements (4 cases)</td><td>
<table>
<tr><th>INPUT</th><th>EXPECTED OUTPUT</th></tr>
<tr><td colspan="2">Test Case 1: Normal Input</td></tr>
<tr><td>data.txt:
apple
banana
apple
orange
banana
apple
Menu option 1 (Exact match)</td><td>Total items: 6 ⎪ Unique items: 3
Duplicates: apple (3 times) ⎪ banana (2 times)
Unique list saved to unique.txt</td></tr>
<tr><td colspan="2">Test Case 2: Boundary / Minimum Input</td></tr>
<tr><td>data.txt: 1 line only
apple
Menu option 1 (Exact match)</td><td>Total items: 1 ⎪ Unique items: 1
No duplicates found.</td></tr>
<tr><td colspan="2">Test Case 3: Invalid Input</td></tr>
<tr><td>(a) File name: dta.txt (does not exist)
(b) data.txt is empty</td><td>(a) File 'dta.txt' not found. Please enter a valid file name.
(b) File is empty. Nothing to check.</td></tr>
<tr><td colspan="2">Test Case 4: Different Valid Scenario</td></tr>
<tr><td>data.txt:
Apple
apple
banana
APPLE
Menu option 2 (Ignore case)</td><td>Total items: 4 ⎪ Unique items: 2
Duplicates: apple (3 times)
Unique list saved to unique.txt</td></tr>
</table>
</td></tr>
<tr><td>Bare Minimum Requirements</td><td>1. Write the solution in Python 3 (one .py file) and run it from the console.
2. Read the entries from a text file into a list, handling a missing file with try-except.
3. Use a set to find the unique entries and a loop to count how many times each entry repeats.
4. Provide an option to ignore case and extra spaces while comparing.
5. Show the total, unique and duplicate counts and save the unique entries to a file.</td></tr>
<tr><td>Instruction</td><td>1. Understand the problem statement and define the problem clearly.
2. Develop the solution using proper logic and programming concepts.
3. Implement all required features and pass the given test cases.
4. Test and debug the project to ensure correct output.
5. Prepare the documentation/report and upload the code and report to GitHub.
6. Complete and submit the project within the given deadline.</td></tr>
</table>

BACHELOR OF COMPUTER APPLICATION

<table>
<tr><td>PS No.</td><td>26</td></tr>
<tr><td>Project Title</td><td>E-Commerce Sales Analyzer</td></tr>
<tr><td>Problem Statement</td><td>Analyse online order data from a CSV file to find total revenue, units sold, best-selling products and category-wise revenue using aggregation, and visualise the results.
Language: Python 3 Concepts: Pandas, CSV, Matplotlib, Aggregation
Note: CSV columns: Order_ID, Product, Category, Quantity, Price. Revenue of a row = Quantity × Price.</td></tr>
<tr><td>Test Cases / Case Statements (4 cases)</td><td>
<table>
<tr><th>INPUT</th><th>EXPECTED OUTPUT</th></tr>
<tr><td colspan="2">Test Case 1: Normal Input</td></tr>
<tr><td>orders.csv:
O1,Phone,Electronics,2,10000
O2,Shirt,Fashion,5,500
O3,Phone,Electronics,1,10000
Menu option 1 (Summary)</td><td>Total Revenue: 32500 │ Orders: 3
Units sold: 8 │ Top product: Phone (30000)</td></tr>
<tr><td colspan="2">Test Case 2: Boundary / Minimum Input</td></tr>
<tr><td>orders.csv: 1 row only
O1,Pen,Stationery,1,10
Menu option 1 (Summary)</td><td>Total Revenue: 10 │ Orders: 1
Units sold: 1 │ Top product: Pen</td></tr>
<tr><td colspan="2">Test Case 3: Invalid Input</td></tr>
<tr><td>orders.csv:
O1,Phone,Electronics,2,10000
O2,Shirt,Fashion,abc,500
O3,Phone,Electronics,1,-10000</td><td>Row 2 skipped: invalid Quantity 'abc'
Row 3 skipped: Price must be positive
Total Revenue: 20000 │ Orders: 1</td></tr>
<tr><td colspan="2">Test Case 4: Different Valid Scenario</td></tr>
<tr><td>Same CSV as Test Case 1
Menu option 2 (Category-wise revenue)</td><td>Electronics: 30000 (92.31%)
Fashion: 2500 (7.69%)
Pie chart 'Category-wise Revenue' displayed</td></tr>
</table>
</td></tr>
<tr><td>Bare Minimum Requirements</td><td>1. Write the solution in Python 3 (one .py file) and run it from the console.
2. Read the CSV file using Pandas and clean invalid or duplicate rows.
3. Calculate revenue for each order and use groupby() to find product-wise and category-wise sales.
4. Show the top 5 selling products by revenue.
5. Plot at least two Matplotlib charts (bar and pie) with title and labels.</td></tr>
<tr><td>Instruction</td><td>1. Understand the problem statement and define the problem clearly.
2. Develop the solution using proper logic and programming concepts.
3. Implement all required features and pass the given test cases.
4. Test and debug the project to ensure correct output.
5. Prepare the documentation/report and upload the code and report to GitHub.
6. Complete and submit the project within the given deadline.</td></tr>
</table>

RUNGTA INTERNATIONAL SKILLS UNIVERSITY
SCHOOL OF INFORMATION TECHNOLOGY

BACHELOR OF COMPUTER APPLICATION

<table>
<tr><td>PS No.</td><td>27</td></tr>
<tr><td>Project Title</td><td>Student Attendance Manager</td></tr>
<tr><td>Problem Statement</td><td>Create a menu-driven console application to add students, mark daily attendance as present or absent, view each student's attendance percentage and list the students below the 75% requirement. Data is saved to a file.
Language: Python 3 Concepts: Lists, Dictionaries, Functions, File Handling
Note: Each student is a dictionary (roll, name, held, attended) stored in attendance.txt. Minimum required attendance is 75%. Test cases run in order on the same records.</td></tr>
<tr><td>Test Cases / Case Statements (4 cases)</td><td>
<table>
<tr><th>INPUT</th><th>EXPECTED OUTPUT</th></tr>
<tr><td colspan="2">Test Case 1: Normal Input</td></tr>
<tr><td>Menu option 1 (Add student)
Roll: 101
Name: Aman</td><td>Student added: 101 │ Aman │ Held: 0 │ Attended: 0</td></tr>
<tr><td colspan="2">Test Case 2: Boundary / Minimum Input</td></tr>
<tr><td>Student 101 has no classes held
Menu option 3 (Percentage): Roll 101</td><td>Aman: Attendance 0.00% (no classes held yet)</td></tr>
<tr><td colspan="2">Test Case 3: Invalid Input</td></tr>
<tr><td>(a) Menu option 2 (Mark): Roll 999
(b) Menu option 2 (Mark): Roll 101, Status: X</td><td>(a) Roll number 999 not found.
(b) Invalid status. Enter P or A.</td></tr>
<tr><td colspan="2">Test Case 4: Different Valid Scenario</td></tr>
<tr><td>Records: 101 Aman (4 held, 3 attended), 102 Riya (4 held, 2 attended)
Menu option 4 (Shortage list)</td><td>Below 75%: 102 │ Riya │ 50.00%
Aman (75.00%) meets the requirement.</td></tr>
</table>
</td></tr>
<tr><td>Bare Minimum Requirements</td><td>1. Write the solution in Python 3 (one .py file) and run it from the console.
2. Store each student as a dictionary inside a list, with a unique roll number.
3. Write functions to add a student, mark attendance (P/A), show the percentage and show the shortage list.
4. Validate the roll number and the attendance status, and avoid dividing by zero when no classes are held.
5. Load data from the file at start and save it after every change.</td></tr>
<tr><td>Instruction</td><td>1. Understand the problem statement and define the problem clearly.
2. Develop the solution using proper logic and programming concepts.
3. Implement all required features and pass the given test cases.
4. Test and debug the project to ensure correct output.
5. Prepare the documentation/report and upload the code and report to GitHub.
6. Complete and submit the project within the given deadline.</td></tr>
</table>

BACHELOR OF COMPUTER APPLICATION

<table>
<tr><td>PS No.</td><td>28</td></tr>
<tr><td>Project Title</td><td>Menu-Driven Calculator</td></tr>
<tr><td>Problem Statement</td><td>Create a console calculator that repeatedly shows a menu of arithmetic operations, takes two numbers, displays the result and handles errors such as division by zero until the user chooses to exit.
Language: Python 3 Concepts: Functions, Loops, Operators, Exceptions
Note: Menu: 1 Add, 2 Subtract, 3 Multiply, 4 Divide, 5 Power, 6 Modulus, 7 Exit. Results are rounded to 2 decimal places.</td></tr>
<tr><td>Test Cases / Case Statements (4 cases)</td><td>
<table>
<tr><th>INPUT</th><th>EXPECTED OUTPUT</th></tr>
<tr><td colspan="2">Test Case 1: Normal Input</td></tr>
<tr><td>Option 1 (Add): 12 and 8</td><td>12 + 8 = 20.00</td></tr>
<tr><td colspan="2">Test Case 2: Boundary / Minimum Input</td></tr>
<tr><td>Option 4 (Divide): 0 and 5 (zero numerator)</td><td>0 / 5 = 0.00</td></tr>
<tr><td colspan="2">Test Case 3: Invalid Input</td></tr>
<tr><td>(a) Option 4 (Divide): 10 and 0
(b) Option 3 (Multiply): abc and 4
(c) Menu choice: 9</td><td>(a) Error: Division by zero is not allowed.
(b) Invalid number. Enter numeric values only.
(c) Invalid choice. Select 1 to 7.</td></tr>
<tr><td colspan="2">Test Case 4: Different Valid Scenario</td></tr>
<tr><td>(a) Option 5 (Power): 2 and 10
(b) Option 6 (Modulus): 17 and 5
(c) Option 2 (Subtract): 5 and 9.5
(d) Option 7 (Exit)</td><td>(a) 2 ^ 10 = 1024.00
(b) 17 % 5 = 2.00
(c) 5 - 9.5 = -4.50
(d) Thank you for using the calculator. Exiting...</td></tr>
</table>
</td></tr>
<tr><td>Bare Minimum Requirements</td><td>1. Write the solution in Python 3 (one .py file) and run it from the console.
2. Show a loop-driven menu with an Exit option.
3. Write one function for each operation (add, subtract, multiply, divide, power, modulus).
4. Handle division by zero and non-numeric input using try-except.
5. Validate the menu choice and display every result rounded to 2 decimal places.</td></tr>
<tr><td>Instruction</td><td>1. Understand the problem statement and define the problem clearly.
2. Develop the solution using proper logic and programming concepts.
3. Implement all required features and pass the given test cases.
4. Test and debug the project to ensure correct output.
5. Prepare the documentation/report and upload the code and report to GitHub.
6. Complete and submit the project within the given deadline.</td></tr>
</table>

BACHELOR OF COMPUTER APPLICATION

<table>
<tr><td>PS No.</td><td>29</td></tr>
<tr><td>Project Title</td><td>Population Data Visualizer</td></tr>
<tr><td>Problem Statement</td><td>Analyse city population data from a CSV file to find the total population, the largest city and each city's growth between years, and visualise the results with charts.
Language: Python 3 Concepts: Pandas, Matplotlib, Data Analysis
Note: CSV columns: City, Year, Population. Growth % = (latest − earliest) ÷ earliest × 100. Use dummy data only.</td></tr>
<tr><td>Test Cases / Case Statements (4 cases)</td><td>
<table>
<tr><th>INPUT</th><th>EXPECTED OUTPUT</th></tr>
<tr><td colspan="2">Test Case 1: Normal Input</td></tr>
<tr><td>population.csv:
Raipur,2020,1000000
Raipur,2024,1100000
Durg,2020,500000
Durg,2024,600000
Menu option 1 (Summary)</td><td>Total population (2024): 1700000
Largest city: Raipur (1100000)
Fastest growing: Durg (+20.00%)</td></tr>
<tr><td colspan="2">Test Case 2: Boundary / Minimum Input</td></tr>
<tr><td>population.csv: 1 row only
Raipur,2024,1100000
Menu option 1 (Summary)</td><td>Total population: 1100000 │ Largest city: Raipur
Growth: not available (only one year of data)</td></tr>
<tr><td colspan="2">Test Case 3: Invalid Input</td></tr>
<tr><td>population.csv:
Raipur,2020,1000000
Durg,2020,abc
Bhilai,2020,-500</td><td>Row 2 skipped: invalid Population 'abc'.
Row 3 skipped: Population must be positive.
Valid rows: 1</td></tr>
<tr><td colspan="2">Test Case 4: Different Valid Scenario</td></tr>
<tr><td>Same CSV as Test Case 1
Menu option 2 (Growth and charts)</td><td>Raipur: +10.00% │ Durg: +20.00%
Bar chart 'Population Growth by City' and line chart 'Population Trend' displayed</td></tr>
</table>
</td></tr>
<tr><td>Bare Minimum Requirements</td><td>1. Write the solution in Python 3 (one .py file) and run it from the console.
2. Load the CSV file using Pandas and clean invalid or duplicate rows.
3. Use groupby() to find the latest population and the growth percentage of every city.
4. Show the total population, the largest city and the fastest growing city.
5. Plot at least two Matplotlib charts (line and bar) with title and labels.</td></tr>
<tr><td>Instruction</td><td>1. Understand the problem statement and define the problem clearly.
2. Develop the solution using proper logic and programming concepts.
3. Implement all required features and pass the given test cases.
4. Test and debug the project to ensure correct output.
5. Prepare the documentation/report and upload the code and report to GitHub.
6. Complete and submit the project within the given deadline.</td></tr>
</table>

BACHELOR OF COMPUTER APPLICATION

<table>
<tr><td>PS No.</td><td>30</td></tr>
<tr><td>Project Title</td><td>Bank Transaction Validator</td></tr>
<tr><td>Problem Statement</td><td>Build a program that reads bank transactions from a file, checks each one against banking rules, reports whether it is valid or invalid with the reason, and shows the final balance and a summary.
Language: Python 3 Concepts: Functions, Conditions, Exceptions, File Handling
Note: Opening balance is 10000. Each line of transactions.txt is Type,Amount (Deposit or Withdraw). Amount must be positive and up to 100000 per transaction, and a withdrawal cannot exceed the balance. Only valid transactions change the balance.</td></tr>
<tr><td>Test Cases / Case Statements (4 cases)</td><td>
<table>
<tr><th>INPUT</th><th>EXPECTED OUTPUT</th></tr>
<tr><td colspan="2">Test Case 1: Normal Input</td></tr>
<tr><td>transactions.txt:
Deposit,5000
Withdraw,2000</td><td>Line 1: Valid │ Balance: 15000
Line 2: Valid │ Balance: 13000
Valid: 2 │ Invalid: 0 │ Final balance: 13000</td></tr>
<tr><td colspan="2">Test Case 2: Boundary / Minimum Input</td></tr>
<tr><td>transactions.txt:
Withdraw,10000 (equal to balance)</td><td>Line 1: Valid │ Balance: 0
Valid: 1 │ Invalid: 0 │ Final balance: 0</td></tr>
<tr><td colspan="2">Test Case 3: Invalid Input</td></tr>
<tr><td>transactions.txt:
Withdraw,20000
Deposit,-500
Transfer,100
Deposit,abc</td><td>Line 1: Invalid – insufficient balance.
Line 2: Invalid – amount must be positive.
Line 3: Invalid – unknown type 'Transfer'.
Line 4: Invalid – amount 'abc' is not a number.
Valid: 0 │ Invalid: 4 │ Final balance: 10000</td></tr>
<tr><td colspan="2">Test Case 4: Different Valid Scenario</td></tr>
<tr><td>transactions.txt:
Deposit,150000
Deposit,50000
Withdraw,60000</td><td>Line 1: Invalid – exceeds limit of 100000.
Line 2: Valid │ Balance: 60000
Line 3: Valid │ Balance: 0
Valid: 2 │ Invalid: 1 │ Final balance: 0</td></tr>
</table>
</td></tr>
<tr><td>Bare Minimum Requirements</td><td>1. Write the solution in Python 3 (one .py file) and run it from the console.
2. Read the transactions from a text file and handle a missing file using try-except.
3. Write a function such as validate_transaction() that checks the type, amount, limit and available balance.
4. Print Valid or Invalid with the reason for every line and update the balance only for valid transactions.
5. Show the count of valid and invalid transactions and the final balance, and save the valid transactions to a file.</td></tr>
<tr><td>Instruction</td><td>1. Understand the problem statement and define the problem clearly.
2. Develop the solution using proper logic and programming concepts.
3. Implement all required features and pass the given test cases.
4. Test and debug the project to ensure correct output.
5. Prepare the documentation/report and upload the code and report to GitHub.
6. Complete and submit the project within the given deadline.</td></tr>
</table>

BACHELOR OF COMPUTER APPLICATION

<table>
<tr><td>PS No.</td><td>31</td></tr>
<tr><td>Project Title</td><td>Movie Rating Analyzer</td></tr>
<tr><td>Problem Statement</td><td>Analyse movie ratings from a CSV file to find the average rating, the top and lowest rated movies and genre-wise average ratings using grouping, and present the results with charts.
Language: Python 3 Concepts: Pandas, Matplotlib, Grouping, Statistics
Note: CSV columns: Movie, Genre, Rating. Ratings are out of 10 (0 to 10 allowed). Test cases use the small CSV rows given in each INPUT.</td></tr>
<tr><td>Test Cases / Case Statements (4 cases)</td><td>
<table>
<tr><th>INPUT</th><th>EXPECTED OUTPUT</th></tr>
<tr><td colspan="2">Test Case 1: Normal Input</td></tr>
<tr><td>movies.csv:
Inception,SciFi,8.8
Titanic,Romance,7.9
Interstellar,SciFi,8.6
Menu option 1 (Summary)</td><td>Average rating: 8.43
Top rated: Inception (8.8)
Lowest: Titanic (7.9)</td></tr>
<tr><td colspan="2">Test Case 2: Boundary / Minimum Input</td></tr>
<tr><td>movies.csv: 1 row only
Sholay,Action,10
Menu option 1 (Summary)</td><td>Average rating: 10.00
Top rated: Sholay (10) │ Lowest: Sholay (10)</td></tr>
<tr><td colspan="2">Test Case 3: Invalid Input</td></tr>
<tr><td>movies.csv:
Inception,SciFi,8.8
Titanic,Romance,11
Dangal,Drama,abc</td><td>Row 2 skipped: Rating 11 must be between 0 and 10.
Row 3 skipped: invalid Rating 'abc'.
Valid rows: 1</td></tr>
<tr><td colspan="2">Test Case 4: Different Valid Scenario</td></tr>
<tr><td>Same CSV as Test Case 1
Menu option 2 (Genre-wise average)</td><td>SciFi: 8.70 │ Romance: 7.90
Best genre: SciFi
Bar chart 'Genre-wise Average Rating' displayed</td></tr>
</table>
</td></tr>
<tr><td>Bare Minimum Requirements</td><td>1. Write the solution in Python 3 (one .py file) and run it from the console.
2. Load the CSV file using Pandas and clean invalid, missing or duplicate rows.
3. Calculate the average, maximum and minimum rating and show the top rated and lowest rated movie.
4. Use groupby() to find the genre-wise average rating and the best genre.
5. Plot at least two Matplotlib charts (bar and histogram) with title and labels.</td></tr>
<tr><td>Instruction</td><td>1. Understand the problem statement and define the problem clearly.
2. Develop the solution using proper logic and programming concepts.
3. Implement all required features and pass the given test cases.
4. Test and debug the project to ensure correct output.
5. Prepare the documentation/report and upload the code and report to GitHub.
6. Complete and submit the project within the given deadline.</td></tr>
</table>

BACHELOR OF COMPUTER APPLICATION

<table>
<tr><td>PS No.</td><td>32</td></tr>
<tr><td>Project Title</td><td>Class Marks Analyzer</td></tr>
<tr><td>Problem Statement</td><td>Create a program that takes the marks of a class as a list, validates them and uses NumPy to calculate the mean, median, highest, lowest, standard deviation, pass count and the number of students above average.
Language: Python 3 Concepts: NumPy, Lists, Functions, Statistics
Note: Marks are entered as numbers separated by spaces and must be from 0 to 100. Pass mark is 40. Standard deviation is the population value (np.std default).</td></tr>
<tr><td>Test Cases / Case Statements (4 cases)</td><td>
<table>
<tr><th>INPUT</th><th>EXPECTED OUTPUT</th></tr>
<tr><td colspan="2">Test Case 1: Normal Input</td></tr>
<tr><td>Marks: 40 50 60 70 80</td><td>Mean: 60.00 │ Median: 60.00
Max: 80 │ Min: 40 │ Std Dev: 14.14
Passed: 5 of 5</td></tr>
<tr><td colspan="2">Test Case 2: Boundary / Minimum Input</td></tr>
<tr><td>Marks: 75 (one student)</td><td>Mean: 75.00 │ Median: 75.00
Max: 75 │ Min: 75 │ Std Dev: 0.00</td></tr>
<tr><td colspan="2">Test Case 3: Invalid Input</td></tr>
<tr><td>(a) Marks: 50 abc 70
(b) Marks: 50 105 70</td><td>(a) Invalid marks 'abc'. Enter numbers only.
(b) Marks must be between 0 and 100.</td></tr>
<tr><td colspan="2">Test Case 4: Different Valid Scenario</td></tr>
<tr><td>Marks: 25 50 75 100</td><td>Mean: 62.50 │ Median: 62.50 │ Std Dev: 27.95
Passed: 3 │ Failed: 1 │ Pass percentage: 75.00%
Above average: 2 students</td></tr>
</table>
</td></tr>
<tr><td>Bare Minimum Requirements</td><td>1. Write the solution in Python 3 (one .py file) and run it from the console.
2. Read the marks of the class into a list and convert it to a NumPy array.
3. Use NumPy to calculate the mean, median, maximum, minimum and standard deviation.
4. Write functions such as get_marks() and analyze_marks() to count passed, failed and above-average students.
5. Validate the marks (numbers only, from 0 to 100) and print the results in a neat format.</td></tr>
<tr><td>Instruction</td><td>1. Understand the problem statement and define the problem clearly.
2. Develop the solution using proper logic and programming concepts.
3. Implement all required features and pass the given test cases.
4. Test and debug the project to ensure correct output.
5. Prepare the documentation/report and upload the code and report to GitHub.
6. Complete and submit the project within the given deadline.</td></tr>
</table>

BACHELOR OF COMPUTER APPLICATION

<table>
<tr><td>PS No.</td><td>33</td></tr>
<tr><td>Project Title</td><td>Simple Billing System</td></tr>
<tr><td>Problem Statement</td><td>Develop a console billing system for a small shop where the cashier adds products with quantities for a customer and generates a bill with subtotal, GST and grand total. The bill is saved to a file.
Language: Python 3 Concepts: Lists, Dictionaries, Functions, File Handling
Note: Products: Rice Rs.50, Sugar Rs.40, Oil Rs.120. GST is 5% of the subtotal. The bill is saved to bill.txt.</td></tr>
<tr><td>Test Cases / Case Statements (4 cases)</td><td>
<table>
<tr><th>INPUT</th><th>EXPECTED OUTPUT</th></tr>
<tr><td colspan="2">Test Case 1: Normal Input</td></tr>
<tr><td>Add item: Rice
Quantity: 2</td><td>Rice x2 added │ Amount: 100</td></tr>
<tr><td colspan="2">Test Case 2: Boundary / Minimum Input</td></tr>
<tr><td>Add item: Sugar
Quantity: 1
Generate bill</td><td>Subtotal: 40 │ GST (5%): 2.00
Grand Total: 42.00</td></tr>
<tr><td colspan="2">Test Case 3: Invalid Input</td></tr>
<tr><td>(a) Add item: Soap
(b) Add item: Rice; Quantity: 0</td><td>(a) Invalid product. Available: Rice, Sugar, Oil
(b) Quantity must be at least 1.</td></tr>
<tr><td colspan="2">Test Case 4: Different Valid Scenario</td></tr>
<tr><td>Customer: Riya
Add Rice x2, Oil x1, Sugar x3
Generate bill</td><td>Rice x2 = 100 │ Oil x1 = 120 │ Sugar x3 = 120
Subtotal: 340 │ GST (5%): 17.00
Grand Total: 357.00
Bill saved to bill.txt</td></tr>
</table>
</td></tr>
<tr><td>Bare Minimum Requirements</td><td>1. Write the solution in Python 3 (one .py file) and run it from the console.
2. Keep the product catalogue in a dictionary and the bill items in a list.
3. Write functions to add an item, view the bill and generate the final bill.
4. Calculate the subtotal, GST and grand total, and print the bill in a neat formatted layout.
5. Validate the product name and quantity, and save the final bill to a file.</td></tr>
<tr><td>Instruction</td><td>1. Understand the problem statement and define the problem clearly.
2. Develop the solution using proper logic and programming concepts.
3. Implement all required features and pass the given test cases.
4. Test and debug the project to ensure correct output.
5. Prepare the documentation/report and upload the code and report to GitHub.
6. Complete and submit the project within the given deadline.</td></tr>
</table>

BACHELOR OF COMPUTER APPLICATION

<table>
<tr><td>PS No.</td><td>34</td></tr>
<tr><td>Project Title</td><td>Restaurant Order Management System</td></tr>
<tr><td>Problem Statement</td><td>Create a console application for a restaurant where a waiter selects a table, takes orders from the menu, views the running order and generates the final bill with service charge and GST.
Language: Python 3 Concepts: Lists, Dictionaries, Loops, Billing
Note: Menu: Burger Rs.100, Pizza Rs.200, Coffee Rs.50. Tables are numbered 1 to 20. Service charge is 10% and GST is 5%, both on the subtotal.</td></tr>
<tr><td>Test Cases / Case Statements (4 cases)</td><td>
<table>
<tr><th>INPUT</th><th>EXPECTED OUTPUT</th></tr>
<tr><td colspan="2">Test Case 1: Normal Input</td></tr>
<tr><td>Table: 5
Order item: Burger
Quantity: 2</td><td>Burger x2 added to Table 5 │ Amount: 200</td></tr>
<tr><td colspan="2">Test Case 2: Boundary / Minimum Input</td></tr>
<tr><td>Table: 1
Order item: Coffee; Quantity: 1
Generate bill</td><td>Subtotal: 50 │ Service charge (10%): 5.00
GST (5%): 2.50 │ Total: 57.50</td></tr>
<tr><td colspan="2">Test Case 3: Invalid Input</td></tr>
<tr><td>(a) Order item: Pasta
(b) Quantity: abc
(c) Table: 0</td><td>(a) Item not on the menu. Available: Burger, Pizza, Coffee
(b) Invalid quantity. Enter a whole number.
(c) Invalid table number. Enter a number between 1 and 20.</td></tr>
<tr><td colspan="2">Test Case 4: Different Valid Scenario</td></tr>
<tr><td>Table: 3
Order Burger x2, Pizza x1, Coffee x4
Generate bill</td><td>Burger x2 = 200 │ Pizza x1 = 200 │ Coffee x4 = 200
Subtotal: 600 │ Service charge (10%): 60.00
GST (5%): 30.00 │ Total: 690.00</td></tr>
</table>
</td></tr>
<tr><td>Bare Minimum Requirements</td><td>1. Write the solution in Python 3 (one .py file) and run it from the console.
2. Keep the menu in a dictionary and the ordered items of each table in a list.
3. Use a loop to take multiple items for a table and show the running order.
4. Calculate the subtotal, service charge, GST and total, and print a neat bill.
5. Validate the table number, item name and quantity before adding an order.</td></tr>
<tr><td>Instruction</td><td>1. Understand the problem statement and define the problem clearly.
2. Develop the solution using proper logic and programming concepts.
3. Implement all required features and pass the given test cases.
4. Test and debug the project to ensure correct output.
5. Prepare the documentation/report and upload the code and report to GitHub.
6. Complete and submit the project within the given deadline.</td></tr>
</table>

BACHELOR OF COMPUTER APPLICATION

<table>
<tr><td>PS No.</td><td>35</td></tr>
<tr><td>Project Title</td><td>Employee Attendance System</td></tr>
<tr><td>Problem Statement</td><td>Build a console application to add employees, mark their daily attendance as present, absent or leave, and view an attendance report for each employee. Data is saved to a file.
Language: Python 3 Concepts: Dictionaries, Lists, Conditions, File Handling
Note: Employee IDs start from E101. Status codes: P Present, A Absent, L Leave. Dates use YYYY-MM-DD. Attendance % = Present days ÷ marked days × 100. Data is stored in attendance.txt. Test cases run in order on the same records.</td></tr>
<tr><td>Test Cases / Case Statements (4 cases)</td><td>
<table>
<tr><th>INPUT</th><th>EXPECTED OUTPUT</th></tr>
<tr><td colspan="2">Test Case 1: Normal Input</td></tr>
<tr><td>Menu option 1 (Add employee)
Name: Aman</td><td>Employee added: E101 │ Aman</td></tr>
<tr><td colspan="2">Test Case 2: Boundary / Minimum Input</td></tr>
<tr><td>Employee E101 has no attendance records
Menu option 3 (Report): E101</td><td>Aman: Present 0 │ Absent 0 │ Leave 0
Attendance: 0.00% (no records)</td></tr>
<tr><td colspan="2">Test Case 3: Invalid Input</td></tr>
<tr><td>(a) Menu option 2 (Mark): E999, 2026-08-01, P
(b) Mark: E101, 2026-13-45, P
(c) Mark: E101, 2026-08-01, X</td><td>(a) Employee E999 not found.
(b) Invalid date. Use YYYY-MM-DD.
(c) Invalid status. Enter P, A or L.</td></tr>
<tr><td colspan="2">Test Case 4: Different Valid Scenario</td></tr>
<tr><td>Employee E101 Aman
Mark: 2026-08-01 P, 2026-08-02 P, 2026-08-03 A, 2026-08-04 L, then 2026-08-02 P again
Menu option 3 (Report): E101</td><td>Attendance for 2026-08-02 is already marked.
Aman: Present 2 │ Absent 1 │ Leave 1
Attendance: 50.00%</td></tr>
</table>
</td></tr>
<tr><td>Bare Minimum Requirements</td><td>1. Write the solution in Python 3 (one .py file) and run it from the console.
2. Store each employee as a dictionary (id, name, records) inside a list.
3. Write functions to add an employee, mark attendance and show the report.
4. Validate the employee ID, the date format and the status, and block duplicate attendance for the same date.
5. Load the data from the file at start and save it after every change.</td></tr>
<tr><td>Instruction</td><td>1. Understand the problem statement and define the problem clearly.
2. Develop the solution using proper logic and programming concepts.
3. Implement all required features and pass the given test cases.
4. Test and debug the project to ensure correct output.
5. Prepare the documentation/report and upload the code and report to GitHub.
6. Complete and submit the project within the given deadline.</td></tr>
</table>

BACHELOR OF COMPUTER APPLICATION

<table>
<tr><td>PS No.</td><td>36</td></tr>
<tr><td>Project Title</td><td>Rainfall Data Analyzer</td></tr>
<tr><td>Problem Statement</td><td>Analyse monthly rainfall data from a CSV file using NumPy and Pandas to find total and average rainfall, the wettest and driest months and heavy-rainfall months, and plot the rainfall trend.
Language: Python 3 Concepts: NumPy, Pandas, Matplotlib, Statistics
Note: CSV columns: Month, Rainfall_mm. A heavy-rainfall month has more than 200 mm. Standard deviation is the population value (np.std default). Test cases use the small CSV rows given in each INPUT.</td></tr>
<tr><td>Test Cases / Case Statements (4 cases)</td><td>
<table>
<tr><th>INPUT</th><th>EXPECTED OUTPUT</th></tr>
<tr><td colspan="2">Test Case 1: Normal Input</td></tr>
<tr><td>rain.csv:
Jun,150
Jul,300
Aug,250
Menu option 1 (Statistics)</td><td>Total: 700 mm | Mean: 233.33
Max: Jul (300) | Min: Jun (150)
Std Dev: 62.36
Heavy rainfall months: Jul, Aug</td></tr>
<tr><td colspan="2">Test Case 2: Boundary / Minimum Input</td></tr>
<tr><td>rain.csv: 1 row only
Jun,200
Menu option 1 (Statistics)</td><td>Total: 200 mm | Mean: 200.00 | Std Dev: 0.00
No month above 200 mm</td></tr>
<tr><td colspan="2">Test Case 3: Invalid Input</td></tr>
<tr><td>rain.csv:
Jun,150
Jul,abc
Aug,-20</td><td>Row 2 skipped: invalid Rainfall 'abc'.
Row 3 skipped: Rainfall cannot be negative.
Valid rows: 1 | Total: 150 mm</td></tr>
<tr><td colspan="2">Test Case 4: Different Valid Scenario</td></tr>
<tr><td>Same CSV as Test Case 1
Menu option 3 (Compare with average and plot)</td><td>Above average (233.33): Jul (300) | Aug (250)
Below average: Jun (150)
Bar chart 'Monthly Rainfall' displayed</td></tr>
</table>
</td></tr>
<tr><td>Bare Minimum Requirements</td><td>1. Write the solution in Python 3 (one .py file) and run it from the console.
2. Load the CSV file using Pandas and handle missing or invalid readings.
3. Use NumPy to calculate total, mean, maximum, minimum and standard deviation.
4. Detect heavy-rainfall months and months above the average using boolean filtering.
5. Plot at least two Matplotlib charts (bar and line) with title and labels.</td></tr>
<tr><td>Instruction</td><td>1. Understand the problem statement and define the problem clearly.
2. Develop the solution using proper logic and programming concepts.
3. Implement all required features and pass the given test cases.
4. Test and debug the project to ensure correct output.
5. Prepare the documentation/report and upload the code and report to GitHub.
6. Complete and submit the project within the given deadline.</td></tr>
</table>

BACHELOR OF COMPUTER APPLICATION

<table>
<tr><td>PS No.</td><td>37</td></tr>
<tr><td>Project Title</td><td>Text-Based Adventure Game</td></tr>
<tr><td>Problem Statement</td><td>Create a text-based adventure game in which the player explores a forest, a cave and a river by choosing options from a menu, faces a random monster attack, collects score and ends the game by quitting or losing all health.
Language: Python 3 Concepts: Conditions, Loops, Functions, Random
Note: Player starts with Health 100 and Score 0 in the Forest. Choices: 1 Cave, 2 River, 3 Quit. Cave: a monster attacks (random damage 10 to 40, fixed to 30 for testing) and, if the player survives, a key gives +20 score. River: treasure gives +50 score. The game ends when Health reaches 0.</td></tr>
<tr><td>Test Cases / Case Statements (4 cases)</td><td>
<table>
<tr><th>INPUT</th><th>EXPECTED OUTPUT</th></tr>
<tr><td colspan="2">Test Case 1: Normal Input</td></tr>
<tr><td>Choice: 2 (River)</td><td>You found a treasure!
Score: 50 | Health: 100</td></tr>
<tr><td colspan="2">Test Case 2: Boundary / Minimum Input</td></tr>
<tr><td>Starting Health set to 30
Choice: 1 (Cave)
Monster damage fixed to 30</td><td>Monster attacks! Damage: 30. Health: 0
Game Over! Final score: 0</td></tr>
<tr><td colspan="2">Test Case 3: Invalid Input</td></tr>
<tr><td>(a) Choice: 7
(b) Choice: abc</td><td>(a) Invalid choice. Enter 1, 2 or 3.
(b) Invalid choice. Enter 1, 2 or 3.
No health lost.</td></tr>
<tr><td colspan="2">Test Case 4: Different Valid Scenario</td></tr>
<tr><td>Monster damage fixed to 30
Choices: 1 (Cave), 2 (River), 3 (Quit)</td><td>Cave: Monster attacks! Damage: 30. Health: 70
You found a key! Score: 20
River: You found a treasure! Score: 70 | Health: 70
You left the adventure. Final score: 70 | Health: 70</td></tr>
</table>
</td></tr>
<tr><td>Bare Minimum Requirements</td><td>1. Write the solution in Python 3 (one .py file) and run it from the console.
2. Use functions for each place (forest, cave, river) and conditions for the player's choices.
3. Use a loop to keep the game running until the player quits or Health reaches 0.
4. Use the random module for the monster damage, and track Health and Score.
5. Validate every menu choice and show the final score at the end.</td></tr>
<tr><td>Instruction</td><td>1. Understand the problem statement and define the problem clearly.
2. Develop the solution using proper logic and programming concepts.
3. Implement all required features and pass the given test cases.
4. Test and debug the project to ensure correct output.
5. Prepare the documentation/report and upload the code and report to GitHub.
6. Complete and submit the project within the given deadline.</td></tr>
</table>

BACHELOR OF COMPUTER APPLICATION

<table>
<tr><td>PS No.</td><td>38</td></tr>
<tr><td>Project Title</td><td>Currency Converter</td></tr>
<tr><td>Problem Statement</td><td>Build a console currency converter that converts an amount between supported currencies using exchange rates stored in a dictionary and displays the result rounded to 2 decimal places.
Language: Python 3 Concepts: Functions, Conditions, Operators, Dictionaries
Note: Rates to INR: 1 USD = 83, 1 EUR = 90, 1 GBP = 105. Supported codes: INR, USD, EUR, GBP. Every conversion goes through INR. Results are rounded to 2 decimal places.</td></tr>
<tr><td>Test Cases / Case Statements (4 cases)</td><td>
<table>
<tr><th>INPUT</th><th>EXPECTED OUTPUT</th></tr>
<tr><td colspan="2">Test Case 1: Normal Input</td></tr>
<tr><td>Amount: 166
From: INR
To: USD</td><td>166 INR = 2.00 USD</td></tr>
<tr><td colspan="2">Test Case 2: Boundary / Minimum Input</td></tr>
<tr><td>Amount: 0 (minimum)
From: USD
To: INR</td><td>0 USD = 0.00 INR</td></tr>
<tr><td colspan="2">Test Case 3: Invalid Input</td></tr>
<tr><td>(a) From: JPY
(b) Amount: abc
(c) Amount: -50</td><td>(a) Unsupported currency 'JPY'. Available: INR, USD, EUR, GBP
(b) Invalid amount. Enter a number.
(c) Amount cannot be negative.</td></tr>
<tr><td colspan="2">Test Case 4: Different Valid Scenario</td></tr>
<tr><td>(a) 100 USD to EUR
(b) 2 GBP to INR
(c) 45 EUR to USD</td><td>(a) 100 USD = 92.22 EUR
(b) 2 GBP = 210.00 INR
(c) 45 EUR = 48.80 USD</td></tr>
</table>
</td></tr>
<tr><td>Bare Minimum Requirements</td><td>1. Write the solution in Python 3 (one .py file) and run it from the console.
2. Keep the exchange rates in a dictionary.
3. Write functions such as convert_to_inr() and convert_currency().
4. Use conditions and operators to convert between any two supported currencies.
5. Validate the currency codes and the amount, and round every result to 2 decimal places.</td></tr>
<tr><td>Instruction</td><td>1. Understand the problem statement and define the problem clearly.
2. Develop the solution using proper logic and programming concepts.
3. Implement all required features and pass the given test cases.
4. Test and debug the project to ensure correct output.
5. Prepare the documentation/report and upload the code and report to GitHub.
6. Complete and submit the project within the given deadline.</td></tr>
</table>

BACHELOR OF COMPUTER APPLICATION

<table>
<tr><td>PS No.</td><td>39</td></tr>
<tr><td>Project Title</td><td>Bus/Train Ticket Booking System</td></tr>
<tr><td>Problem Statement</td><td>Develop a console ticket booking system for a bus with numbered seats where passengers can view seats, book a seat, view bookings and cancel a booking with a refund.
Language: Python 3 Concepts: Lists, Dictionaries, Functions, Seat Management
Note: The bus has 10 seats (1 to 10). Fare is Rs.500 per seat. Cancelling a booking refunds 80% of the fare. Menu: 1 View seats, 2 Book, 3 Bookings, 4 Cancel. Each test case starts with all seats free unless stated.</td></tr>
<tr><td>Test Cases / Case Statements (4 cases)</td><td>
<table>
<tr><th>INPUT</th><th>EXPECTED OUTPUT</th></tr>
<tr><td colspan="2">Test Case 1: Normal Input</td></tr>
<tr><td>Menu option 2 (Book)
Seat: 3
Passenger: Aman</td><td>Seat 3 booked for Aman. Fare: 500
Seats left: 9</td></tr>
<tr><td colspan="2">Test Case 2: Boundary / Minimum Input</td></tr>
<tr><td>Seats 1 to 9 already booked
Menu option 2 (Book)
Seat: 10 (last seat)
Passenger: Riya</td><td>Seat 10 booked for Riya. Fare: 500
Seats left: 0
Bus is now full.</td></tr>
<tr><td colspan="2">Test Case 3: Invalid Input</td></tr>
<tr><td>Seat 3 is booked by Aman
(a) Book seat 3
(b) Book seat 11</td><td>(a) Seat 3 is already booked.
(b) Invalid seat. Choose a seat from 1 to 10.</td></tr>
<tr><td colspan="2">Test Case 4: Different Valid Scenario</td></tr>
<tr><td>Booked: seat 3 (Aman), seat 5 (Riya)
(a) Option 4 (Cancel): seat 3
(b) Option 1 (View seats)</td><td>(a) Booking for seat 3 cancelled. Refund: 400
(b) Booked: 5 (Riya)
Available: 1, 2, 3, 4, 6, 7, 8, 9, 10</td></tr>
</table>
</td></tr>
<tr><td>Bare Minimum Requirements</td><td>1. Write the solution in Python 3 (one .py file) and run it from the console.
2. Store the seats in a list and the passenger details in a dictionary.
3. Write functions to view seats, book a seat, view bookings and cancel a booking.
4. Do not allow booking a seat that is already booked, and calculate the refund on cancellation.
5. Validate the seat number and the passenger name, and show the seats left after every change.</td></tr>
<tr><td>Instruction</td><td>1. Understand the problem statement and define the problem clearly.
2. Develop the solution using proper logic and programming concepts.
3. Implement all required features and pass the given test cases.
4. Test and debug the project to ensure correct output.
5. Prepare the documentation/report and upload the code and report to GitHub.
6. Complete and submit the project within the given deadline.</td></tr>
</table>

BACHELOR OF COMPUTER APPLICATION

<table>
<tr><td>PS No.</td><td>40</td></tr>
<tr><td>Project Title</td><td>Daily Task Manager</td></tr>
<tr><td>Problem Statement</td><td>Create a console task manager where each task has a title, a priority and a due date, and the user can add tasks, view them by priority, list pending tasks and mark tasks as done. Tasks are saved to a file.
Language: Python 3 Concepts: Lists, Dictionaries, Functions, File Handling
Note: Tasks are stored in tasks.txt. Task IDs start from T101. Priority is High, Medium or Low. Dates use YYYY-MM-DD. Menu: 1 Add, 2 View by priority, 3 Show pending, 4 Complete. Each test case starts with the data stated in it.</td></tr>
<tr><td>Test Cases / Case Statements (4 cases)</td><td>
<table>
<tr><th>INPUT</th><th>EXPECTED OUTPUT</th></tr>
<tr><td colspan="2">Test Case 1: Normal Input</td></tr>
<tr><td>Menu option 1 (Add)
Title: Submit report
Priority: High
Due: 2026-10-15</td><td>Task added.
T101 | Submit report | High | 2026-10-15 | Pending</td></tr>
<tr><td colspan="2">Test Case 2: Boundary / Minimum Input</td></tr>
<tr><td>No tasks in tasks.txt
Menu option 3 (Show pending)</td><td>No pending tasks. You are all caught up!</td></tr>
<tr><td colspan="2">Test Case 3: Invalid Input</td></tr>
<tr><td>(a) Add: Pay bill, Priority: Urgent
(b) Add: Pay bill, Due: 15-10-2026
(c) Option 4 (Complete): T999</td><td>(a) Invalid priority. Enter High, Medium or Low.
(b) Invalid date. Use YYYY-MM-DD.
(c) Task T999 not found.</td></tr>
<tr><td colspan="2">Test Case 4: Different Valid Scenario</td></tr>
<tr><td>Tasks: T101 Submit report (High, 2026-10-15), T102 Buy milk (Low, 2026-10-11), T103 Call bank (Medium, 2026-10-12)
(a) Option 2 (View by priority)
(b) Option 4 (Complete): T101</td><td>(a) T101 | Submit report | High | 2026-10-15
T103 | Call bank | Medium | 2026-10-12
T102 | Buy milk | Low | 2026-10-11
(b) Task T101 marked as done. Pending tasks: 2</td></tr>
</table>
</td></tr>
<tr><td>Bare Minimum Requirements</td><td>1. Write the solution in Python 3 (one .py file) and run it from the console.
2. Store each task as a dictionary inside a list, with a unique task ID.
3. Write one function each for add, view by priority, show pending and complete.
4. Validate the priority and the due date format before saving a task.
5. Load the tasks from the file at start and save them after every change.</td></tr>
<tr><td>Instruction</td><td>1. Understand the problem statement and define the problem clearly.
2. Develop the solution using proper logic and programming concepts.
3. Implement all required features and pass the given test cases.
4. Test and debug the project to ensure correct output.
5. Prepare the documentation/report and upload the code and report to GitHub.
6. Complete and submit the project within the given deadline.</td></tr>
</table>

BACHELOR OF COMPUTER APPLICATION

<table>
<tr><td>PS No.</td><td>41</td></tr>
<tr><td>Project Title</td><td>Restaurant Billing System</td></tr>
<tr><td>Problem Statement</td><td>Write a console billing program for a restaurant where the cashier enters the customer's items and quantities, and the program prints a bill with subtotal, discount, GST and grand total.
Language: Python 3 Concepts: Dictionaries, Loops, Functions, Calculations
Note: Menu: Dosa Rs.80, Idli Rs.40, Thali Rs.150, Tea Rs.20. A 10% discount is given if the subtotal is more than 500. GST is 5% of the amount after discount.</td></tr>
<tr><td>Test Cases / Case Statements (4 cases)</td><td>
<table>
<tr><th>INPUT</th><th>EXPECTED OUTPUT</th></tr>
<tr><td colspan="2">Test Case 1: Normal Input</td></tr>
<tr><td>Order item: Dosa
Quantity: 2</td><td>Dosa x2 added │ Amount: 160</td></tr>
<tr><td colspan="2">Test Case 2: Boundary / Minimum Input</td></tr>
<tr><td>Order item: Tea
Quantity: 1
Generate bill</td><td>Subtotal: 20 │ Discount: 0.00
GST (5%): 1.00 │ Grand Total: 21.00</td></tr>
<tr><td colspan="2">Test Case 3: Invalid Input</td></tr>
<tr><td>(a) Order item: Pizza
(b) Quantity: 0
(c) Quantity: abc</td><td>(a) Item not on the menu. Available: Dosa, Idli, Thali, Tea
(b) Quantity must be at least 1.
(c) Invalid quantity. Enter a whole number.</td></tr>
<tr><td colspan="2">Test Case 4: Different Valid Scenario</td></tr>
<tr><td>Order Thali x4, Dosa x2, Tea x2
Generate bill</td><td>Thali x4 = 600 │ Dosa x2 = 160 │ Tea x2 = 40
Subtotal: 800 │ Discount (10%): 80.00
GST (5%): 36.00
Grand Total: 756.00</td></tr>
</table>
</td></tr>
<tr><td>Bare Minimum Requirements</td><td>1. Write the solution in Python 3 (one .py file) and run it from the console.
2. Keep the menu in a dictionary and the ordered items in a list.
3. Use a loop to take many items and write functions to calculate the subtotal, discount, GST and total.
4. Apply the discount rule and the GST rule correctly.
5. Validate the item name and the quantity, and print the bill in a neat formatted layout.</td></tr>
<tr><td>Instruction</td><td>1. Understand the problem statement and define the problem clearly.
2. Develop the solution using proper logic and programming concepts.
3. Implement all required features and pass the given test cases.
4. Test and debug the project to ensure correct output.
5. Prepare the documentation/report and upload the code and report to GitHub.
6. Complete and submit the project within the given deadline.</td></tr>
</table>

BACHELOR OF COMPUTER APPLICATION

<table>
<tr><td>PS No.</td><td>42</td></tr>
<tr><td>Project Title</td><td>Personal Expense & Budget Manager</td></tr>
<tr><td>Problem Statement</td><td>Build a tool that stores expenses in a CSV file, lets the user set a monthly budget, and reports category-wise spending, the remaining budget and warnings when the budget is almost used or exceeded.
Language: Python 3 Concepts: Functions, Dictionaries, CSV, Pandas
Note: Expenses are saved in expenses.csv with columns Date, Category, Amount. Menu: 1 Set budget, 2 Add expense, 3 Category summary, 4 Report. A warning is shown when 80% of the budget is used and 'Budget exceeded' above 100%.</td></tr>
<tr><td>Test Cases / Case Statements (4 cases)</td><td>
<table>
<tr><th>INPUT</th><th>EXPECTED OUTPUT</th></tr>
<tr><td colspan="2">Test Case 1: Normal Input</td></tr>
<tr><td>Menu option 1 (Set budget)
Budget: 5000</td><td>Monthly budget set: 5000</td></tr>
<tr><td colspan="2">Test Case 2: Boundary / Minimum Input</td></tr>
<tr><td>Budget: 1000
Menu option 2 (Add)
2026-10-05, Food, 1000</td><td>Expense saved.
Total spent: 1000 | Remaining: 0
Budget fully used (100.00%).</td></tr>
<tr><td colspan="2">Test Case 3: Invalid Input</td></tr>
<tr><td>(a) Add: 2026-10-05, Food, -200
(b) Add: 05-10-2026, Food, 200</td><td>(a) Invalid amount. Amount must be a positive number.
Expense not saved.
(b) Invalid date. Use YYYY-MM-DD.</td></tr>
<tr><td colspan="2">Test Case 4: Different Valid Scenario</td></tr>
<tr><td>Budget: 5000
Expenses: Food 1500, Travel 1000, Food 1700
Menu option 4 (Report)</td><td>Food: 3200 | Travel: 1000
Total spent: 4200 | Remaining: 800
Warning: 84.00% of the budget used.</td></tr>
</table>
</td></tr>
<tr><td>Bare Minimum Requirements</td><td>1. Write the solution in Python 3 (one .py file) and run it from the console.
2. Store and read the expenses using a CSV file and a Pandas DataFrame.
3. Use groupby() to show the category-wise spending.
4. Calculate the total spent, the remaining budget and the percentage of the budget used.
5. Show a warning at 80% and a 'Budget exceeded' message above 100%, and validate the date and the amount.</td></tr>
<tr><td>Instruction</td><td>1. Understand the problem statement and define the problem clearly.
2. Develop the solution using proper logic and programming concepts.
3. Implement all required features and pass the given test cases.
4. Test and debug the project to ensure correct output.
5. Prepare the documentation/report and upload the code and report to GitHub.
6. Complete and submit the project within the given deadline.</td></tr>
</table>

BACHELOR OF COMPUTER APPLICATION

<table>
<tr><td>PS No.</td><td>43</td></tr>
<tr><td>Project Title</td><td>Online Food Ordering System</td></tr>
<tr><td>Problem Statement</td><td>Develop a console food ordering system where a customer adds items to a cart, applies a coupon and checks out to get a bill with discount, GST and delivery charge.
Language: Python 3 Concepts: Lists, Dictionaries, Functions, Billing
Note: Menu: Burger Rs.120, Pizza Rs.250, Fries Rs.80, Coke Rs.40. Coupon SAVE10 gives 10% off the subtotal. GST is 5% of the amount after discount. Delivery is Rs.40, free if the subtotal is Rs.300 or more.</td></tr>
<tr><td>Test Cases / Case Statements (4 cases)</td><td>
<table>
<tr><th>INPUT</th><th>EXPECTED OUTPUT</th></tr>
<tr><td colspan="2">Test Case 1: Normal Input</td></tr>
<tr><td>Add to cart: Burger
Quantity: 2</td><td>Burger x2 added to cart │ Cart subtotal: 240</td></tr>
<tr><td colspan="2">Test Case 2: Boundary / Minimum Input</td></tr>
<tr><td>Add to cart: Fries x1
Checkout (no coupon)</td><td>Subtotal: 80 │ Discount: 0.00
GST (5%): 4.00 │ Delivery: 40
Total: 124.00</td></tr>
<tr><td colspan="2">Test Case 3: Invalid Input</td></tr>
<tr><td>(a) Add to cart: Sushi
(b) Coupon: FREE50</td><td>(a) Item not available. Menu: Burger, Pizza, Fries, Coke
(b) Invalid coupon code. Only SAVE10 is accepted.</td></tr>
<tr><td colspan="2">Test Case 4: Different Valid Scenario</td></tr>
<tr><td>Add Pizza x2, Fries x1
Coupon: SAVE10
Checkout</td><td>Subtotal: 580 │ Discount (10%): 58.00
GST (5%): 26.10 │ Delivery: 0 (free)
Total: 548.10</td></tr>
</table>
</td></tr>
<tr><td>Bare Minimum Requirements</td><td>1. Write the solution in Python 3 (one .py file) and run it from the console.
2. Keep the menu in a dictionary and the cart in a list.
3. Write functions to add items, remove items, apply a coupon and checkout.
4. Calculate the subtotal, discount, GST, delivery charge and total.
5. Validate the item name, the quantity and the coupon code.</td></tr>
<tr><td>Instruction</td><td>1. Understand the problem statement and define the problem clearly.
2. Develop the solution using proper logic and programming concepts.
3. Implement all required features and pass the given test cases.
4. Test and debug the project to ensure correct output.
5. Prepare the documentation/report and upload the code and report to GitHub.
6. Complete and submit the project within the given deadline.</td></tr>
</table>

BACHELOR OF COMPUTER APPLICATION

<table>
<tr><td>PS No.</td><td>44</td></tr>
<tr><td>Project Title</td><td>Student Attendance & Performance Tracker</td></tr>
<tr><td>Problem Statement</td><td>Analyse student attendance and marks from a CSV file to calculate attendance percentage, identify students at risk and present the results with charts.
Language: Python 3 Concepts: Pandas, Dictionaries, Calculations, Visualization
Note: CSV columns: Name, Classes_Held, Classes_Attended, Marks (out of 100). A student is At Risk if attendance is below 75% or marks are below 40.</td></tr>
<tr><td>Test Cases / Case Statements (4 cases)</td><td>
<table>
<tr><th>INPUT</th><th>EXPECTED OUTPUT</th></tr>
<tr><td colspan="2">Test Case 1: Normal Input</td></tr>
<tr><td>students.csv:
Aman,40,36,80
Riya,40,28,65
Karan,40,30,35
Menu option 1 (Report)</td><td>Aman: 90.00% │ Marks 80 │ OK
Riya: 70.00% │ Marks 65 │ At Risk (attendance below 75%)
Karan: 75.00% │ Marks 35 │ At Risk (marks below 40)</td></tr>
<tr><td colspan="2">Test Case 2: Boundary / Minimum Input</td></tr>
<tr><td>students.csv: 1 row only
Neha,40,30,40
Menu option 1 (Report)</td><td>Neha: 75.00% │ Marks 40 │ OK</td></tr>
<tr><td colspan="2">Test Case 3: Invalid Input</td></tr>
<tr><td>students.csv:
Aman,40,45,80
Riya,40,30,abc</td><td>Row 1 skipped: attended (45) cannot exceed held (40).
Row 2 skipped: invalid Marks 'abc'.
No valid data to analyse.</td></tr>
<tr><td colspan="2">Test Case 4: Different Valid Scenario</td></tr>
<tr><td>Same CSV as Test Case 1
Menu option 2 (Summary and charts)</td><td>Average attendance: 78.33% │ Average marks: 60.00
At Risk students: 2 (Riya, Karan)
Bar chart and scatter chart 'Attendance vs Marks' displayed</td></tr>
</table>
</td></tr>
<tr><td>Bare Minimum Requirements</td><td>1. Write the solution in Python 3 (one .py file) and run it from the console.
2. Load the CSV file using Pandas and clean invalid rows.
3. Calculate the attendance percentage of every student using Pandas columns.
4. Use boolean filtering to find the At Risk students and calculate the averages.
5. Plot at least two Matplotlib charts (bar and scatter) with title and labels.</td></tr>
<tr><td>Instruction</td><td>1. Understand the problem statement and define the problem clearly.
2. Develop the solution using proper logic and programming concepts.
3. Implement all required features and pass the given test cases.
4. Test and debug the project to ensure correct output.
5. Prepare the documentation/report and upload the code and report to GitHub.
6. Complete and submit the project within the given deadline.</td></tr>
</table>

BACHELOR OF COMPUTER APPLICATION

<table>
<tr><td>PS No.</td><td>45</td></tr>
<tr><td>Project Title</td><td>Mini E-Commerce System</td></tr>
<tr><td>Problem Statement</td><td>Create a console shopping application where an admin adds, updates and deletes products and a customer adds products to a cart and checks out. Product data is stored in a file.
Language: Python 3 Concepts: Products, Cart, CRUD, File Handling
Note: Products are stored in products.txt (ID, name, price, stock). The first product ID is P101. Checkout reduces the stock. Each test case starts with an empty product list unless stated.</td></tr>
<tr><td>Test Cases / Case Statements (4 cases)</td><td>
<table>
<tr><th>INPUT</th><th>EXPECTED OUTPUT</th></tr>
<tr><td colspan="2">Test Case 1: Normal Input</td></tr>
<tr><td>Menu option 1 (Add product)
Laptop, Price: 50000, Stock: 5</td><td>Product added.
ID: P101 │ Laptop │ Price: 50000 │ Stock: 5</td></tr>
<tr><td colspan="2">Test Case 2: Boundary / Minimum Input</td></tr>
<tr><td>Menu option 1 (Add product)
Pen, Price: 1 (minimum), Stock: 0</td><td>Product added. ID: P101 │ Pen │ Price: 1 │ Stock: 0
Warning: Pen is out of stock.</td></tr>
<tr><td colspan="2">Test Case 3: Invalid Input</td></tr>
<tr><td>(a) Add product: Mouse, Price: -500
(b) Add to cart: P999</td><td>(a) Invalid price. Price must be a positive number.
(b) Product P999 not found.</td></tr>
<tr><td colspan="2">Test Case 4: Different Valid Scenario</td></tr>
<tr><td>Product: P101 Laptop (Price 50000, Stock 5)
(a) Add to cart: P101, Quantity: 2
(b) Checkout
(c) Delete product P101</td><td>(a) Laptop x2 added │ Amount: 100000
(b) Order total: 100000 │ Laptop stock left: 3
(c) Product 'Laptop' deleted. Products left: 0</td></tr>
</table>
</td></tr>
<tr><td>Bare Minimum Requirements</td><td>1. Write the solution in Python 3 (one .py file) and run it from the console.
2. Store the products in a list of dictionaries and the cart in a dictionary.
3. Write functions for add, update, delete, view, add to cart and checkout (CRUD).
4. Reduce the stock at checkout and do not allow ordering more than the stock available.
5. Load the products from the file at start and save them after every change.</td></tr>
<tr><td>Instruction</td><td>1. Understand the problem statement and define the problem clearly.
2. Develop the solution using proper logic and programming concepts.
3. Implement all required features and pass the given test cases.
4. Test and debug the project to ensure correct output.
5. Prepare the documentation/report and upload the code and report to GitHub.
6. Complete and submit the project within the given deadline.</td></tr>
</table>

BACHELOR OF COMPUTER APPLICATION

<table>
<tr><td>PS No.</td><td>46</td></tr>
<tr><td>Project Title</td><td>Movie Ticket Booking System</td></tr>
<tr><td>Problem Statement</td><td>Develop a console movie ticket booking system with a seat map where the user can book seats, view the seat map and see the amount payable. Bookings are saved to a file.
Language: Python 3 Concepts: Lists, Seat Management, Functions, File Handling
Note: The hall has 15 seats: rows A to C and seat numbers 1 to 5 (A1 to C5). Ticket price is Rs.200 per seat. Bookings are saved in bookings.txt. Menu: 1 Book, 2 Cancel, 3 Seat map. Each test case starts with an empty hall unless stated.</td></tr>
<tr><td>Test Cases / Case Statements (4 cases)</td><td>
<table>
<tr><th>INPUT</th><th>EXPECTED OUTPUT</th></tr>
<tr><td colspan="2">Test Case 1: Normal Input</td></tr>
<tr><td>Menu option 1 (Book)
Seat: A1
Name: Aman</td><td>Seat A1 booked for Aman. Amount: 200</td></tr>
<tr><td colspan="2">Test Case 2: Boundary / Minimum Input</td></tr>
<tr><td>All seats except C5 are booked
Menu option 1 (Book)
Seat: C5
Name: Riya</td><td>Seat C5 booked for Riya. Amount: 200
Seats left: 0
Show is now housefull.</td></tr>
<tr><td colspan="2">Test Case 3: Invalid Input</td></tr>
<tr><td>A1 is already booked
(a) Book: A1
(b) Book: Z9</td><td>(a) Seat A1 is already booked.
(b) Invalid seat. Rows are A to C and seat numbers are 1 to 5.</td></tr>
<tr><td colspan="2">Test Case 4: Different Valid Scenario</td></tr>
<tr><td>A1 booked by Aman
(a) Book: A2 and A3 for Neha
(b) Option 3 (Seat map)</td><td>(a) Seats A2, A3 booked for Neha. Amount: 400
(b) Row A: X X X _ _
Row B: _ _ _ _ _
Row C: _ _ _ _ _
Seats left: 12</td></tr>
</table>
</td></tr>
<tr><td>Bare Minimum Requirements</td><td>1. Write the solution in Python 3 (one .py file) and run it from the console.
2. Store the seat map as a 2D list (rows A to C, seats 1 to 5).
3. Write functions to book seats, cancel a booking and display the seat map.
4. Do not allow booking a seat that is already booked, and calculate the amount for the tickets.
5. Validate the seat code, and load the bookings from the file at start and save them after every change.</td></tr>
<tr><td>Instruction</td><td>1. Understand the problem statement and define the problem clearly.
2. Develop the solution using proper logic and programming concepts.
3. Implement all required features and pass the given test cases.
4. Test and debug the project to ensure correct output.
5. Prepare the documentation/report and upload the code and report to GitHub.
6. Complete and submit the project within the given deadline.</td></tr>
</table>

BACHELOR OF COMPUTER APPLICATION

<table>
<tr><td>PS No.</td><td>47</td></tr>
<tr><td>Project Title</td><td>Parking Management System</td></tr>
<tr><td>Problem Statement</td><td>Build a console parking system for a small lot where vehicles are parked in numbered slots, removed on exit, and charged according to the vehicle type and the parking time.
Language: Python 3 Concepts: Lists, Dictionaries, Conditions, File Handling
Note: The lot has 5 slots. Rates: car Rs.20 per hour, bike Rs.10 per hour. Hours are rounded up and the minimum charge is 1 hour. Data is saved in parking.txt. Menu: 1 Entry, 2 Exit, 3 Status. Each test case starts with an empty lot unless stated.</td></tr>
<tr><td>Test Cases / Case Statements (4 cases)</td><td>
<table>
<tr><th>INPUT</th><th>EXPECTED OUTPUT</th></tr>
<tr><td colspan="2">Test Case 1: Normal Input</td></tr>
<tr><td>Menu option 1 (Entry)
Vehicle: MH12AB1234
Type: car</td><td>Vehicle MH12AB1234 parked in slot 1.
Slots free: 4</td></tr>
<tr><td colspan="2">Test Case 2: Boundary / Minimum Input</td></tr>
<tr><td>MH12AB1234 (car) is in slot 1
Menu option 2 (Exit)
Vehicle: MH12AB1234
Hours: 0.5</td><td>Hours charged: 1 (minimum) │ Fee: 20
Slot 1 is now free.</td></tr>
<tr><td colspan="2">Test Case 3: Invalid Input</td></tr>
<tr><td>MH12AB1234 is parked
(a) Entry: MH12AB1234
(b) Entry: KA01XY9999, Type: truck
(c) Exit: XX00XX0000</td><td>(a) Vehicle MH12AB1234 is already parked.
(b) Invalid type. Enter car or bike.
(c) Vehicle XX00XX0000 not found.</td></tr>
<tr><td colspan="2">Test Case 4: Different Valid Scenario</td></tr>
<tr><td>(a) Entry: KA01AB1111, Type: bike
(b) Exit: KA01AB1111, Hours: 2.5
(c) Option 3 (Status)</td><td>(a) Vehicle KA01AB1111 parked in slot 1.
(b) Hours charged: 3 (rounded up) │ Fee: 30
(c) Slots free: 5 of 5</td></tr>
</table>
</td></tr>
<tr><td>Bare Minimum Requirements</td><td>1. Write the solution in Python 3 (one .py file) and run it from the console.
2. Store the slots in a list and each parked vehicle as a dictionary.
3. Write functions for entry, exit, fee calculation and status.
4. Use conditions for the vehicle type, the full-lot check and the minimum charge rule.
5. Validate the vehicle number, the type and the hours, and save the data to a file after every change.</td></tr>
<tr><td>Instruction</td><td>1. Understand the problem statement and define the problem clearly.
2. Develop the solution using proper logic and programming concepts.
3. Implement all required features and pass the given test cases.
4. Test and debug the project to ensure correct output.
5. Prepare the documentation/report and upload the code and report to GitHub.
6. Complete and submit the project within the given deadline.</td></tr>
</table>

BACHELOR OF COMPUTER APPLICATION

<table>
<tr><td>PS No.</td><td>48</td></tr>
<tr><td>Project Title</td><td>Employee Payroll System</td></tr>
<tr><td>Problem Statement</td><td>Develop a payroll program using classes to calculate the net salary of employees and managers from basic pay, overtime and bonus, with PF and tax deductions, and save the payslips to a file.
Language: Python 3 Concepts: Functions, Calculations, OOP, File Handling
Note: Class Employee (id, name, basic, overtime hours) and class Manager (inherits Employee, adds a 10% bonus on basic). Overtime is Rs.200 per hour. Gross = basic + overtime + bonus. PF = 12% of basic. Tax = 10% of gross if gross is above 50000, else 0. Net = Gross − PF − Tax. Payslips are saved in payroll.txt.</td></tr>
<tr><td>Test Cases / Case Statements (4 cases)</td><td>
<table>
<tr><th>INPUT</th><th>EXPECTED OUTPUT</th></tr>
<tr><td colspan="2">Test Case 1: Normal Input</td></tr>
<tr><td>Employee: E1, Aman
Basic: 30000
Overtime hours: 10</td><td>Overtime: 2000 │ Gross: 32000
PF: 3600 │ Tax: 0
Net Salary: 28400</td></tr>
<tr><td colspan="2">Test Case 2: Boundary / Minimum Input</td></tr>
<tr><td>Employee: E2, Neha
Basic: 10000
Overtime hours: 0</td><td>Overtime: 0 │ Gross: 10000
PF: 1200 │ Tax: 0
Net Salary: 8800</td></tr>
<tr><td colspan="2">Test Case 3: Invalid Input</td></tr>
<tr><td>(a) Basic: -5000
(b) Overtime hours: abc</td><td>(a) Invalid basic salary. Enter a positive number.
(b) Invalid overtime hours. Enter a non-negative number.</td></tr>
<tr><td colspan="2">Test Case 4: Different Valid Scenario</td></tr>
<tr><td>Manager: M1, Riya
Basic: 60000
Overtime hours: 5</td><td>Overtime: 1000 │ Bonus: 6000 │ Gross: 67000
PF: 7200 │ Tax: 6700
Net Salary: 53100</td></tr>
</table>
</td></tr>
<tr><td>Bare Minimum Requirements</td><td>1. Write the solution in Python 3 (one .py file) and run it from the console.
2. Create a class Employee and a class Manager that inherits from it.
3. Write methods such as calculate_gross() and calculate_net() and a function to print the payslip.
4. Apply the overtime, bonus, PF and tax rules correctly.
5. Validate the inputs, allow more employees using a loop, and save every payslip to a file.</td></tr>
<tr><td>Instruction</td><td>1. Understand the problem statement and define the problem clearly.
2. Develop the solution using proper logic and programming concepts.
3. Implement all required features and pass the given test cases.
4. Test and debug the project to ensure correct output.
5. Prepare the documentation/report and upload the code and report to GitHub.
6. Complete and submit the project within the given deadline.</td></tr>
</table>

BACHELOR OF COMPUTER APPLICATION

<table>
<tr><td>PS No.</td><td>49</td></tr>
<tr><td>Project Title</td><td>Exam Result Management System</td></tr>
<tr><td>Problem Statement</td><td>Create a system that stores student records with marks in three subjects, calculates total, percentage and result, and displays a rank list sorted by total marks. Records are saved to a file.
Language: Python 3 Concepts: Records, Calculations, Sorting, File Handling
Note: Marks are out of 100 in Maths, Physics and Chemistry. A student passes only if every subject has 40 or more. Records are saved in results.txt. Menu: 1 Add, 2 View all, 3 Rank list. Roll numbers must be unique.</td></tr>
<tr><td>Test Cases / Case Statements (4 cases)</td><td>
<table>
<tr><th>INPUT</th><th>EXPECTED OUTPUT</th></tr>
<tr><td colspan="2">Test Case 1: Normal Input</td></tr>
<tr><td>Menu option 1 (Add)
101, Aman, Marks: 80, 70, 90</td><td>Record added: 101 | Aman
Total: 240 | Percentage: 80.00% | Result: Pass</td></tr>
<tr><td colspan="2">Test Case 2: Boundary / Minimum Input</td></tr>
<tr><td>Menu option 1 (Add)
102, Riya, Marks: 40, 40, 40 (pass limit)</td><td>Record added: 102 | Riya
Total: 120 | Percentage: 40.00% | Result: Pass</td></tr>
<tr><td colspan="2">Test Case 3: Invalid Input</td></tr>
<tr><td>Roll 101 already exists
(a) Add: 101, Neha, 70, 70, 70
(b) Add: 103, Ravi, 105, 60, 60</td><td>(a) Roll number 101 already exists.
(b) Marks must be between 0 and 100.</td></tr>
<tr><td colspan="2">Test Case 4: Different Valid Scenario</td></tr>
<tr><td>Records: 101 Aman (80, 70, 90), 102 Riya (40, 40, 40), 103 Karan (60, 60, 60)
Menu option 3 (Rank list)</td><td>Rank 1 | 101 | Aman | 240 | 80.00% | Pass
Rank 2 | 103 | Karan | 180 | 60.00% | Pass
Rank 3 | 102 | Riya | 120 | 40.00% | Pass</td></tr>
</table>
</td></tr>
<tr><td>Bare Minimum Requirements</td><td>1. Write the solution in Python 3 (one .py file) and run it from the console.
2. Store each student record as a dictionary inside a list.
3. Calculate the total, percentage and the pass/fail result using functions.
4. Sort the records by total marks in descending order to show the rank list.
5. Validate the roll number and the marks, and load and save the records using a file.</td></tr>
<tr><td>Instruction</td><td>1. Understand the problem statement and define the problem clearly.
2. Develop the solution using proper logic and programming concepts.
3. Implement all required features and pass the given test cases.
4. Test and debug the project to ensure correct output.
5. Prepare the documentation/report and upload the code and report to GitHub.
6. Complete and submit the project within the given deadline.</td></tr>
</table>

BACHELOR OF COMPUTER APPLICATION

<table>
<tr><td>PS No.</td><td>50</td></tr>
<tr><td>Project Title</td><td>Inventory + Sales Management System</td></tr>
<tr><td>Problem Statement</td><td>Build a system that manages inventory items and records sales: items can be added, updated and deleted, each sale reduces the stock, and a sales report shows units sold and revenue per item.
Language: Python 3 Concepts: CRUD, Dictionaries, Pandas, File Handling
Note: Files: inventory.csv (ID, Name, Stock, Price) and sales.csv (ID, Quantity, Amount). The first item ID is I101. Menu: 1 Add item, 2 Sell, 3 Update, 4 Delete, 5 Sales report.</td></tr>
<tr><td>Test Cases / Case Statements (4 cases)</td><td>
<table>
<tr><th>INPUT</th><th>EXPECTED OUTPUT</th></tr>
<tr><td colspan="2">Test Case 1: Normal Input</td></tr>
<tr><td>Menu option 1 (Add item)
Name: Pen
Stock: 100
Price: 10</td><td>Item added.
ID: I101 │ Pen │ Stock: 100 │ Price: 10</td></tr>
<tr><td colspan="2">Test Case 2: Boundary / Minimum Input</td></tr>
<tr><td>Item: I101 Pen (Stock 5, Price 10)
Menu option 2 (Sell)
ID: I101, Quantity: 5</td><td>Sale recorded: Pen x5 = 50
Stock left: 0 │ Item is now out of stock.</td></tr>
<tr><td colspan="2">Test Case 3: Invalid Input</td></tr>
<tr><td>Item: I101 Pen (Stock 5)
(a) Sell: I101, Quantity: 10
(b) Sell: I999, Quantity: 1</td><td>(a) Not enough stock. Available: 5
(b) Item ID I999 not found.</td></tr>
<tr><td colspan="2">Test Case 4: Different Valid Scenario</td></tr>
<tr><td>Items: I101 Pen (Stock 100, Price 10), I102 Book (Stock 20, Price 50)
Sales: Pen x10, Book x2, Pen x5
Menu option 5 (Sales report)</td><td>Pen: 15 units │ Revenue: 150
Book: 2 units │ Revenue: 100
Total revenue: 250
Best seller: Pen (15 units)</td></tr>
</table>
</td></tr>
<tr><td>Bare Minimum Requirements</td><td>1. Write the solution in Python 3 (one .py file) and run it from the console.
2. Store the inventory and the sales in CSV files and load them with Pandas.
3. Write one function each for add, update, delete and sell (CRUD).
4. Reduce the stock on every sale and do not allow selling more than the stock available.
5. Use groupby() to prepare the sales report with units sold and revenue per item.</td></tr>
<tr><td>Instruction</td><td>1. Understand the problem statement and define the problem clearly.
2. Develop the solution using proper logic and programming concepts.
3. Implement all required features and pass the given test cases.
4. Test and debug the project to ensure correct output.
5. Prepare the documentation/report and upload the code and report to GitHub.
6. Complete and submit the project within the given deadline.</td></tr>
</table>

BACHELOR OF COMPUTER APPLICATION

<table>
<tr><td>PS No.</td><td>51</td></tr>
<tr><td>Project Title</td><td>Digital Diary / Notes Manager</td></tr>
<tr><td>Problem Statement</td><td>Develop a console diary application where the user can add dated notes, view all notes, search notes by a keyword and delete notes. Notes are saved to a text file.
Language: Python 3 Concepts: Strings, Files, Search, Functions
Note: Notes are saved in diary.txt, each with a date (YYYY-MM-DD) and text. Note numbers start from 1. Search ignores upper and lower case. Menu: 1 Add, 2 View all, 3 Search, 4 Delete.</td></tr>
<tr><td>Test Cases / Case Statements (4 cases)</td><td>
<table>
<tr><th>INPUT</th><th>EXPECTED OUTPUT</th></tr>
<tr><td colspan="2">Test Case 1: Normal Input</td></tr>
<tr><td>Menu option 1 (Add)
Date: 2026-10-10
Text: Started Python project</td><td>Note saved.
1 | 2026-10-10 | Started Python project</td></tr>
<tr><td colspan="2">Test Case 2: Boundary / Minimum Input</td></tr>
<tr><td>Diary is empty
Menu option 2 (View all)</td><td>No notes found. Your diary is empty.</td></tr>
<tr><td colspan="2">Test Case 3: Invalid Input</td></tr>
<tr><td>(a) Add: Date 2026-10-10, Text: (empty)
(b) Add: Date 10/10/2026
(c) Option 4 (Delete): note 9</td><td>(a) Note text cannot be empty.
(b) Invalid date. Use YYYY-MM-DD.
(c) Note number 9 not found.</td></tr>
<tr><td colspan="2">Test Case 4: Different Valid Scenario</td></tr>
<tr><td>Notes: 1 (2026-10-10) Started Python project, 2 (2026-10-11) Python exam on Monday, 3 (2026-10-12) Buy groceries
(a) Option 3 (Search): python
(b) Option 4 (Delete): 3</td><td>(a) Found 2 notes:
1 | 2026-10-10 | Started Python project
2 | 2026-10-11 | Python exam on Monday
(b) Note 3 deleted. Notes left: 2</td></tr>
</table>
</td></tr>
<tr><td>Bare Minimum Requirements</td><td>1. Write the solution in Python 3 (one .py file) and run it from the console.
2. Save and load the notes using a text file with proper file handling.
3. Write functions to add, view, search and delete notes.
4. Use string methods (lower, strip, find) to search the notes by keyword.
5. Validate the date format, the note text and the note number.</td></tr>
<tr><td>Instruction</td><td>1. Understand the problem statement and define the problem clearly.
2. Develop the solution using proper logic and programming concepts.
3. Implement all required features and pass the given test cases.
4. Test and debug the project to ensure correct output.
5. Prepare the documentation/report and upload the code and report to GitHub.
6. Complete and submit the project within the given deadline.</td></tr>
</table>

BACHELOR OF COMPUTER APPLICATION

<table>
<tr><td>PS No.</td><td>52</td></tr>
<tr><td>Project Title</td><td>Rock Paper Scissors</td></tr>
<tr><td>Problem Statement</td><td>Create a console game where the player plays three rounds of Rock, Paper, Scissors against the computer, the winner of each round is shown, and the final result of the match is declared.
Language: Python 3 Concepts: Conditions, Functions, Random
Note: Rock beats Scissors, Scissors beats Paper, Paper beats Rock. For testing, the computer's choices in the 3 rounds are fixed as Scissors, Rock, Paper.</td></tr>
<tr><td>Test Cases / Case Statements (4 cases)</td><td>
<table>
<tr><th>INPUT</th><th>EXPECTED OUTPUT</th></tr>
<tr><td colspan="2">Test Case 1: Normal Input</td></tr>
<tr><td>Player: rock, rock, rock</td><td>Round 1: You win │ Round 2: Draw │ Round 3: Computer wins
You 1 │ Computer 1 │ Draws 1
Result: Match drawn.</td></tr>
<tr><td colspan="2">Test Case 2: Boundary / Minimum Input</td></tr>
<tr><td>Player: rock, paper, scissors</td><td>Round 1: You win │ Round 2: You win │ Round 3: You win
You 3 │ Computer 0 │ Draws 0
Result: You win the match!</td></tr>
<tr><td colspan="2">Test Case 3: Invalid Input</td></tr>
<tr><td>Round 1 input: stone, then rock</td><td>Invalid choice. Enter rock, paper or scissors.
Round 1: You win (no round lost)</td></tr>
<tr><td colspan="2">Test Case 4: Different Valid Scenario</td></tr>
<tr><td>Player: scissors, scissors, rock
Play again? n</td><td>Round 1: Draw │ Round 2: Computer wins │ Round 3: Computer wins
You 0 │ Computer 2 │ Draws 1
Result: Computer wins the match.
Thanks for playing!</td></tr>
</table>
</td></tr>
<tr><td>Bare Minimum Requirements</td><td>1. Write the solution in Python 3 (one .py file) and run it from the console.
2. Use the random module to make the computer's choice.
3. Write a function that compares the two choices using conditions and returns the winner.
4. Use a loop for the 3 rounds, keep the score and declare the match result.
5. Validate the player's input and ask 'Play again? (y/n)' at the end.</td></tr>
<tr><td>Instruction</td><td>1. Understand the problem statement and define the problem clearly.
2. Develop the solution using proper logic and programming concepts.
3. Implement all required features and pass the given test cases.
4. Test and debug the project to ensure correct output.
5. Prepare the documentation/report and upload the code and report to GitHub.
6. Complete and submit the project within the given deadline.</td></tr>
</table>

BACHELOR OF COMPUTER APPLICATION

<table>
<tr><td>PS No.</td><td>53</td></tr>
<tr><td>Project Title</td><td>Dice Rolling Game</td></tr>
<tr><td>Problem Statement</td><td>Build a program that rolls a die a number of times chosen by the user, shows the frequency of each face, the total, the average and the most frequent face.
Language: Python 3 Concepts: Random, Loops, Functions, Statistics
Note: The number of rolls must be from 1 to 1000. A die has faces 1 to 6. For testing, the rolls are fixed as given in each INPUT.</td></tr>
<tr><td>Test Cases / Case Statements (4 cases)</td><td>
<table>
<tr><th>INPUT</th><th>EXPECTED OUTPUT</th></tr>
<tr><td colspan="2">Test Case 1: Normal Input</td></tr>
<tr><td>Number of rolls: 5
Rolls: 3, 6, 3, 1, 5</td><td>Frequency: 1:1 | 2:0 | 3:2 | 4:0 | 5:1 | 6:1
Total: 18 | Average: 3.60
Most frequent: 3 (2 times)</td></tr>
<tr><td colspan="2">Test Case 2: Boundary / Minimum Input</td></tr>
<tr><td>Number of rolls: 1
Roll: 6</td><td>Frequency: 1:0 | 2:0 | 3:0 | 4:0 | 5:0 | 6:1
Total: 6 | Average: 6.00
Most frequent: 6 (1 time)</td></tr>
<tr><td colspan="2">Test Case 3: Invalid Input</td></tr>
<tr><td>Number of rolls: 0, then abc, then -3, then 2</td><td>Invalid input. Enter a whole number from 1 to 1000.
(shown for each wrong entry, then 2 rolls are accepted)</td></tr>
<tr><td colspan="2">Test Case 4: Different Valid Scenario</td></tr>
<tr><td>Number of rolls: 6
Rolls: 2, 2, 4, 4, 5, 1
Play again? n</td><td>Frequency: 1:1 | 2:2 | 3:0 | 4:2 | 5:1 | 6:0
Total: 18 | Average: 3.00
Most frequent: 2 and 4 (2 times each)
Thanks for playing!</td></tr>
</table>
</td></tr>
<tr><td>Bare Minimum Requirements</td><td>1. Write the solution in Python 3 (one .py file) and run it from the console.
2. Use the random module and a loop to roll the die.
3. Store the frequency of each face in a dictionary or list.
4. Write functions to roll the die, calculate the statistics and display the results.
5. Validate the number of rolls and ask 'Play again? (y/n)' at the end.</td></tr>
<tr><td>Instruction</td><td>1. Understand the problem statement and define the problem clearly.
2. Develop the solution using proper logic and programming concepts.
3. Implement all required features and pass the given test cases.
4. Test and debug the project to ensure correct output.
5. Prepare the documentation/report and upload the code and report to GitHub.
6. Complete and submit the project within the given deadline.</td></tr>
</table>

BACHELOR OF COMPUTER APPLICATION

<table>
<tr><td>PS No.</td><td>54</td></tr>
<tr><td>Project Title</td><td>Hangman Game</td></tr>
<tr><td>Problem Statement</td><td>Create a word guessing game in which the player guesses the letters of a hidden word one at a time with a limited number of wrong guesses.
Language: Python 3 Concepts: Strings, Lists, Loops, Random
Note: The player gets 6 wrong guesses. The word is chosen randomly from a list; for testing the secret word is fixed to 'python'. Letters are not case-sensitive.</td></tr>
<tr><td>Test Cases / Case Statements (4 cases)</td><td>
<table>
<tr><th>INPUT</th><th>EXPECTED OUTPUT</th></tr>
<tr><td colspan="2">Test Case 1: Normal Input</td></tr>
<tr><td>Guesses: p, y, t, h, o, n</td><td>Progress: p _ _ _ _ _ → p y _ _ _ _ → p y t _ _ _ → p y t h _ _ → p y t h o _
You won! The word was python.
Wrong guesses: 0/6</td></tr>
<tr><td colspan="2">Test Case 2: Boundary / Minimum Input</td></tr>
<tr><td>Guesses: a, b, c, d, e, f</td><td>Wrong guess (1/6) ... Wrong guess (6/6)
Game Over! The word was python.</td></tr>
<tr><td colspan="2">Test Case 3: Invalid Input</td></tr>
<tr><td>Guesses: 1, then ab, then p, then p again</td><td>Invalid input. Enter a single letter.
Invalid input. Enter a single letter.
p: Correct
You already guessed 'p'. Try another letter. (no attempt lost)</td></tr>
<tr><td colspan="2">Test Case 4: Different Valid Scenario</td></tr>
<tr><td>Guesses: p, z, y, x, t, h, o, n</td><td>p: Correct │ z: Wrong (1/6) │ y: Correct │ x: Wrong (2/6)
t, h, o, n: Correct
You won! The word was python.
Wrong guesses: 2/6</td></tr>
</table>
</td></tr>
<tr><td>Bare Minimum Requirements</td><td>1. Write the solution in Python 3 (one .py file) and run it from the console.
2. Keep a list of words and choose the secret word using the random module.
3. Use a loop to take guesses and show the word with unguessed letters as underscores.
4. Track the correct and wrong guesses, and end the game on a win or after 6 wrong guesses.
5. Validate the input (a single letter only) and do not count a repeated letter as a wrong guess.</td></tr>
<tr><td>Instruction</td><td>1. Understand the problem statement and define the problem clearly.
2. Develop the solution using proper logic and programming concepts.
3. Implement all required features and pass the given test cases.
4. Test and debug the project to ensure correct output.
5. Prepare the documentation/report and upload the code and report to GitHub.
6. Complete and submit the project within the given deadline.</td></tr>
</table>

BACHELOR OF COMPUTER APPLICATION

<table>
<tr><td>PS No.</td><td>55</td></tr>
<tr><td>Project Title</td><td>Tic-Tac-Toe</td></tr>
<tr><td>Problem Statement</td><td>Develop a two-player console Tic-Tac-Toe game on a 3 × 3 board where players take turns, the board is displayed after every move, and the winner or a draw is announced.
Language: Python 3 Concepts: Lists, Functions, Conditions, Game Logic
Note: Positions are numbered 1 to 9 (1-2-3 top row, 4-5-6 middle row, 7-8-9 bottom row). Player X always plays first.</td></tr>
<tr><td>Test Cases / Case Statements (4 cases)</td><td>
<table>
<tr><th>INPUT</th><th>EXPECTED OUTPUT</th></tr>
<tr><td colspan="2">Test Case 1: Normal Input</td></tr>
<tr><td>Moves: X:1, O:4, X:2, O:5, X:3</td><td>Player X wins!
Winning line: 1-2-3</td></tr>
<tr><td colspan="2">Test Case 2: Boundary / Minimum Input</td></tr>
<tr><td>Moves: X:1, O:2, X:3, O:5, X:4, O:6, X:8, O:7, X:9</td><td>Board is full after 9 moves.
It's a draw! No moves left.</td></tr>
<tr><td colspan="2">Test Case 3: Invalid Input</td></tr>
<tr><td>(a) Position: 10
(b) Position: abc
(c) X plays 1, then O plays 1</td><td>(a) Invalid position. Enter a number from 1 to 9.
(b) Invalid position. Enter a number from 1 to 9.
(c) Position 1 is already taken. Choose another position.
(turn does not change)</td></tr>
<tr><td colspan="2">Test Case 4: Different Valid Scenario</td></tr>
<tr><td>Moves: X:1, O:4, X:2, O:5, X:9, O:6
Play again? n</td><td>Player O wins!
Winning line: 4-5-6
Thanks for playing!</td></tr>
</table>
</td></tr>
<tr><td>Bare Minimum Requirements</td><td>1. Write the solution in Python 3 (one .py file) and run it from the console.
2. Store the board in a list of 9 cells and display it after every move.
3. Write functions to display the board, take a move, check for a win and check for a draw.
4. Check all 8 winning lines (3 rows, 3 columns, 2 diagonals) using conditions.
5. Validate every move and ask 'Play again? (y/n)' at the end.</td></tr>
<tr><td>Instruction</td><td>1. Understand the problem statement and define the problem clearly.
2. Develop the solution using proper logic and programming concepts.
3. Implement all required features and pass the given test cases.
4. Test and debug the project to ensure correct output.
5. Prepare the documentation/report and upload the code and report to GitHub.
6. Complete and submit the project within the given deadline.</td></tr>
</table>

BACHELOR OF COMPUTER APPLICATION

<table>
<tr><td>PS No.</td><td>56</td></tr>
<tr><td>Project Title</td><td>Snake Game</td></tr>
<tr><td>Problem Statement</td><td>Create the classic Snake game with Pygame in which the player steers a snake to eat food, the snake grows and the score increases, and the game ends when the snake hits the wall or itself.
Language: Python 3 Concepts: Loops, Lists, Coordinates, Pygame
Note: Grid is 10 × 10 (x and y from 0 to 9). The snake starts as one segment at (5, 5) moving Right; U decreases y. Eating food adds 10 to the score and 1 to the length. The game ends on a wall or self collision. For testing, the first food is fixed at (7, 5), the next at (0, 0), and the moves are given as keys R, L, U, D.</td></tr>
<tr><td>Test Cases / Case Statements (4 cases)</td><td>
<table>
<tr><th>INPUT</th><th>EXPECTED OUTPUT</th></tr>
<tr><td colspan="2">Test Case 1: Normal Input</td></tr>
<tr><td>Moves: R, R</td><td>Head: (6, 5)
Food eaten at (7, 5)!
Score: 10 │ Length: 2 │ Head: (7, 5)</td></tr>
<tr><td colspan="2">Test Case 2: Boundary / Minimum Input</td></tr>
<tr><td>Moves: R, R, R, R, R</td><td>Food eaten at (7, 5)! Score: 10
Head: (8, 5), then (9, 5)
Game Over! Wall hit at (10, 5).
Final score: 10</td></tr>
<tr><td colspan="2">Test Case 3: Invalid Input</td></tr>
<tr><td>(a) Key: X
(b) Key: L (snake is moving Right)</td><td>(a) Invalid key. Use R, L, U or D.
(b) Reverse move not allowed. Snake continues Right.</td></tr>
<tr><td colspan="2">Test Case 4: Different Valid Scenario</td></tr>
<tr><td>Moves: R, R, U, U</td><td>Food eaten at (7, 5)! Score: 10
Snake: (7, 3), (7, 4)
Length: 2 │ Head: (7, 3)</td></tr>
</table>
</td></tr>
<tr><td>Bare Minimum Requirements</td><td>1. Write the solution in Python 3 (one .py file) and run it from the console. Use the pygame library for the game window.
2. Store the snake as a list of coordinates and move it one cell at a time in the game loop.
3. Generate the food randomly, grow the snake and increase the score when the food is eaten.
4. Detect wall collision and self collision to end the game and show Game Over with the final score.
5. Control the snake with the arrow keys and do not allow an immediate reverse move.</td></tr>
<tr><td>Instruction</td><td>1. Understand the problem statement and define the problem clearly.
2. Develop the solution using proper logic and programming concepts.
3. Implement all required features and pass the given test cases.
4. Test and debug the project to ensure correct output.
5. Prepare the documentation/report and upload the code and report to GitHub.
6. Complete and submit the project within the given deadline.</td></tr>
</table>

<table>
<tr><td>PS No.</td><td>57</td></tr>
<tr><td>Project Title</td><td>Car Racing Game</td></tr>
<tr><td>Problem Statement</td><td>Create a 2D car racing game in which the player steers a car left and right on a road using the arrow keys, avoids obstacle cars coming down from the top, and scores a point for every obstacle passed. The game ends when the player's car collides with an obstacle.
Language: Python 3 Concepts: Pygame, Collision Detection, Game Loop
Note: Window 400x600; road spans x = 50 to 350. Player car is 40x80 at y = 500 and moves 10 px per key press. Obstacle cars are 40x80. Score +1 for each obstacle passed. Positions given as (x, y) = top-left corner.</td></tr>
<tr><td>Test Cases / Case Statements (4 cases)</td><td>
<table>
<tr><th>INPUT</th><th>EXPECTED OUTPUT</th></tr>
<tr><td colspan="2">Test Case 1: Normal Input</td></tr>
<tr><td>Player car x = 180
Key pressed: RIGHT arrow</td><td>Player car x = 190 | Score: 0 | Game running</td></tr>
<tr><td colspan="2">Test Case 2: Boundary / Minimum Input</td></tr>
<tr><td>(a) Player car x = 50 (left edge of road), key: LEFT arrow
(b) Player car x = 310 (right edge of road), key: RIGHT arrow</td><td>(a) Player car x = 50 (no movement, stays on road)
(b) Player car x = 310 (no movement, stays on road)</td></tr>
<tr><td colspan="2">Test Case 3: Invalid Input</td></tr>
<tr><td>Player car x = 180
Key pressed: SPACE (not a control key)</td><td>Key ignored. Player car x = 180 | Game running</td></tr>
<tr><td colspan="2">Test Case 4: Different Valid Scenario</td></tr>
<tr><td>Starting score: 3
(a) Obstacle at (300, 600) leaves the screen
(b) Obstacle at (200, 520) while player car is at (180, 500)</td><td>(a) Obstacle passed. Score: 4
(b) Collision detected | Game Over | Final Score: 4</td></tr>
</table>
</td></tr>
<tr><td>Bare Minimum Requirements</td><td>1. Write the solution in Python 3 (one .py file) using Pygame and run it from the console.
2. Create a 400x600 window with a game loop running at 60 FPS; draw the road, player car and obstacles.
3. Move the player car left/right with the arrow keys and keep it inside the road.
4. Spawn obstacle cars at random x positions moving downwards; add 1 to the score for each obstacle passed.
5. Detect collisions using Rect.colliderect(), then show 'Game Over' with the final score and allow restart with the R key.</td></tr>
<tr><td>Instruction</td><td>1. Understand the problem statement and define the problem clearly.
2. Develop the solution using proper logic and programming concepts.
3. Implement all required features and pass the given test cases.
4. Test and debug the project to ensure correct output.
5. Prepare the documentation/report and upload the code and report to GitHub.
6. Complete and submit the project within the given deadline.</td></tr>
</table>

BACHELOR OF COMPUTER APPLICATION

<table>
<tr><td>PS No.</td><td>58</td></tr>
<tr><td>Project Title</td><td>Space Shooter</td></tr>
<tr><td>Problem Statement</td><td>Build a space shooter game in which the player's spaceship moves left and right at the bottom of the screen and fires bullets upward to destroy enemy ships descending from the top. The player earns points for each hit and loses a life when an enemy reaches the bottom.
Language: Python 3 Concepts: Pygame, Objects, Collision Detection
Note: Window 600x600. Ship is 50x50 at y = 520 and moves 8 px per key press. Bullet is 5x10 and moves 10 px per frame upward; maximum 5 bullets on screen. Enemy is 40x40. Hit = +10 score; an enemy reaching y = 600 costs 1 life (start with 3 lives).</td></tr>
<tr><td>Test Cases / Case Statements (4 cases)</td><td>
<table>
<tr><th>INPUT</th><th>EXPECTED OUTPUT</th></tr>
<tr><td colspan="2">Test Case 1: Normal Input</td></tr>
<tr><td>Ship x = 275, no bullets on screen
Key pressed: SPACE</td><td>Bullet fired at (300, 520) | Bullets on screen: 1</td></tr>
<tr><td colspan="2">Test Case 2: Boundary / Minimum Input</td></tr>
<tr><td>(a) Ship x = 0, key: LEFT arrow
(b) Ship x = 550, key: RIGHT arrow</td><td>(a) Ship x = 0 (no movement, stays inside window)
(b) Ship x = 550 (no movement, stays inside window)</td></tr>
<tr><td colspan="2">Test Case 3: Invalid Input</td></tr>
<tr><td>5 bullets already on screen
Key pressed: SPACE</td><td>Maximum 5 bullets on screen. No bullet fired.
Bullets on screen: 5</td></tr>
<tr><td colspan="2">Test Case 4: Different Valid Scenario</td></tr>
<tr><td>(a) Bullet at (300, 100) and enemy at (280, 90)
(b) Enemy reaches y = 600, Lives before: 3</td><td>(a) Enemy destroyed | Score: 10 | Lives: 3
(b) Enemy escaped | Lives: 2</td></tr>
</table>
</td></tr>
<tr><td>Bare Minimum Requirements</td><td>1. Write the solution in Python 3 (one .py file) using Pygame and run it from the console.
2. Create Ship, Bullet and Enemy as classes (objects) with position, size and speed.
3. Move the ship with LEFT/RIGHT arrows, fire bullets with SPACE and limit the bullets on screen to 5.
4. Detect bullet-enemy collisions using Rect.colliderect(); add 10 to the score for every enemy destroyed.
5. Reduce a life when an enemy reaches the bottom; show 'Game Over' with the final score when lives reach 0.</td></tr>
<tr><td>Instruction</td><td>1. Understand the problem statement and define the problem clearly.
2. Develop the solution using proper logic and programming concepts.
3. Implement all required features and pass the given test cases.
4. Test and debug the project to ensure correct output.
5. Prepare the documentation/report and upload the code and report to GitHub.
6. Complete and submit the project within the given deadline.</td></tr>
</table>

BACHELOR OF COMPUTER APPLICATION

<table>
<tr><td>PS No.</td><td>59</td></tr>
<tr><td>Project Title</td><td>Maze Escape Game</td></tr>
<tr><td>Problem Statement</td><td>Develop a console-based maze game in which the player moves through a maze stored as a 2D list using W/A/S/D keys and must reach the exit. Walls block movement, and the program also finds the shortest path from start to exit using a search algorithm.
Language: Python 3 Concepts: 2D Lists, Algorithms, Movement
Note: Maze (5x5): row 0 = P..## | row 1 = ##.## | row 2 = #...# | row 3 = #.### | row 4 = #...E (# wall, . path, P start at row 0 col 0, E exit). Only valid moves are counted.</td></tr>
<tr><td>Test Cases / Case Statements (4 cases)</td><td>
<table>
<tr><th>INPUT</th><th>EXPECTED OUTPUT</th></tr>
<tr><td colspan="2">Test Case 1: Normal Input</td></tr>
<tr><td>Move: D</td><td>Player moved to (row 0, col 1) | Moves: 1</td></tr>
<tr><td colspan="2">Test Case 2: Boundary / Minimum Input</td></tr>
<tr><td>Player at (0, 0)
Move: W (up, outside the maze)</td><td>Cannot move outside the maze. Player stays at (0, 0) | Moves: 0</td></tr>
<tr><td colspan="2">Test Case 3: Invalid Input</td></tr>
<tr><td>Moves: D, D, D, X</td><td>Wall ahead. Move blocked. Player stays at (0, 2)
Invalid key 'X'. Use W, A, S or D only.
Moves counted: 2</td></tr>
<tr><td colspan="2">Test Case 4: Different Valid Scenario</td></tr>
<tr><td>(a) Moves: D, D, S, S, A, S, S, D, D, D
(b) Menu option 2 (Shortest path)</td><td>(a) You escaped the maze! Total moves: 10
(b) Shortest path length: 10 steps</td></tr>
</table>
</td></tr>
<tr><td>Bare Minimum Requirements</td><td>1. Write the solution in Python 3 (one .py file) and run it from the console.
2. Store the maze as a 2D list (list of lists) and print it after every move with the player's position.
3. Move the player with W/A/S/D; block moves into walls or outside the maze and do not count them.
4. Show a winning message with the total number of moves when the player reaches E.
5. Implement BFS (or DFS) to find and display the shortest path length from start to exit.</td></tr>
<tr><td>Instruction</td><td>1. Understand the problem statement and define the problem clearly.
2. Develop the solution using proper logic and programming concepts.
3. Implement all required features and pass the given test cases.
4. Test and debug the project to ensure correct output.
5. Prepare the documentation/report and upload the code and report to GitHub.
6. Complete and submit the project within the given deadline.</td></tr>
</table>

BACHELOR OF COMPUTER APPLICATION

<table>
<tr><td>PS No.</td><td>60</td></tr>
<tr><td>Project Title</td><td>Flappy Bird Clone</td></tr>
<tr><td>Problem Statement</td><td>Create a Flappy Bird style game in which a bird constantly falls due to gravity and flies upward when the player presses SPACE. The player must pass through gaps between pipes to score, and the game ends if the bird hits a pipe, the ground or leaves the top of the screen.
Language: Python 3 Concepts: Pygame, Gravity, Collision, Game Loop
Note: Window 400x600. Bird is 30x30 at x = 100 and starts at y = 300. Each frame: velocity += 1, then y += velocity. SPACE sets velocity = -10 before the frame update. Pipes are 60 px wide with a 150 px gap; ground is at y = 600.</td></tr>
<tr><td>Test Cases / Case Statements (4 cases)</td><td>
<table>
<tr><th>INPUT</th><th>EXPECTED OUTPUT</th></tr>
<tr><td colspan="2">Test Case 1: Normal Input</td></tr>
<tr><td>Bird y = 300, velocity = 0
No key pressed, 2 frames</td><td>Frame 1: velocity = 1 | y = 301
Frame 2: velocity = 2 | y = 303</td></tr>
<tr><td colspan="2">Test Case 2: Boundary / Minimum Input</td></tr>
<tr><td>(a) Bird y = 570 (bottom edge touches y = 600)
(b) Bird y = 5, velocity = 0, SPACE pressed</td><td>(a) Collision with ground | Game Over
(b) velocity = -9 | y = -4 | Bird left the screen at the top | Game Over</td></tr>
<tr><td colspan="2">Test Case 3: Invalid Input</td></tr>
<tr><td>Bird y = 300, velocity = 0
Key pressed: Z (not SPACE), 1 frame</td><td>Key ignored | velocity = 1 | y = 301</td></tr>
<tr><td colspan="2">Test Case 4: Different Valid Scenario</td></tr>
<tr><td>(a) Bird y = 300, velocity = 5, SPACE pressed, 1 frame
(b) Pipe at x = 39 (width 60), bird x = 100, score = 0</td><td>(a) velocity = -9 | y = 291 (bird moves up)
(b) Pipe passed | Score: 1</td></tr>
</table>
</td></tr>
<tr><td>Bare Minimum Requirements</td><td>1. Write the solution in Python 3 (one .py file) using Pygame and run it from the console.
2. Apply gravity to the bird every frame and make it flap upward when SPACE is pressed.
3. Generate pipes with a random gap position that move from right to left; remove pipes that leave the screen.
4. Add 1 to the score each time the bird passes a pipe and display it on screen.
5. Detect collision with pipes, ground and top edge using Rect.colliderect(); show 'Game Over' and allow restart.</td></tr>
<tr><td>Instruction</td><td>1. Understand the problem statement and define the problem clearly.
2. Develop the solution using proper logic and programming concepts.
3. Implement all required features and pass the given test cases.
4. Test and debug the project to ensure correct output.
5. Prepare the documentation/report and upload the code and report to GitHub.
6. Complete and submit the project within the given deadline.</td></tr>
</table>

BACHELOR OF COMPUTER APPLICATION

<table>
<tr><td>PS No.</td><td>61</td></tr>
<tr><td>Project Title</td><td>Catch the Falling Objects</td></tr>
<tr><td>Problem Statement</td><td>Build a game in which objects fall from random positions at the top of the screen and the player moves a basket left and right to catch them. Each catch adds to the score, and every missed object costs a life.
Language: Python 3 Concepts: Pygame, Coordinates, Score, Random
Note: Window 500x600. Basket is 80x20 at y = 560 and moves 10 px per key press. Object is 20x20, falls 5 px per frame from a random x (random.randint(0, 480)). Start with 3 lives. Test cases use fixed positions (x, y = top-left corner).</td></tr>
<tr><td>Test Cases / Case Statements (4 cases)</td><td>
<table>
<tr><th>INPUT</th><th>EXPECTED OUTPUT</th></tr>
<tr><td colspan="2">Test Case 1: Normal Input</td></tr>
<tr><td>Basket x = 210
Object at (230, 545)</td><td>Object caught | Score: 1 | Lives: 3</td></tr>
<tr><td colspan="2">Test Case 2: Boundary / Minimum Input</td></tr>
<tr><td>(a) Basket x = 0, key: LEFT arrow
(b) Basket x = 420, key: RIGHT arrow</td><td>(a) Basket x = 0 (no movement, stays inside window)
(b) Basket x = 420 (no movement, stays inside window)</td></tr>
<tr><td colspan="2">Test Case 3: Invalid Input</td></tr>
<tr><td>Basket x = 210
Key pressed: UP arrow (not a control key)</td><td>Key ignored. Basket x = 210</td></tr>
<tr><td colspan="2">Test Case 4: Different Valid Scenario</td></tr>
<tr><td>(a) Basket x = 210, object falls past y = 600 at x = 400, Lives before: 3
(b) Same miss with Lives before: 1, Score: 7</td><td>(a) Object missed | Lives: 2
(b) Object missed | Lives: 0 | Game Over | Final Score: 7</td></tr>
</table>
</td></tr>
<tr><td>Bare Minimum Requirements</td><td>1. Write the solution in Python 3 (one .py file) using Pygame and run it from the console.
2. Move the basket with LEFT/RIGHT arrows and keep it inside the window.
3. Create falling objects at random x positions using the random module and move them down every frame.
4. Add 1 to the score when an object is caught (Rect collision) and reduce a life when it is missed.
5. Display score and lives on screen; show 'Game Over' with the final score when lives reach 0.</td></tr>
<tr><td>Instruction</td><td>1. Understand the problem statement and define the problem clearly.
2. Develop the solution using proper logic and programming concepts.
3. Implement all required features and pass the given test cases.
4. Test and debug the project to ensure correct output.
5. Prepare the documentation/report and upload the code and report to GitHub.
6. Complete and submit the project within the given deadline.</td></tr>
</table>

BACHELOR OF COMPUTER APPLICATION

<table>
<tr><td>PS No.</td><td>62</td></tr>
<tr><td>Project Title</td><td>Hospital Management System</td></tr>
<tr><td>Problem Statement</td><td>Develop a menu-driven console application to manage hospital patient records. The user can add a new patient, view all patients, search by ID, update details and delete a patient record.
Language: Python 3 Concepts: Dictionaries, Functions, Records, CRUD
Note: Patient IDs start from P101 and increase by 1. Fields: Name, Age, Disease, Doctor. Age must be 0 to 120. Data is kept in memory while the program runs (no file).</td></tr>
<tr><td>Test Cases / Case Statements (4 cases)</td><td>
<table>
<tr><th>INPUT</th><th>EXPECTED OUTPUT</th></tr>
<tr><td colspan="2">Test Case 1: Normal Input</td></tr>
<tr><td>Menu option 1 (Add)
Name: Ravi Kumar
Age: 35
Disease: Fever
Doctor: Dr. Mehta</td><td>Patient added. ID: P101 | Ravi Kumar | 35 | Fever | Dr. Mehta</td></tr>
<tr><td colspan="2">Test Case 2: Boundary / Minimum Input</td></tr>
<tr><td>Menu option 1 (Add)
Name: Baby Sen
Age: 0 (minimum age)
Disease: Jaundice
Doctor: Dr. Rao</td><td>Patient added. ID: P101 | Baby Sen | 0 | Jaundice | Dr. Rao</td></tr>
<tr><td colspan="2">Test Case 3: Invalid Input</td></tr>
<tr><td>Menu option 1 (Add)
Name: Amit Das
Age: abc</td><td>Invalid age. Enter a number between 0 and 120. Patient not added.</td></tr>
<tr><td colspan="2">Test Case 4: Different Valid Scenario</td></tr>
<tr><td>Patient P101 (Ravi Kumar, Fever) exists
(a) Menu option 4 (Update): ID P101, new Disease: Typhoid
(b) Menu option 5 (Delete): ID P101
(c) Menu option 2 (View all)</td><td>(a) Patient P101 updated. Disease: Typhoid
(b) Patient P101 deleted. Total patients: 0
(c) No patient records found.</td></tr>
</table>
</td></tr>
<tr><td>Bare Minimum Requirements</td><td>1. Write the solution in Python 3 (one .py file) and run it from the console.
2. Store each patient as a dictionary inside a main dictionary keyed by patient ID.
3. Write separate functions for add, view, search, update and delete (CRUD).
4. Validate the name (not empty) and age (0 to 120) before saving a record.
5. Show clear messages when an ID is not found or when there are no records.</td></tr>
<tr><td>Instruction</td><td>1. Understand the problem statement and define the problem clearly.
2. Develop the solution using proper logic and programming concepts.
3. Implement all required features and pass the given test cases.
4. Test and debug the project to ensure correct output.
5. Prepare the documentation/report and upload the code and report to GitHub.
6. Complete and submit the project within the given deadline.</td></tr>
</table>

BACHELOR OF COMPUTER APPLICATION

<table>
<tr><td>PS No.</td><td>63</td></tr>
<tr><td>Project Title</td><td>Digital Notes Management System</td></tr>
<tr><td>Problem Statement</td><td>Create a console application to write, view, search, edit and delete text notes. Each note has an ID, a title and its content, and all notes are saved permanently in a text file.
Language: Python 3 Concepts: Strings, Files, Search, CRUD
Note: Notes are stored in notes.txt, one note per line as ID|Title|Content. The first note ID is N1. Search is case-insensitive and checks both title and content.</td></tr>
<tr><td>Test Cases / Case Statements (4 cases)</td><td>
<table>
<tr><th>INPUT</th><th>EXPECTED OUTPUT</th></tr>
<tr><td colspan="2">Test Case 1: Normal Input</td></tr>
<tr><td>Menu option 1 (Add)
Title: Python
Content: Learn loops</td><td>Note saved. ID: N1 | Python</td></tr>
<tr><td colspan="2">Test Case 2: Boundary / Minimum Input</td></tr>
<tr><td>notes.txt is empty
Menu option 2 (View all)</td><td>No notes found.</td></tr>
<tr><td colspan="2">Test Case 3: Invalid Input</td></tr>
<tr><td>Menu option 1 (Add)
Title: (empty)
Content: Buy books</td><td>Title cannot be empty. Note not saved.</td></tr>
<tr><td colspan="2">Test Case 4: Different Valid Scenario</td></tr>
<tr><td>notes.txt contains:
N1|Python|Learn loops
N2|Maths|Revise algebra
(a) Option 3 (Search): Keyword: python
(b) Option 4 (Edit): ID N2, new content: Revise calculus
(c) Option 5 (Delete): ID N1</td><td>(a) N1 | Python | Learn loops
(b) Note N2 updated.
(c) Note N1 deleted. Notes left: 1</td></tr>
</table>
</td></tr>
<tr><td>Bare Minimum Requirements</td><td>1. Write the solution in Python 3 (one .py file) and run it from the console.
2. Save every note in notes.txt in the format ID|Title|Content and load it again at program start.
3. Use string methods (split, strip, lower, in) to read and search the notes.
4. Provide add, view, search, edit and delete options through functions.
5. Reject empty titles and show a message when a note ID does not exist.</td></tr>
<tr><td>Instruction</td><td>1. Understand the problem statement and define the problem clearly.
2. Develop the solution using proper logic and programming concepts.
3. Implement all required features and pass the given test cases.
4. Test and debug the project to ensure correct output.
5. Prepare the documentation/report and upload the code and report to GitHub.
6. Complete and submit the project within the given deadline.</td></tr>
</table>

BACHELOR OF COMPUTER APPLICATION

<table>
<tr><td>PS No.</td><td>64</td></tr>
<tr><td>Project Title</td><td>Student Fee Management System</td></tr>
<tr><td>Problem Statement</td><td>Build a console application to manage student fee records. The user can register a student with a course, accept full or partial fee payments, check the fee status and list students having pending dues. Records are saved in a file.
Language: Python 3 Concepts: Dictionaries, Functions, Conditions, File Handling
Note: Course fees (dictionary): BCA 45000, BTech 60000, MBA 80000. Records are stored in fees.txt as Roll,Name,Course,Paid. Status: Paid (due = 0), Partial (0 < paid < total), Unpaid (paid = 0).</td></tr>
<tr><td>Test Cases / Case Statements (4 cases)</td><td>
<table>
<tr><th>INPUT</th><th>EXPECTED OUTPUT</th></tr>
<tr><td colspan="2">Test Case 1: Normal Input</td></tr>
<tr><td>Menu option 1 (Add)
Roll: 101
Name: Riya
Course: BCA</td><td>Student added: 101 | Riya | BCA | Total Fee: 45000 | Paid: 0 | Due: 45000 | Status: Unpaid</td></tr>
<tr><td colspan="2">Test Case 2: Boundary / Minimum Input</td></tr>
<tr><td>Student 101 (BCA) exists, Paid: 0
Menu option 2 (Pay)
Roll: 101
Amount: 45000 (equal to due)</td><td>Payment received. Paid: 45000 | Due: 0 | Status: Paid</td></tr>
<tr><td colspan="2">Test Case 3: Invalid Input</td></tr>
<tr><td>Student 101 (BCA) exists, Paid: 0
Menu option 2 (Pay)
Roll: 101
Amount: 50000</td><td>Amount exceeds due (45000). Payment rejected.</td></tr>
<tr><td colspan="2">Test Case 4: Different Valid Scenario</td></tr>
<tr><td>Student 102 (Aman, BTech) exists, Paid: 0
(a) Menu option 2 (Pay): Roll 102, Amount: 20000
(b) Menu option 4 (Pending dues)</td><td>(a) Payment received. Paid: 20000 | Due: 40000 | Status: Partial
(b) 102 | Aman | Due: 40000</td></tr>
</table>
</td></tr>
<tr><td>Bare Minimum Requirements</td><td>1. Write the solution in Python 3 (one .py file) and run it from the console.
2. Keep the course-fee table in a dictionary and store each student as a dictionary inside a list.
3. Use conditions to calculate due amount and status; reject payments that are zero, negative or above the due amount.
4. Provide add, pay, search by roll number and pending-dues list using functions.
5. Load records from fees.txt at start and save them back after every change.</td></tr>
<tr><td>Instruction</td><td>1. Understand the problem statement and define the problem clearly.
2. Develop the solution using proper logic and programming concepts.
3. Implement all required features and pass the given test cases.
4. Test and debug the project to ensure correct output.
5. Prepare the documentation/report and upload the code and report to GitHub.
6. Complete and submit the project within the given deadline.</td></tr>
</table>

BACHELOR OF COMPUTER APPLICATION

<table>
<tr><td>PS No.</td><td>65</td></tr>
<tr><td>Project Title</td><td>Vehicle Rental Management System</td></tr>
<tr><td>Problem Statement</td><td>Develop an object-oriented console application for a vehicle rental shop. The user can add vehicles, rent a vehicle to a customer for a number of days, return it and view available vehicles. The bill is calculated automatically and data is saved in a file.
Language: Python 3 Concepts: OOP, Lists, Dictionaries, File Handling
Note: Vehicle IDs start from V1. Bill = rate per day x days; a 10% discount is given for 7 or more days. Vehicles are stored in vehicles.txt. Days must be 1 or more.</td></tr>
<tr><td>Test Cases / Case Statements (4 cases)</td><td>
<table>
<tr><th>INPUT</th><th>EXPECTED OUTPUT</th></tr>
<tr><td colspan="2">Test Case 1: Normal Input</td></tr>
<tr><td>Menu option 1 (Add vehicle)
Type: Car
Model: Swift
Rate per day: 1500</td><td>Vehicle added. ID: V1 | Car | Swift | Rs. 1500/day | Available</td></tr>
<tr><td colspan="2">Test Case 2: Boundary / Minimum Input</td></tr>
<tr><td>V1 (Car, Rs. 1500/day) is available
Menu option 2 (Rent)
Vehicle ID: V1, Customer: Aman
Days: 7 (discount boundary)</td><td>V1 rented to Aman for 7 days. 10% discount applied. Bill: Rs. 9450</td></tr>
<tr><td colspan="2">Test Case 3: Invalid Input</td></tr>
<tr><td>V1 is already rented to Aman
Menu option 2 (Rent)
Vehicle ID: V1, Customer: Riya, Days: 2</td><td>Vehicle V1 is not available.</td></tr>
<tr><td colspan="2">Test Case 4: Different Valid Scenario</td></tr>
<tr><td>V2 (Bike, Splendor, Rs. 500/day) is available
(a) Menu option 2 (Rent): V2, Customer: Neha, Days: 3
(b) Menu option 3 (Return): V2</td><td>(a) V2 rented to Neha for 3 days. Bill: Rs. 1500
(b) V2 returned. Status: Available</td></tr>
</table>
</td></tr>
<tr><td>Bare Minimum Requirements</td><td>1. Write the solution in Python 3 (one .py file) and run it from the console.
2. Create a Vehicle class (id, type, model, rate, availability) and a Rental class or methods for rent and return.
3. Keep all vehicle objects in a list and use a dictionary to map vehicle ID to its rental details.
4. Calculate the bill with the 10% discount rule and block renting a vehicle that is already rented.
5. Save vehicles and their status in vehicles.txt and load them when the program starts.</td></tr>
<tr><td>Instruction</td><td>1. Understand the problem statement and define the problem clearly.
2. Develop the solution using proper logic and programming concepts.
3. Implement all required features and pass the given test cases.
4. Test and debug the project to ensure correct output.
5. Prepare the documentation/report and upload the code and report to GitHub.
6. Complete and submit the project within the given deadline.</td></tr>
</table>

BACHELOR OF COMPUTER APPLICATION

<table>
<tr><td>PS No.</td><td>66</td></tr>
<tr><td>Project Title</td><td>Online Examination System</td></tr>
<tr><td>Problem Statement</td><td>Create a console-based online exam with multiple-choice questions. The student answers each question, may skip a question, and at the end the program calculates the score, percentage and the Pass/Fail result.
Language: Python 3 Concepts: Lists, Dictionaries, Functions, Score Calculation
Note: Exam has 5 MCQs (options a-d), 2 marks each, total 10; Pass if score is at least 40%. Answer key: Q1-b, Q2-c, Q3-b, Q4-b, Q5-b. Enter 's' to skip a question; skipped and wrong answers get 0 marks.</td></tr>
<tr><td>Test Cases / Case Statements (4 cases)</td><td>
<table>
<tr><th>INPUT</th><th>EXPECTED OUTPUT</th></tr>
<tr><td colspan="2">Test Case 1: Normal Input</td></tr>
<tr><td>Answers: b, c, a, b, b</td><td>Correct: 4 | Wrong: 1 | Skipped: 0
Score: 8/10 | Percentage: 80.00% | Result: Pass</td></tr>
<tr><td colspan="2">Test Case 2: Boundary / Minimum Input</td></tr>
<tr><td>Answers: b, c, a, a, a</td><td>Correct: 2 | Wrong: 3 | Skipped: 0
Score: 4/10 | Percentage: 40.00% | Result: Pass (minimum pass marks)</td></tr>
<tr><td colspan="2">Test Case 3: Invalid Input</td></tr>
<tr><td>Q1 answer: e (invalid), then b</td><td>Invalid option. Enter a, b, c, d or s to skip.
Answer recorded for Q1: b</td></tr>
<tr><td colspan="2">Test Case 4: Different Valid Scenario</td></tr>
<tr><td>Answers: b, s, b, s, b</td><td>Correct: 3 | Wrong: 0 | Skipped: 2
Score: 6/10 | Percentage: 60.00% | Result: Pass</td></tr>
</table>
</td></tr>
<tr><td>Bare Minimum Requirements</td><td>1. Write the solution in Python 3 (one .py file) and run it from the console.
2. Store the questions as a list of dictionaries with keys: question, options, answer.
3. Ask questions one by one in a loop, validate the entered option and allow skipping with 's'.
4. Calculate correct, wrong and skipped counts, the score, percentage and Pass/Fail using functions.
5. Display a result summary at the end of the exam.</td></tr>
<tr><td>Instruction</td><td>1. Understand the problem statement and define the problem clearly.
2. Develop the solution using proper logic and programming concepts.
3. Implement all required features and pass the given test cases.
4. Test and debug the project to ensure correct output.
5. Prepare the documentation/report and upload the code and report to GitHub.
6. Complete and submit the project within the given deadline.</td></tr>
</table>

BACHELOR OF COMPUTER APPLICATION

<table>
<tr><td>PS No.</td><td>67</td></tr>
<tr><td>Project Title</td><td>Hotel Room Booking System</td></tr>
<tr><td>Problem Statement</td><td>Build a console application for a hotel where the user can view available rooms, book a room for a guest for a number of nights, cancel a booking and generate the bill. Booking details are saved in a file.
Language: Python 3 Concepts: Dictionaries, Lists, Functions, File Handling
Note: Rooms (per night): 101 Single Rs. 2000, 102 Double Rs. 3500, 201 Suite Rs. 6000. Bill = rate x nights + 12% GST. Nights must be 1 to 30. Bookings are saved in bookings.txt.</td></tr>
<tr><td>Test Cases / Case Statements (4 cases)</td><td>
<table>
<tr><th>INPUT</th><th>EXPECTED OUTPUT</th></tr>
<tr><td colspan="2">Test Case 1: Normal Input</td></tr>
<tr><td>Menu option 2 (Book)
Guest: Amit
Room: 101
Nights: 2</td><td>Room 101 booked for Amit | 2 nights
Bill: 4000 + GST 480 = Rs. 4480</td></tr>
<tr><td colspan="2">Test Case 2: Boundary / Minimum Input</td></tr>
<tr><td>Menu option 2 (Book)
Guest: Neha
Room: 102
Nights: 1 (minimum)</td><td>Room 102 booked for Neha | 1 night
Bill: 3500 + GST 420 = Rs. 3920</td></tr>
<tr><td colspan="2">Test Case 3: Invalid Input</td></tr>
<tr><td>Room 101 is already booked by Amit
Menu option 2 (Book)
Guest: Neha, Room: 101, Nights: 3</td><td>Room 101 is already booked. Please choose another room.</td></tr>
<tr><td colspan="2">Test Case 4: Different Valid Scenario</td></tr>
<tr><td>bookings.txt has: Room 101 - Amit - 2 nights
(a) Menu option 1 (Available rooms)
(b) Menu option 4 (Cancel): Room 101</td><td>(a) 102 Double Rs. 3500 | 201 Suite Rs. 6000
(b) Booking for room 101 cancelled. Room 101 is now available.</td></tr>
</table>
</td></tr>
<tr><td>Bare Minimum Requirements</td><td>1. Write the solution in Python 3 (one .py file) and run it from the console.
2. Store room details in a dictionary and current bookings in a list of dictionaries.
3. Provide view-available, book, cancel and bill generation through functions.
4. Validate room number and number of nights; do not allow double booking of a room.
5. Save bookings in bookings.txt after every change and load them at program start.</td></tr>
<tr><td>Instruction</td><td>1. Understand the problem statement and define the problem clearly.
2. Develop the solution using proper logic and programming concepts.
3. Implement all required features and pass the given test cases.
4. Test and debug the project to ensure correct output.
5. Prepare the documentation/report and upload the code and report to GitHub.
6. Complete and submit the project within the given deadline.</td></tr>
</table>

BACHELOR OF COMPUTER APPLICATION

<table>
<tr><td>PS No.</td><td>68</td></tr>
<tr><td>Project Title</td><td>Pharmacy Inventory System</td></tr>
<tr><td>Problem Statement</td><td>Develop a console application to manage a pharmacy's medicine stock. The user can add medicines, view stock, sell medicines (reducing stock and generating a bill), delete medicines and see a low-stock report. Data is stored in a CSV file.
Language: Python 3 Concepts: Dictionaries, Lists, CSV, CRUD
Note: Medicines are stored in medicines.csv with columns ID, Name, Quantity, Price. IDs start from M1. A medicine is 'Low stock' when quantity is below 10.</td></tr>
<tr><td>Test Cases / Case Statements (4 cases)</td><td>
<table>
<tr><th>INPUT</th><th>EXPECTED OUTPUT</th></tr>
<tr><td colspan="2">Test Case 1: Normal Input</td></tr>
<tr><td>Menu option 1 (Add)
Name: Paracetamol
Quantity: 50
Price: 2.50</td><td>Medicine added. ID: M1 | Paracetamol | Qty: 50 | Price: 2.50</td></tr>
<tr><td colspan="2">Test Case 2: Boundary / Minimum Input</td></tr>
<tr><td>M1 Paracetamol, Qty 50, Price 2.50
Menu option 3 (Sell)
ID: M1, Quantity: 50 (entire stock)</td><td>Sold 50 units. Bill: 125.00
Stock left: 0 | Out of stock!</td></tr>
<tr><td colspan="2">Test Case 3: Invalid Input</td></tr>
<tr><td>M1 Paracetamol, Qty 5, Price 2.50
Menu option 3 (Sell)
ID: M1, Quantity: 8</td><td>Insufficient stock. Available: 5</td></tr>
<tr><td colspan="2">Test Case 4: Different Valid Scenario</td></tr>
<tr><td>medicines.csv contains:
M1,Paracetamol,50,2.50
M2,Cetirizine,8,5.00
M3,Amoxicillin,30,12.00
(a) Menu option 5 (Low stock report)
(b) Menu option 4 (Delete): ID M3
(c) Menu option 6 (Inventory value)</td><td>(a) M2 | Cetirizine | Qty: 8 (Low stock)
(b) Medicine M3 deleted.
(c) Total inventory value: 165.00</td></tr>
</table>
</td></tr>
<tr><td>Bare Minimum Requirements</td><td>1. Write the solution in Python 3 (one .py file) and run it from the console.
2. Load medicines from medicines.csv into a list of dictionaries and write them back after every change.
3. Implement add, view, sell, delete, low-stock report and inventory value using functions.
4. Reject negative quantity/price and selling more than the available stock.
5. Calculate the bill (quantity x price) for every sale and update the stock.</td></tr>
<tr><td>Instruction</td><td>1. Understand the problem statement and define the problem clearly.
2. Develop the solution using proper logic and programming concepts.
3. Implement all required features and pass the given test cases.
4. Test and debug the project to ensure correct output.
5. Prepare the documentation/report and upload the code and report to GitHub.
6. Complete and submit the project within the given deadline.</td></tr>
</table>

BACHELOR OF COMPUTER APPLICATION

<table>
<tr><td>PS No.</td><td>69</td></tr>
<tr><td>Project Title</td><td>Bank Loan Calculator</td></tr>
<tr><td>Problem Statement</td><td>Write a program that calculates the monthly EMI, total payable amount and total interest for a bank loan, and checks whether the applicant is eligible based on monthly income. All calculations are done using functions.
Language: Python 3 Concepts: Functions, Operators, EMI Formula, Conditions
Note: EMI = P x r x (1+r)^n / ((1+r)^n - 1), where r = annual rate / 12 / 100 and n = months. If the rate is 0, EMI = P / n. EMI is rounded to 2 decimals first, then Total Payable = EMI x n. Eligible if EMI <= 40% of monthly income. Valid input: amount > 0, rate 0 to 30, tenure 1 to 360 months. Amounts are shown to 2 decimals.</td></tr>
<tr><td>Test Cases / Case Statements (4 cases)</td><td>
<table>
<tr><th>INPUT</th><th>EXPECTED OUTPUT</th></tr>
<tr><td colspan="2">Test Case 1: Normal Input</td></tr>
<tr><td>Principal: 500000
Annual rate: 10
Tenure: 60 months
Monthly income: 50000</td><td>EMI: Rs. 10623.52 | Total Payable: Rs. 637411.20
Total Interest: Rs. 137411.20 | Eligible: Yes</td></tr>
<tr><td colspan="2">Test Case 2: Boundary / Minimum Input</td></tr>
<tr><td>Principal: 120000
Annual rate: 0 (zero interest)
Tenure: 12 months
Monthly income: 30000</td><td>EMI: Rs. 10000.00 | Total Payable: Rs. 120000.00
Total Interest: Rs. 0.00 | Eligible: Yes</td></tr>
<tr><td colspan="2">Test Case 3: Invalid Input</td></tr>
<tr><td>Principal: -50000
Annual rate: 9
Tenure: 24 months</td><td>Invalid loan amount. Enter a positive amount.</td></tr>
<tr><td colspan="2">Test Case 4: Different Valid Scenario</td></tr>
<tr><td>Principal: 1000000
Annual rate: 9
Tenure: 120 months
Monthly income: 20000</td><td>EMI: Rs. 12667.58
Maximum allowed EMI (40% of income): Rs. 8000.00
Loan not eligible: EMI exceeds the allowed limit.</td></tr>
</table>
</td></tr>
<tr><td>Bare Minimum Requirements</td><td>1. Write the solution in Python 3 (one .py file) and run it from the console.
2. Create separate functions for EMI calculation, total payable, total interest and eligibility check.
3. Use the arithmetic/exponent operators correctly and handle the zero-interest case without a division error.
4. Validate amount, rate, tenure and income; show a clear message for invalid values.
5. Display the result in the exact format shown in the test cases.</td></tr>
<tr><td>Instruction</td><td>1. Understand the problem statement and define the problem clearly.
2. Develop the solution using proper logic and programming concepts.
3. Implement all required features and pass the given test cases.
4. Test and debug the project to ensure correct output.
5. Prepare the documentation/report and upload the code and report to GitHub.
6. Complete and submit the project within the given deadline.</td></tr>
</table>

BACHELOR OF COMPUTER APPLICATION

<table>
<tr><td>PS No.</td><td>70</td></tr>
<tr><td>Project Title</td><td>Bus Pass Management System</td></tr>
<tr><td>Problem Statement</td><td>Create a console application to issue and manage bus passes. The user can issue a new pass, renew an existing pass, search a pass by ID and view all passes. Fees depend on the pass type and passenger category, and passes are saved in a file.
Language: Python 3 Concepts: Lists, Dictionaries, Functions, File Handling
Note: Pass types (dictionary): Monthly (30 days) Rs. 500, Quarterly (90 days) Rs. 1350, Yearly (365 days) Rs. 4800. Students get a 25% discount. Passes are saved in buspass.txt. First pass ID is BP101.</td></tr>
<tr><td>Test Cases / Case Statements (4 cases)</td><td>
<table>
<tr><th>INPUT</th><th>EXPECTED OUTPUT</th></tr>
<tr><td colspan="2">Test Case 1: Normal Input</td></tr>
<tr><td>Menu option 1 (Issue)
Name: Riya
Route: Route 5
Type: Monthly
Category: General</td><td>Pass issued. ID: BP101 | Riya | Route 5 | Monthly | Fee: Rs. 500 | Valid: 30 days</td></tr>
<tr><td colspan="2">Test Case 2: Boundary / Minimum Input</td></tr>
<tr><td>buspass.txt is empty
Menu option 4 (View all)</td><td>No passes issued yet.</td></tr>
<tr><td colspan="2">Test Case 3: Invalid Input</td></tr>
<tr><td>Menu option 1 (Issue)
Name: Aman
Route: Route 7
Type: Weekly
Category: General</td><td>Invalid pass type. Choose Monthly, Quarterly or Yearly. Pass not issued.</td></tr>
<tr><td colspan="2">Test Case 4: Different Valid Scenario</td></tr>
<tr><td>buspass.txt has: BP101 | Riya | Route 5 | Monthly | Rs. 500
(a) Menu option 1 (Issue): Aman, Route 7, Yearly, Student
(b) Menu option 3 (Search): ID BP101</td><td>(a) Pass issued. ID: BP102 | Aman | Route 7 | Yearly | Fee: Rs. 3600 | Valid: 365 days
(b) BP101 | Riya | Route 5 | Monthly | Fee: Rs. 500</td></tr>
</table>
</td></tr>
<tr><td>Bare Minimum Requirements</td><td>1. Write the solution in Python 3 (one .py file) and run it from the console.
2. Keep pass types and fees in a dictionary and each issued pass as a dictionary inside a list.
3. Provide issue, renew, search by ID and view-all options using functions.
4. Apply the 25% student discount and validate pass type and category.
5. Save passes in buspass.txt and load them when the program starts.</td></tr>
<tr><td>Instruction</td><td>1. Understand the problem statement and define the problem clearly.
2. Develop the solution using proper logic and programming concepts.
3. Implement all required features and pass the given test cases.
4. Test and debug the project to ensure correct output.
5. Prepare the documentation/report and upload the code and report to GitHub.
6. Complete and submit the project within the given deadline.</td></tr>
</table>

BACHELOR OF COMPUTER APPLICATION

<table>
<tr><td>PS No.</td><td>71</td></tr>
<tr><td>Project Title</td><td>Employee Leave Management System</td></tr>
<tr><td>Problem Statement</td><td>Develop a console application that manages employee leave. An employee applies for leave of a given type between two dates; the program calculates the number of days, checks the leave balance and approves or rejects the request.
Language: Python 3 Concepts: Dictionaries, Conditions, Functions, Dates
Note: Employee E101 (Amit) has the yearly balance: Casual 12, Sick 10, Earned 15 days. Dates are entered as YYYY-MM-DD and days are counted inclusive of both dates. Leave is approved only if the balance is enough.</td></tr>
<tr><td>Test Cases / Case Statements (4 cases)</td><td>
<table>
<tr><th>INPUT</th><th>EXPECTED OUTPUT</th></tr>
<tr><td colspan="2">Test Case 1: Normal Input</td></tr>
<tr><td>Menu option 1 (Apply)
ID: E101, Type: Casual
From: 2026-12-10, To: 2026-12-12</td><td>Leave approved: 3 days Casual | Balance Casual: 9</td></tr>
<tr><td colspan="2">Test Case 2: Boundary / Minimum Input</td></tr>
<tr><td>Menu option 1 (Apply)
ID: E101, Type: Casual
From: 2026-11-02, To: 2026-11-13 (12 days = full balance)</td><td>Leave approved: 12 days Casual | Balance Casual: 0</td></tr>
<tr><td colspan="2">Test Case 3: Invalid Input</td></tr>
<tr><td>Menu option 1 (Apply)
ID: E101, Type: Sick
From: 2026-12-15, To: 2026-12-10</td><td>Invalid dates. To-date cannot be before from-date.</td></tr>
<tr><td colspan="2">Test Case 4: Different Valid Scenario</td></tr>
<tr><td>(a) Menu option 1 (Apply): ID E101, Casual, From: 2027-01-01, To: 2027-01-14
(b) Menu option 2 (View balance): ID E101</td><td>(a) Leave rejected. Requested: 14 days | Available: 12 days
(b) Casual: 12 | Sick: 10 | Earned: 15</td></tr>
</table>
</td></tr>
<tr><td>Bare Minimum Requirements</td><td>1. Write the solution in Python 3 (one .py file) and run it from the console.
2. Store employees and their leave balances in a nested dictionary.
3. Use the datetime module to parse the dates and calculate the number of leave days.
4. Validate employee ID, leave type, date format and date order; approve or reject using conditions.
5. Reduce the balance on approval and provide a view-balance option using functions.</td></tr>
<tr><td>Instruction</td><td>1. Understand the problem statement and define the problem clearly.
2. Develop the solution using proper logic and programming concepts.
3. Implement all required features and pass the given test cases.
4. Test and debug the project to ensure correct output.
5. Prepare the documentation/report and upload the code and report to GitHub.
6. Complete and submit the project within the given deadline.</td></tr>
</table>

BACHELOR OF COMPUTER APPLICATION

<table>
<tr><td>PS No.</td><td>72</td></tr>
<tr><td>Project Title</td><td>Grocery Store Management System</td></tr>
<tr><td>Problem Statement</td><td>Build a console billing application for a grocery store. The cashier selects items and quantities from the inventory, the program calculates the subtotal, discount, GST and final bill, reduces the stock and saves every bill in a file.
Language: Python 3 Concepts: Lists, Dictionaries, Billing, File Handling
Note: Inventory (name, price, stock): Rice 50/kg, 100; Sugar 45/kg, 50; Milk 30/L, 40; Oil 150/L, 20. A 10% discount is given if the subtotal is Rs. 1000 or more. GST is 5% on the amount after discount. Bills are saved in bills.txt.</td></tr>
<tr><td>Test Cases / Case Statements (4 cases)</td><td>
<table>
<tr><th>INPUT</th><th>EXPECTED OUTPUT</th></tr>
<tr><td colspan="2">Test Case 1: Normal Input</td></tr>
<tr><td>Menu option 1 (New bill)
Rice: 2, Milk: 3</td><td>Subtotal: 190.00 | Discount: 0.00
GST (5%): 9.50 | Total: Rs. 199.50</td></tr>
<tr><td colspan="2">Test Case 2: Boundary / Minimum Input</td></tr>
<tr><td>Menu option 1 (New bill)
Oil: 4, Rice: 8 (subtotal exactly 1000)</td><td>Subtotal: 1000.00 | Discount (10%): 100.00
GST (5%): 45.00 | Total: Rs. 945.00</td></tr>
<tr><td colspan="2">Test Case 3: Invalid Input</td></tr>
<tr><td>Menu option 1 (New bill)
Item: Butter, Quantity: 2
Item: Milk, Quantity: 50</td><td>Item 'Butter' not found in inventory.
Insufficient stock for Milk. Available: 40</td></tr>
<tr><td colspan="2">Test Case 4: Different Valid Scenario</td></tr>
<tr><td>(a) Menu option 1 (New bill): Sugar: 10, Milk: 5
(b) Menu option 2 (View stock)</td><td>(a) Subtotal: 600.00 | Discount: 0.00 | GST (5%): 30.00 | Total: Rs. 630.00
(b) Rice: 100 | Sugar: 40 | Milk: 35 | Oil: 20</td></tr>
</table>
</td></tr>
<tr><td>Bare Minimum Requirements</td><td>1. Write the solution in Python 3 (one .py file) and run it from the console.
2. Store the inventory as a dictionary and the current cart as a list of items.
3. Calculate subtotal, discount, GST and total with the rules given in the note.
4. Check that the item exists and the stock is enough; reduce the stock after billing.
5. Append every final bill to bills.txt and save the updated stock.</td></tr>
<tr><td>Instruction</td><td>1. Understand the problem statement and define the problem clearly.
2. Develop the solution using proper logic and programming concepts.
3. Implement all required features and pass the given test cases.
4. Test and debug the project to ensure correct output.
5. Prepare the documentation/report and upload the code and report to GitHub.
6. Complete and submit the project within the given deadline.</td></tr>
</table>

BACHELOR OF COMPUTER APPLICATION

<table>
<tr><td>PS No.</td><td>73</td></tr>
<tr><td>Project Title</td><td>Fitness Data Tracker</td></tr>
<tr><td>Problem Statement</td><td>Create a program to record daily fitness data (steps and calories burned), check whether the daily step goal is achieved, calculate BMI and show a weekly summary with a bar chart of the daily steps.
Language: Python 3 Concepts: Lists, Functions, Calculations, Visualization
Note: Daily goal = 10000 steps. BMI = weight (kg) / height (m) squared: below 18.5 Underweight, 18.5 to 24.9 Normal, 25 to 29.9 Overweight, 30 and above Obese. Records are kept in Python lists (steps, calories); days are numbered Day 1, Day 2, ...</td></tr>
<tr><td>Test Cases / Case Statements (4 cases)</td><td>
<table>
<tr><th>INPUT</th><th>EXPECTED OUTPUT</th></tr>
<tr><td colspan="2">Test Case 1: Normal Input</td></tr>
<tr><td>Menu option 1 (Add record)
Steps: 8000, Calories: 320</td><td>Record saved for Day 1. Goal achieved: No (80.00% of goal)</td></tr>
<tr><td colspan="2">Test Case 2: Boundary / Minimum Input</td></tr>
<tr><td>Menu option 1 (Add record)
Steps: 10000 (exactly the goal), Calories: 400</td><td>Record saved for Day 1. Goal achieved: Yes (100.00% of goal)</td></tr>
<tr><td colspan="2">Test Case 3: Invalid Input</td></tr>
<tr><td>Menu option 1 (Add record)
Steps: -500, Calories: 200</td><td>Invalid steps. Enter a non-negative whole number. Record not saved.</td></tr>
<tr><td colspan="2">Test Case 4: Different Valid Scenario</td></tr>
<tr><td>Records: Day 1 (8000, 320), Day 2 (12000, 480), Day 3 (10000, 400)
(a) Menu option 2 (Summary)
(b) Menu option 3 (BMI): Weight 70 kg, Height 1.75 m
(c) Menu option 4 (Chart)</td><td>(a) Average steps: 10000.00 | Total calories: 1200 | Best day: Day 2 (12000 steps) | Goal days: 2/3
(b) BMI: 22.86 | Category: Normal
(c) Bar chart 'Daily Steps' displayed</td></tr>
</table>
</td></tr>
<tr><td>Bare Minimum Requirements</td><td>1. Write the solution in Python 3 (one .py file) and run it from the console.
2. Store steps and calories in lists and write separate functions for average, total, best day and BMI.
3. Validate the inputs (non-negative numbers; positive weight and height).
4. Plot a Matplotlib bar chart of daily steps with title, axis labels and a line for the 10000-step goal.
5. Show the summary exactly in the format given in the test cases.</td></tr>
<tr><td>Instruction</td><td>1. Understand the problem statement and define the problem clearly.
2. Develop the solution using proper logic and programming concepts.
3. Implement all required features and pass the given test cases.
4. Test and debug the project to ensure correct output.
5. Prepare the documentation/report and upload the code and report to GitHub.
6. Complete and submit the project within the given deadline.</td></tr>
</table>

BACHELOR OF COMPUTER APPLICATION

<table>
<tr><td>PS No.</td><td>74</td></tr>
<tr><td>Project Title</td><td>Electricity Usage Predictor</td></tr>
<tr><td>Problem Statement</td><td>Load past monthly electricity usage from a CSV file, fit a simple linear regression line using NumPy, predict the units consumed in a future month, estimate the bill and plot the actual data with the trend line.
Language: Python 3 Concepts: Pandas, NumPy, Matplotlib, Regression Basics
Note: CSV columns: Month (1, 2, 3, ...) and Units. Model: Units = slope x Month + intercept (use numpy.polyfit with degree 1). Estimated bill = predicted units x Rs. 6 per unit. R2 is rounded to 2 decimals.</td></tr>
<tr><td>Test Cases / Case Statements (4 cases)</td><td>
<table>
<tr><th>INPUT</th><th>EXPECTED OUTPUT</th></tr>
<tr><td colspan="2">Test Case 1: Normal Input</td></tr>
<tr><td>usage.csv:
1,200
2,220
3,240
4,260
Menu option 1 (Predict): Month 5</td><td>Slope: 20.00 | Intercept: 180.00
Predicted units (Month 5): 280.00
Estimated bill: Rs. 1680.00</td></tr>
<tr><td colspan="2">Test Case 2: Boundary / Minimum Input</td></tr>
<tr><td>usage.csv: 2 rows only (minimum for regression)
1,100
2,150
Menu option 1 (Predict): Month 3</td><td>Slope: 50.00 | Intercept: 50.00
Predicted units (Month 3): 200.00
Estimated bill: Rs. 1200.00</td></tr>
<tr><td colspan="2">Test Case 3: Invalid Input</td></tr>
<tr><td>usage.csv: 1 row only
1,100
Menu option 1 (Predict): Month 2</td><td>Error: At least 2 months of data are required for prediction.</td></tr>
<tr><td colspan="2">Test Case 4: Different Valid Scenario</td></tr>
<tr><td>usage.csv:
1,100
2,120
3,150
4,170
Menu option 1 (Predict): Month 6</td><td>Slope: 24.00 | Intercept: 75.00 | R2: 0.99
Predicted units (Month 6): 219.00
Estimated bill: Rs. 1314.00
Line chart 'Actual vs Predicted' displayed</td></tr>
</table>
</td></tr>
<tr><td>Bare Minimum Requirements</td><td>1. Write the solution in Python 3 (one .py file) and run it from the console.
2. Load the CSV with Pandas, check for missing or non-numeric values and show a short data summary.
3. Use NumPy (polyfit) to find slope and intercept and calculate R2; predict units for the entered month.
4. Calculate the estimated bill and handle the case of fewer than 2 data rows.
5. Plot actual usage (points) and the regression line using Matplotlib with title, labels and legend.</td></tr>
<tr><td>Instruction</td><td>1. Understand the problem statement and define the problem clearly.
2. Develop the solution using proper logic and programming concepts.
3. Implement all required features and pass the given test cases.
4. Test and debug the project to ensure correct output.
5. Prepare the documentation/report and upload the code and report to GitHub.
6. Complete and submit the project within the given deadline.</td></tr>
</table>

BACHELOR OF COMPUTER APPLICATION

<table>
<tr><td>PS No.</td><td>75</td></tr>
<tr><td>Project Title</td><td>Student Scholarship Eligibility Analyzer</td></tr>
<tr><td>Problem Statement</td><td>Analyse a student dataset to decide scholarship eligibility using marks, family income and attendance. The program lists each student's result, filters students by marks and summarises the number of scholarships and the total amount.
Language: Python 3 Concepts: Pandas, Conditions, Filtering, Aggregation
Note: CSV columns: Name, Marks (%), Income (per year), Attendance (%). Eligible if Attendance >= 75, Marks >= 75 and Income <= 500000. Full scholarship (Rs. 50000) if Marks >= 90 and Income <= 300000; otherwise Partial (Rs. 25000).</td></tr>
<tr><td>Test Cases / Case Statements (4 cases)</td><td>
<table>
<tr><th>INPUT</th><th>EXPECTED OUTPUT</th></tr>
<tr><td colspan="2">Test Case 1: Normal Input</td></tr>
<tr><td>students.csv:
Aman,92,250000,90
Riya,80,400000,85
Karan,60,200000,95
Neha,88,700000,80
Menu option 1 (Eligibility)</td><td>Aman | Full | Rs. 50000
Riya | Partial | Rs. 25000
Karan | Not Eligible
Neha | Not Eligible</td></tr>
<tr><td colspan="2">Test Case 2: Boundary / Minimum Input</td></tr>
<tr><td>students.csv:
Ravi,90,300000,75
Sita,75,500000,75
Menu option 1 (Eligibility)</td><td>Ravi | Full | Rs. 50000
Sita | Partial | Rs. 25000</td></tr>
<tr><td colspan="2">Test Case 3: Invalid Input</td></tr>
<tr><td>students.csv:
Amit,abc,250000,80</td><td>Invalid marks for Amit: 'abc'. Row skipped.</td></tr>
<tr><td colspan="2">Test Case 4: Different Valid Scenario</td></tr>
<tr><td>Same CSV as Test Case 1
(a) Menu option 2 (Summary)
(b) Menu option 3 (Filter by minimum marks): 85</td><td>(a) Eligible: 2 | Not Eligible: 2 | Full: 1 | Partial: 1
Total scholarship: Rs. 75000 | Average marks (eligible): 86.00
(b) Aman | 92
Neha | 88</td></tr>
</table>
</td></tr>
<tr><td>Bare Minimum Requirements</td><td>1. Write the solution in Python 3 (one .py file) and run it from the console.
2. Load the CSV using Pandas; skip rows with invalid or missing values and print a message for each.
3. Apply the eligibility and scholarship rules using conditions (np.where or a function with apply).
4. Use filtering to select students above a given mark and aggregation (count, sum, mean) for the summary.
5. Display the results in the format shown in the test cases.</td></tr>
<tr><td>Instruction</td><td>1. Understand the problem statement and define the problem clearly.
2. Develop the solution using proper logic and programming concepts.
3. Implement all required features and pass the given test cases.
4. Test and debug the project to ensure correct output.
5. Prepare the documentation/report and upload the code and report to GitHub.
6. Complete and submit the project within the given deadline.</td></tr>
</table>

BACHELOR OF COMPUTER APPLICATION

<table>
<tr><td>PS No.</td><td>76</td></tr>
<tr><td>Project Title</td><td>Employee Performance Dashboard</td></tr>
<tr><td>Problem Statement</td><td>Build a console dashboard that analyses employee performance data. It shows department-wise average ratings, the top performer, the performance category of each employee and total projects per department, with Matplotlib charts.
Language: Python 3 Concepts: Pandas, Matplotlib, Grouping, Visualization
Note: CSV columns: Name, Department, Rating (1 to 5), Projects. Categories: Rating >= 4.5 Outstanding, >= 3.5 Good, >= 2.5 Average, below 2.5 Needs Improvement. Department results are shown in alphabetical order.</td></tr>
<tr><td>Test Cases / Case Statements (4 cases)</td><td>
<table>
<tr><th>INPUT</th><th>EXPECTED OUTPUT</th></tr>
<tr><td colspan="2">Test Case 1: Normal Input</td></tr>
<tr><td>employees.csv:
Amit,IT,4.5,8
Riya,IT,3.5,6
Neha,HR,4.0,5
Karan,HR,2.0,3
Sita,Sales,3.0,4
Menu option 1 (Department average)</td><td>HR: 3.00 | IT: 4.00 | Sales: 3.00
Bar chart 'Average Rating by Department' displayed</td></tr>
<tr><td colspan="2">Test Case 2: Boundary / Minimum Input</td></tr>
<tr><td>employees.csv: 1 row only
Amit,IT,5.0,10
Menu option 2 (Top performer)</td><td>Top performer: Amit (IT) | Rating: 5.0 | Category: Outstanding</td></tr>
<tr><td colspan="2">Test Case 3: Invalid Input</td></tr>
<tr><td>employees.csv:
Rohit,IT,7,5
Menu option 1 (Department average)</td><td>Invalid rating 7 for Rohit. Rating must be between 1 and 5. Row skipped.</td></tr>
<tr><td colspan="2">Test Case 4: Different Valid Scenario</td></tr>
<tr><td>Same CSV as Test Case 1
(a) Menu option 2 (Top performer)
(b) Menu option 3 (Categories)
(c) Menu option 4 (Projects per department)</td><td>(a) Top performer: Amit (IT) | Rating: 4.5 | Category: Outstanding
(b) Outstanding: 1 (Amit) | Good: 2 (Riya, Neha) | Average: 1 (Sita) | Needs Improvement: 1 (Karan)
(c) HR: 8 | IT: 14 | Sales: 4
Bar chart 'Projects per Department' displayed</td></tr>
</table>
</td></tr>
<tr><td>Bare Minimum Requirements</td><td>1. Write the solution in Python 3 (one .py file) and run it from the console.
2. Load the CSV with Pandas, validate the rating (1 to 5) and skip invalid rows with a message.
3. Use groupby to find department-wise average rating and total projects; use idxmax() for the top performer.
4. Assign the performance category to every employee using conditions.
5. Plot at least two Matplotlib charts (bar chart and pie chart/histogram) with titles and labels.</td></tr>
<tr><td>Instruction</td><td>1. Understand the problem statement and define the problem clearly.
2. Develop the solution using proper logic and programming concepts.
3. Implement all required features and pass the given test cases.
4. Test and debug the project to ensure correct output.
5. Prepare the documentation/report and upload the code and report to GitHub.
6. Complete and submit the project within the given deadline.</td></tr>
</table>

RUNGTA INTERNATIONAL SKILLS UNIVERSITY

with AI in association with Google

First Semester

PS No.	77
Project Title	**Hospital Appointment Scheduler**
Problem Statement	Build a console program for a clinic receptionist to book, view, reschedule and cancel patient appointments. Store appointments in a dictionary (ID, patient, doctor, date, time). Slots are 30 minutes from 09:00 to 16:30; one doctor cannot have two appointments at the same time. Check dates with datetime. **Language:** Python 3 **Concepts:** Dictionaries, Datetime, Functions, CRUD **Note:** Doctors: Dr. Mehta, Dr. Rao. Test cases run in order from an empty schedule; the first ID generated is A101.
Test Cases / Case Statements (4 cases)	**INPUT** / **EXPECTED OUTPUT** **Test Case 1: Normal Input** Menu option 1 (Book) Patient: Ravi Sharma Doctor: Dr. Mehta Date: 15-11-2026 Time: 10:00 → Appointment booked. ID: A101 | Ravi Sharma | Dr. Mehta | 15-11-2026 10:00 **Test Case 2: Boundary / Minimum Input** Menu option 1 (Book) Patient: Sita Devi Doctor: Dr. Mehta Date: 15-11-2026 Time: 16:30 → Appointment booked. ID: A102 | Sita Devi | Dr. Mehta | 15-11-2026 16:30 **Test Case 3: Invalid Input** (a) Menu option 1: Anil Kumar, Dr. Mehta, 15-11-2026, 10:00 → (a) Slot unavailable: Dr. Mehta already has an appointment on 15-11-2026 at 10:00. (b) Menu option 1: Anil Kumar, Dr. Rao, 31-11-2026, 10:00 → (b) Invalid date. Please enter the date as DD-MM-YYYY. **Test Case 4: Different Valid Scenario** (a) Menu option 3 (Reschedule): ID A101, new date 16-11-2026, new time 11:30 → (a) Appointment A101 rescheduled to 16-11-2026 11:30. (b) Menu option 4 (Cancel): ID A102 → (b) Appointment A102 cancelled. (c) Menu option 2 (View all) → (c) A101 | Ravi Sharma | Dr. Mehta | 16-11-2026 11:30 Total appointments: 1
Bare Minimum Requirements	**1.** Write the solution in Python 3 (one .py file) and run it from the console. **2.** Show a menu (Book, View all, Reschedule, Cancel, Exit) and repeat until the user chooses Exit. **3.** Store appointments in a dictionary using an auto-generated appointment ID as the key. **4.** Validate the date (DD-MM-YYYY) and time (HH:MM) using the datetime module; accept only 30-minute slots from 09:00 to 16:30. **5.** Test the program with all four test cases.
Instruction	**1.** Understand the problem statement and define the problem clearly. **2.** Develop the solution using proper logic and programming concepts.

	3. Implement all required features and pass the given test cases. **4.** Test and debug the project to ensure correct output. **5.** Prepare the documentation/report and upload the code and report to GitHub. **6.** Complete and submit the project within the given deadline.

PS No.	**78**
Project Title	**Blood Bank Management System**
Problem Statement	Build a program for blood-bank staff to add donors, issue blood, view stock and search donors by blood group. Keep stock in a dictionary and donors in a list saved to donors.txt. Issue blood only if enough units are available. **Language:** Python 3 **Concepts:** Dictionaries, Lists, File Handling, Search **Note:** Initial stock: A+ 10, B+ 5, O+ 8, AB+ 2, all other groups 0. **Note:** donors.txt for Test Case 4 contains: Meena Joshi, O+, 1 and Suresh Rao, O+, 2. **Note:** Each test case starts from this initial data.
Test Cases / Case Statements (4 cases)	**INPUT** / **EXPECTED OUTPUT** **Test Case 1: Normal Input** Menu: Add donor Name: Anil Verma Blood group: B+ Units: 1 → Donor saved to donors.txt. Stock of B+ is now 6 units. **Test Case 2: Boundary / Minimum Input** Menu: Issue blood Blood group: AB+ Units: 2 → 2 unit(s) of AB+ issued. Stock of AB+ is now 0 units. **Test Case 3: Invalid Input** (a) Menu: Issue blood, group A-, units 3 (b) Menu: Add donor, name Rina Das, group C+, units 1 → (a) Insufficient stock: only 0 unit(s) of A- available. (b) Invalid blood group. Valid groups: A+, A-, B+, B-, AB+, AB-, O+, O-. **Test Case 4: Different Valid Scenario** Menu: Search donors Blood group: O+ → Donors with blood group O+: 1. Meena Joshi - 1 unit(s) 2. Suresh Rao - 2 unit(s) Total donors found: 2
Bare Minimum Requirements	**1.** Write the solution in Python 3 (one .py file) and run it from the console. **2.** Show a menu (Add donor, Issue blood, View stock, Search donors, Exit). **3.** Keep blood stock in a dictionary and donor records in a list of dictionaries. **4.** Accept only the eight valid blood groups and positive whole-number units. **5.** Test the program with all four test cases.
Instruction	**1.** Understand the problem statement and define the problem clearly. **2.** Develop the solution using proper logic and programming concepts. **3.** Implement all required features and pass the given test cases. **4.** Test and debug the project to ensure correct output. **5.** Prepare the documentation/report and upload the code and report to GitHub. **6.** Complete and submit the project within the given deadline.

BACHELOR OF COMPUTER APPLICATION

<table>
<tr><td>PS No.</td><td>79</td></tr>
<tr><td>Project Title</td><td>Vehicle Service Management System</td></tr>
<tr><td>Problem Statement</td><td>Build a program for a service centre to add, view, update and delete vehicle service records using a class ServiceRecord. Cost comes from a price list plus 18% GST. Status is Pending, In Progress or Completed. Save records to a file.
Language: Python 3 Concepts: OOP, Dictionaries, File Handling, CRUD
Note: Price list (before GST): Oil Change Rs. 1500, Wheel Alignment Rs. 800, Full Service Rs. 4500, Brake Repair Rs. 2200.
Note: Cost with GST = price x 1.18. Test cases run in order from an empty services.txt.</td></tr>
<tr><td>Test Cases / Case Statements (4 cases)</td><td>
<table>
<tr><th>INPUT</th><th>EXPECTED OUTPUT</th></tr>
<tr><td colspan="2">Test Case 1: Normal Input</td></tr>
<tr><td>Menu: Add record
Reg no: MH12AB1234
Owner: Rohit Patil
Vehicle: Car
Service: Oil Change</td><td>Record added. MH12AB1234 | Oil Change | Cost incl. 18% GST: Rs. 1770.00 | Status: Pending</td></tr>
<tr><td colspan="2">Test Case 2: Boundary / Minimum Input</td></tr>
<tr><td>Menu: Update status
Reg no: MH12AB1234
New status: Completed</td><td>Status updated: Pending -> Completed</td></tr>
<tr><td colspan="2">Test Case 3: Invalid Input</td></tr>
<tr><td>(a) Menu: Add record, MH14ZZ0001, Asha Nair, Car, service Painting
(b) Menu: Delete record, KA01ZZ9999</td><td>(a) Invalid service type. Choose from: Oil Change, Wheel Alignment, Full Service, Brake Repair.
(b) No record found for KA01ZZ9999.</td></tr>
<tr><td colspan="2">Test Case 4: Different Valid Scenario</td></tr>
<tr><td>(a) Menu: Add record, CG07XY4321, Amit Das, Bike, Full Service
(b) Menu: Delete record, CG07XY4321</td><td>(a) Record added. CG07XY4321 | Full Service | Cost incl. 18% GST: Rs. 5310.00 | Status: Pending
(b) Record for CG07XY4321 deleted.</td></tr>
</table>
</td></tr>
<tr><td>Bare Minimum Requirements</td><td>1. Write the solution in Python 3 (one .py file) and run it from the console.
2. Create a class ServiceRecord with attributes and at least two methods (cost calculation and display).
3. Store records in a dictionary keyed by registration number.
4. Provide Add, View, Update status and Delete operations (CRUD).
5. Test the program with all four test cases.</td></tr>
<tr><td>Instruction</td><td>1. Understand the problem statement and define the problem clearly.
2. Develop the solution using proper logic and programming concepts.
3. Implement all required features and pass the given test cases.
4. Test and debug the project to ensure correct output.
5. Prepare the documentation/report and upload the code and report to GitHub.
6. Complete and submit the project within the given deadline.</td></tr>
</table>

BACHELOR OF COMPUTER APPLICATION

<table>
<tr><td>PS No.</td><td>80</td></tr>
<tr><td>Project Title</td><td>Courier Tracking System</td></tr>
<tr><td>Problem Statement</td><td>Build a program for a courier office to book parcels and track them. Each parcel gets an ID (CT1001, ...) and its status moves only forward: Booked, Picked Up, In Transit, Out for Delivery, Delivered. Keep the history of each parcel in a list.
Language: Python 3 Concepts: Dictionaries, Lists, Functions, Search
Note: Test cases run in order from an empty system. In Test Case 3(b) parcel CT1002 already has the status Delivered.</td></tr>
<tr><td>Test Cases / Case Statements (4 cases)</td><td>
<table>
<tr><th>INPUT</th><th>EXPECTED OUTPUT</th></tr>
<tr><td colspan="2">Test Case 1: Normal Input</td></tr>
<tr><td>Menu: Book parcel
Sender: Neha
Receiver: Karan
Destination: Nagpur</td><td>Parcel booked. Tracking ID: CT1001 | Status: Booked</td></tr>
<tr><td colspan="2">Test Case 2: Boundary / Minimum Input</td></tr>
<tr><td>Menu: Update status
Tracking ID: CT1001</td><td>Status updated: Booked -> Picked Up</td></tr>
<tr><td colspan="2">Test Case 3: Invalid Input</td></tr>
<tr><td>(a) Menu: Track parcel, ID CT9999
(b) Menu: Update status, ID CT1002 (already Delivered)</td><td>(a) Tracking ID CT9999 not found.
(b) Parcel CT1002 is already delivered. No further updates allowed.</td></tr>
<tr><td colspan="2">Test Case 4: Different Valid Scenario</td></tr>
<tr><td>(a) Menu: Update status, ID CT1001
(b) Menu: Track parcel, ID CT1001</td><td>(a) Status updated: Picked Up -> In Transit
(b) Tracking ID: CT1001 | Receiver: Karan | Destination: Nagpur
History: Booked -> Picked Up -> In Transit
Current status: In Transit</td></tr>
</table>
</td></tr>
<tr><td>Bare Minimum Requirements</td><td>1. Write the solution in Python 3 (one .py file) and run it from the console.
2. Show a menu (Book, Update status, Track, Search by receiver, Exit).
3. Store parcels in a dictionary and keep the status history in a list.
4. Generate unique tracking IDs automatically.
5. Test the program with all four test cases.</td></tr>
<tr><td>Instruction</td><td>1. Understand the problem statement and define the problem clearly.
2. Develop the solution using proper logic and programming concepts.
3. Implement all required features and pass the given test cases.
4. Test and debug the project to ensure correct output.
5. Prepare the documentation/report and upload the code and report to GitHub.
6. Complete and submit the project within the given deadline.</td></tr>
</table>

RUNGTA INTERNATIONAL SKILLS UNIVERSITY
SCHOOL OF INFORMATION TECHNOLOGY

BACHELOR OF COMPUTER APPLICATION

<table>
<tr><td>PS No.</td><td>81</td></tr>
<tr><td>Project Title</td><td>Bus Route Management System</td></tr>
<tr><td>Problem Statement</td><td>Store a bus network as a dictionary of stops and distances. The program shows a stop's connections and finds the route with the fewest stops (BFS) between two stops, with distance and fare (Rs. 1.50 per km).
Language: Python 3 Concepts: Lists, Dictionaries, Graph Basics
Note: Roads (km): Durg-Bhilai 10, Bhilai-Raipur 30, Durg-Raipur 40, Raipur-Bilaspur 110, Raipur-Dhamtari 80, Durg-Rajnandgaon 45, Jagdalpur-Kondagaon 75.
Note: Roads work in both directions. Jagdalpur and Kondagaon are not connected to the other stops.</td></tr>
<tr><td>Test Cases / Case Statements (4 cases)</td><td>
<table>
<tr><th>INPUT</th><th>EXPECTED OUTPUT</th></tr>
<tr><td colspan="2">Test Case 1: Normal Input</td></tr>
<tr><td>Menu: Show connections
Stop: Raipur</td><td>Raipur connects to: Bhilai (30 km), Bilaspur (110 km), Dhamtari (80 km), Durg (40 km)</td></tr>
<tr><td colspan="2">Test Case 2: Boundary / Minimum Input</td></tr>
<tr><td>Menu: Find route
From: Durg
To: Durg</td><td>Source and destination are the same. Distance: 0 km. Fare: Rs. 0.00</td></tr>
<tr><td colspan="2">Test Case 3: Invalid Input</td></tr>
<tr><td>(a) Menu: Find route, From Durg, To Jagdalpur
(b) Menu: Show connections, Stop: Delhi</td><td>(a) No route available between Durg and Jagdalpur.
(b) Stop 'Delhi' not found.</td></tr>
<tr><td colspan="2">Test Case 4: Different Valid Scenario</td></tr>
<tr><td>Menu: Find route
From: Bhilai
To: Bilaspur</td><td>Route: Bhilai -> Raipur -> Bilaspur
Stops: 3 | Distance: 140 km | Fare: Rs. 210.00</td></tr>
</table>
</td></tr>
<tr><td>Bare Minimum Requirements</td><td>1. Write the solution in Python 3 (one .py file) and run it from the console.
2. Store the bus network as a dictionary of dictionaries (stop -> neighbours -> distance).
3. Show all stops and the direct connections of a chosen stop.
4. Find the route with the fewest stops between two stops using BFS.
5. Test the program with all four test cases.</td></tr>
<tr><td>Instruction</td><td>1. Understand the problem statement and define the problem clearly.
2. Develop the solution using proper logic and programming concepts.
3. Implement all required features and pass the given test cases.
4. Test and debug the project to ensure correct output.
5. Prepare the documentation/report and upload the code and report to GitHub.
6. Complete and submit the project within the given deadline.</td></tr>
</table>

BACHELOR OF COMPUTER APPLICATION

<table>
<tr><td>PS No.</td><td>82</td></tr>
<tr><td>Project Title</td><td>Railway Reservation Analyzer</td></tr>
<tr><td>Problem Statement</td><td>Read bookings.csv with Pandas. Show bookings per train and revenue per train (Confirmed tickets only), skip rows with an invalid fare, and draw a bar chart with Matplotlib.
Language: Python 3 Concepts: Pandas, CSV, Matplotlib, Grouping
Sample data (5 columns, 10 rows):
<pre>PNR,Train,Class,Fare,Status
1001,Shatabdi,CC,950,Confirmed
1002,Shatabdi,CC,950,Confirmed
1003,Shatabdi,EC,1800,Waiting
1004,Rajdhani,3A,1700,Confirmed
1005,Rajdhani,3A,1700,Cancelled
1006,Rajdhani,2A,2400,Confirmed
1007,Gondwana,SL,520,Confirmed
1008,Gondwana,SL,520,Waiting
1009,Gondwana,3A,1500,Confirmed
1010,Gondwana,SL,520,Confirmed</pre></td></tr>
<tr><td>Test Cases / Case Statements (4 cases)</td><td>
<table>
<tr><th>INPUT</th><th>EXPECTED OUTPUT</th></tr>
<tr><td colspan="2">Test Case 1: Normal Input</td></tr>
<tr><td>File: bookings.csv (sample above)
Menu: Bookings per train</td><td>Gondwana : 4
Rajdhani : 3
Shatabdi : 3
Total bookings: 10
(bar chart with three bars is displayed)</td></tr>
<tr><td colspan="2">Test Case 2: Boundary / Minimum Input</td></tr>
<tr><td>File: bookings.csv containing only one data row:
1001,Shatabdi,CC,950,Confirmed
Menu: Revenue per train</td><td>Shatabdi : Rs. 950
Total confirmed revenue: Rs. 950
(bar chart with one bar is displayed)</td></tr>
<tr><td colspan="2">Test Case 3: Invalid Input</td></tr>
<tr><td>(a) File: missing.csv (does not exist)
(b) bookings.csv with the sample rows plus the extra row 1011,Gondwana,SL,abc,Confirmed</td><td>(a) Error: File 'missing.csv' not found.
(b) Warning: Row with PNR 1011 skipped - invalid fare.
Valid records analysed: 10</td></tr>
<tr><td colspan="2">Test Case 4: Different Valid Scenario</td></tr>
<tr><td>File: bookings.csv (sample above)
Menu: Revenue per train (Confirmed only)</td><td>Rajdhani : Rs. 4100
Gondwana : Rs. 2540
Shatabdi : Rs. 1900
Total confirmed revenue: Rs. 8540</td></tr>
</table>
</td></tr>
<tr><td>Bare Minimum Requirements</td><td>1. Write the solution in Python 3 (one .py file) and run it from the console.
2. Read bookings.csv using Pandas and show the number of valid records.
3. Handle a missing file and rows with invalid fares without crashing.
4. Group the data by train to count bookings and by status to count Confirmed, Waiting and Cancelled tickets.
5. Test the program with all four test cases.</td></tr>
<tr><td>Instruction</td><td>1. Understand the problem statement and define the problem clearly.
2. Develop the solution using proper logic and programming concepts.
3. Implement all required features and pass the given test cases.
4. Test and debug the project to ensure correct output.
5. Prepare the documentation/report and upload the code and report to GitHub.
6. Complete and submit the project within the given deadline.</td></tr>
</table>

BACHELOR OF COMPUTER APPLICATION

<table>
<tr><td>PS No.</td><td>83</td></tr>
<tr><td>Project Title</td><td>Flight Booking Management System</td></tr>
<tr><td>Problem Statement</td><td>Build a flight booking program using a class Flight. Load flights from flights.csv, search by origin and destination, book seats (total = seats x fare) and cancel bookings. Seat counts must always stay correct.
Language: Python 3 Concepts: OOP, CSV, Lists, Functions
Note: flights.csv:
Note: Each test case starts from this data.
Sample data (5 columns, 3 rows):
<pre>FlightNo,Origin,Destination,Seats,Fare
AI101,Raipur,Delhi,5,6000
AI202,Raipur,Mumbai,2,5500
AI303,Delhi,Mumbai,0,4800</pre></td></tr>
<tr><td>Test Cases / Case Statements (4 cases)</td><td>
<table>
<tr><th>INPUT</th><th>EXPECTED OUTPUT</th></tr>
<tr><td colspan="2">Test Case 1: Normal Input</td></tr>
<tr><td>Menu: Search flights
Origin: Raipur
Destination: Delhi</td><td>AI101 | Raipur -> Delhi | Seats: 5 | Fare: Rs. 6000 per seat</td></tr>
<tr><td colspan="2">Test Case 2: Boundary / Minimum Input</td></tr>
<tr><td>Menu: Book
Flight: AI202
Passenger: Kiran Rao
Seats: 2</td><td>Booking confirmed. ID: BK1 | Kiran Rao | AI202 | 2 seat(s) | Total: Rs. 11000 | Seats left: 0</td></tr>
<tr><td colspan="2">Test Case 3: Invalid Input</td></tr>
<tr><td>(a) Menu: Book, AI303, 1 seat
(b) Menu: Book, AI101, 6 seats
(c) Menu: Book, AI101, 0 seats</td><td>(a) Flight AI303 is full.
(b) Only 5 seat(s) available on AI101.
(c) Number of seats must be at least 1.</td></tr>
<tr><td colspan="2">Test Case 4: Different Valid Scenario</td></tr>
<tr><td>(a) Menu: Book, AI101, passenger Priya Nair, 3 seats
(b) Menu: Cancel booking, ID BK1</td><td>(a) Booking confirmed. ID: BK1 | Priya Nair | AI101 | 3 seat(s) | Total: Rs. 18000 | Seats left: 2
(b) Booking BK1 cancelled. Seats left on AI101: 5</td></tr>
</table>
</td></tr>
<tr><td>Bare Minimum Requirements</td><td>1. Write the solution in Python 3 (one .py file) and run it from the console.
2. Create a class Flight with attributes and methods to book and cancel seats.
3. Load flights from flights.csv into a list of objects and handle a missing file.
4. Search flights by origin and destination (case-insensitive).
5. Test the program with all four test cases.</td></tr>
<tr><td>Instruction</td><td>1. Understand the problem statement and define the problem clearly.
2. Develop the solution using proper logic and programming concepts.
3. Implement all required features and pass the given test cases.
4. Test and debug the project to ensure correct output.
5. Prepare the documentation/report and upload the code and report to GitHub.
6. Complete and submit the project within the given deadline.</td></tr>
</table>

BACHELOR OF COMPUTER APPLICATION

<table>
<tr><td>PS No.</td><td>84</td></tr>
<tr><td>Project Title</td><td>Hotel Revenue Analyzer</td></tr>
<tr><td>Problem Statement</td><td>Read hotel_sales.csv with Pandas. Revenue = RoomsSold x Rate. Show daily revenue, the best day and the revenue share of each room type, with charts. Skip rows with negative or non-numeric values.
Language: Python 3 Concepts: Pandas, CSV, Visualization
Note: 2026-10-01,Standard,10,2000
Note: 2026-10-01,Deluxe,5,3500
Note: 2026-10-02,Standard,12,2000
Note: 2026-10-02,Deluxe,6,3500
Note: 2026-10-03,Standard,8,2000
Note: 2026-10-03,Deluxe,4,3500
Sample data (4 columns, 0 rows):
<code>Date,RoomType,RoomsSold,Rate</code></td></tr>
<tr><td>Test Cases / Case Statements (4 cases)</td><td>
<table>
<tr><th>INPUT</th><th>EXPECTED OUTPUT</th></tr>
<tr><td colspan="2">Test Case 1: Normal Input</td></tr>
<tr><td>File: hotel_sales.csv (sample above)
Menu: Daily revenue</td><td>2026-10-01 : Rs. 37500
2026-10-02 : Rs. 45000
2026-10-03 : Rs. 30000
Total revenue: Rs. 112500
Best day: 2026-10-02 (Rs. 45000)</td></tr>
<tr><td colspan="2">Test Case 2: Boundary / Minimum Input</td></tr>
<tr><td>File with one data row:
2026-10-01,Standard,1,2000
Menu: Daily revenue</td><td>2026-10-01 : Rs. 2000
Total revenue: Rs. 2000
Best day: 2026-10-01 (Rs. 2000)</td></tr>
<tr><td colspan="2">Test Case 3: Invalid Input</td></tr>
<tr><td>(a) File: hotel_sales.csv with an extra row 2026-10-04,Deluxe,-3,3500
(b) File: sales_old.csv (does not exist)</td><td>(a) Warning: Row 7 skipped - RoomsSold must be 0 or more.
Valid records analysed: 6
(b) Error: File 'sales_old.csv' not found.</td></tr>
<tr><td colspan="2">Test Case 4: Different Valid Scenario</td></tr>
<tr><td>File: hotel_sales.csv (sample above)
Menu: Revenue by room type</td><td>Standard : Rs. 60000 (53.33%)
Deluxe : Rs. 52500 (46.67%)
Total revenue: Rs. 112500</td></tr>
</table>
</td></tr>
<tr><td>Bare Minimum Requirements</td><td>1. Write the solution in Python 3 (one .py file) and run it from the console.
2. Load the CSV file with Pandas and report how many valid rows were read.
3. Create a Revenue column (RoomsSold x Rate).
4. Skip rows with negative or non-numeric values and show a warning.
5. Test the program with all four test cases.</td></tr>
<tr><td>Instruction</td><td>1. Understand the problem statement and define the problem clearly.
2. Develop the solution using proper logic and programming concepts.
3. Implement all required features and pass the given test cases.
4. Test and debug the project to ensure correct output.
5. Prepare the documentation/report and upload the code and report to GitHub.
6. Complete and submit the project within the given deadline.</td></tr>
</table>

BACHELOR OF COMPUTER APPLICATION

<table>
<tr><td>PS No.</td><td>85</td></tr>
<tr><td>Project Title</td><td>Restaurant Sales Dashboard</td></tr>
<tr><td>Problem Statement</td><td>Read restaurant_sales.csv with Pandas. Sales = Quantity x Price. Use GroupBy to show sales by category, by date and the top item by quantity and by revenue, with charts. Skip invalid rows.
Language: Python 3 Concepts: Pandas, Matplotlib, CSV, GroupBy
Note: 2026-10-01,Paneer Tikka,Starter,10,220
Note: 2026-10-01,Dal Makhani,Main,15,260
Note: 2026-10-01,Gulab Jamun,Dessert,20,80
Note: 2026-10-02,Paneer Tikka,Starter,12,220
Note: 2026-10-02,Veg Biryani,Main,10,240
Note: 2026-10-02,Gulab Jamun,Dessert,25,80
Sample data (5 columns, 0 rows):
<code>Date,Item,Category,Quantity,Price</code></td></tr>
<tr><td>Test Cases / Case Statements (4 cases)</td><td>
<table>
<tr><th>INPUT</th><th>EXPECTED OUTPUT</th></tr>
<tr><td colspan="2">Test Case 1: Normal Input</td></tr>
<tr><td>File: restaurant_sales.csv (sample above)
Menu: Sales by category</td><td>Main : Rs. 6300
Starter : Rs. 4840
Dessert : Rs. 3600
Total sales: Rs. 14740</td></tr>
<tr><td colspan="2">Test Case 2: Boundary / Minimum Input</td></tr>
<tr><td>File with one row:
2026-10-01,Paneer Tikka,Starter,1,220
Menu: Top items</td><td>Total sales: Rs. 220
Top item by quantity: Paneer Tikka (1 unit)
Top item by revenue: Paneer Tikka (Rs. 220)</td></tr>
<tr><td colspan="2">Test Case 3: Invalid Input</td></tr>
<tr><td>(a) Extra row in file: 2026-10-03,Lassi,Drink,five,60
(b) File: sales2025.csv (does not exist)</td><td>(a) Warning: Row 8 skipped - Quantity must be a number.
(b) Error: File 'sales2025.csv' not found.</td></tr>
<tr><td colspan="2">Test Case 4: Different Valid Scenario</td></tr>
<tr><td>File: restaurant_sales.csv (sample above)
Menu: Top items and daily sales</td><td>Top item by quantity: Gulab Jamun (45 units)
Top item by revenue: Paneer Tikka (Rs. 4840)
2026-10-01 : Rs. 7700
2026-10-02 : Rs. 7040
Best day: 2026-10-01</td></tr>
</table>
</td></tr>
<tr><td>Bare Minimum Requirements</td><td>1. Write the solution in Python 3 (one .py file) and run it from the console.
2. Load the CSV with Pandas and compute a Sales column (Quantity x Price).
3. Skip invalid rows and handle a missing file.
4. Use GroupBy to total sales by category, item and date.
5. Test the program with all four test cases.</td></tr>
<tr><td>Instruction</td><td>1. Understand the problem statement and define the problem clearly.
2. Develop the solution using proper logic and programming concepts.
3. Implement all required features and pass the given test cases.
4. Test and debug the project to ensure correct output.
5. Prepare the documentation/report and upload the code and report to GitHub.
6. Complete and submit the project within the given deadline.</td></tr>
</table>

BACHELOR OF COMPUTER APPLICATION

<table>
<tr><td>PS No.</td><td>86</td></tr>
<tr><td>Project Title</td><td>Retail Customer Analyzer</td></tr>
<tr><td>Problem Statement</td><td>Read customers.csv with Pandas. Show mean, median, highest and lowest spending, list customers who spent at least a limit entered by the user, and show city-wise total and average spending.
Language: Python 3 Concepts: Pandas, Filtering, Grouping, Statistics
Sample data (5 columns, 8 rows):
<pre>CustomerID,Name,City,Purchases,TotalSpent
C01,Asha,Raipur,5,12000
C02,Bhavin,Durg,2,3000
C03,Chitra,Raipur,8,25000
C04,Deepak,Bhilai,1,800
C05,Esha,Durg,4,9000
C06,Farhan,Raipur,3,6000
C07,Gita,Bhilai,6,15000
C08,Hari,Durg,2,2200</pre></td></tr>
<tr><td>Test Cases / Case Statements (4 cases)</td><td>
<table>
<tr><th>INPUT</th><th>EXPECTED OUTPUT</th></tr>
<tr><td colspan="2">Test Case 1: Normal Input</td></tr>
<tr><td>File: customers.csv (sample above)
Menu: Spending statistics</td><td>Mean spend: Rs. 9125.00
Median spend: Rs. 7500.00
Highest: Chitra (Rs. 25000)
Lowest: Deepak (Rs. 800)</td></tr>
<tr><td colspan="2">Test Case 2: Boundary / Minimum Input</td></tr>
<tr><td>Menu: High-value customers
Limit: 25000</td><td>High-value customers (TotalSpent >= 25000): 1
C03 | Chitra | Raipur | Rs. 25000</td></tr>
<tr><td colspan="2">Test Case 3: Invalid Input</td></tr>
<tr><td>(a) Menu: High-value customers, Limit: abc
(b) Menu: High-value customers, Limit: -500</td><td>(a) Invalid amount. Please enter a number.
(b) Amount must be 0 or more.</td></tr>
<tr><td colspan="2">Test Case 4: Different Valid Scenario</td></tr>
<tr><td>File: customers.csv (sample above)
Menu: City-wise summary</td><td>Raipur : Total Rs. 43000 | Average Rs. 14333.33
Bhilai : Total Rs. 15800 | Average Rs. 7900.00
Durg : Total Rs. 14200 | Average Rs. 4733.33</td></tr>
</table>
</td></tr>
<tr><td>Bare Minimum Requirements</td><td>1. Write the solution in Python 3 (one .py file) and run it from the console.
2. Load customers.csv with Pandas and handle a missing file.
3. Calculate mean, median, highest and lowest TotalSpent.
4. Filter and list customers whose TotalSpent is at least the limit entered by the user.
5. Test the program with all four test cases.</td></tr>
<tr><td>Instruction</td><td>1. Understand the problem statement and define the problem clearly.
2. Develop the solution using proper logic and programming concepts.
3. Implement all required features and pass the given test cases.
4. Test and debug the project to ensure correct output.
5. Prepare the documentation/report and upload the code and report to GitHub.
6. Complete and submit the project within the given deadline.</td></tr>
</table>

BACHELOR OF COMPUTER APPLICATION

PS No.	**87**
Project Title	**Customer Purchase Pattern Analyzer**

Problem Statement	Read orders.csv with Pandas and NumPy. Show monthly spending, the best month, each category's percentage share, and the mean and median order value, with charts. Skip invalid amounts.
	Language: Python 3 **Concepts:** Pandas, NumPy, Visualization, Statistics
	Sample data (4 columns, 8 rows):
	`OrderID,Month,Category,Amount`
	`1,Jan,Grocery,2000`
	`2,Jan,Clothing,3000`
	`3,Feb,Grocery,2500`
	`4,Feb,Electronics,10000`
	`5,Mar,Grocery,3000`
	`6,Mar,Clothing,2000`
	`7,Apr,Electronics,8000`
	`8,Apr,Grocery,3500`

Test Cases / Case Statements (4 cases)	**INPUT** / **EXPECTED OUTPUT** **Test Case 1: Normal Input** File: orders.csv (sample above) Menu: Monthly spending → Jan : Rs. 5000 Feb : Rs. 12500 Mar : Rs. 5000 Apr : Rs. 11500 Total: Rs. 34000 Best month: Feb (Rs. 12500) (bar chart is displayed) **Test Case 2: Boundary / Minimum Input** File with one row: 1,Jan,Grocery,2000 Menu: Order statistics → Orders analysed: 1 Mean order value: Rs. 2000.00 Median order value: Rs. 2000.00 Best month: Jan (Rs. 2000) **Test Case 3: Invalid Input** (a) Extra row in file: 9,May,Grocery,N/A (b) Extra row in file: 10,May,Clothing,-500 → (a) Warning: Row 9 skipped - Amount must be a number. (b) Warning: Row 10 skipped - Amount cannot be negative. Valid orders analysed: 8 **Test Case 4: Different Valid Scenario** File: orders.csv (sample above) Menu: Category share and statistics → Electronics : Rs. 18000 (52.94%) Grocery : Rs. 11000 (32.35%) Clothing : Rs. 5000 (14.71%) Mean order value: Rs. 4250.00 Median order value: Rs. 3000.00
Bare Minimum Requirements	**1.** Write the solution in Python 3 (one .py file) and run it from the console. **2.** Load orders.csv with Pandas and handle a missing file. **3.** Skip rows with invalid or negative amounts and show a warning. **4.** Calculate monthly totals and identify the best month. **5.** Test the program with all four test cases.
Instruction	**1.** Understand the problem statement and define the problem clearly. **2.** Develop the solution using proper logic and programming concepts. **3.** Implement all required features and pass the given test cases. **4.** Test and debug the project to ensure correct output. **5.** Prepare the documentation/report and upload the code and report to GitHub. **6.** Complete and submit the project within the given deadline.

BACHELOR OF COMPUTER APPLICATION

<table>
<tr><td>PS No.</td><td>88</td></tr>
<tr><td>Project Title</td><td>Product Recommendation System</td></tr>
<tr><td>Problem Statement</td><td>Recommend products using Jaccard similarity (common products / all different products). Find the customer most similar to the chosen one and recommend what the chosen customer has not bought. If all similarities are 0, show the most popular product.
Language: Python 3 Concepts: Dictionaries, Similarity, Lists, Functions
Note: Amit: Laptop, Mouse, Keyboard
Note: Bina: Laptop, Mouse, Headphones
Note: Chetan: Phone, Charger
Note: Divya: Laptop, Keyboard, Monitor, Webcam</td></tr>
<tr><td>Test Cases / Case Statements (4 cases)</td><td>
<table>
<tr><th>INPUT</th><th>EXPECTED OUTPUT</th></tr>
<tr><td colspan="2">Test Case 1: Normal Input</td></tr>
<tr><td>Recommend for: Amit</td><td>Similarity scores:
Bina : 0.50
Divya : 0.40
Chetan : 0.00
Most similar customer: Bina
Recommended: Headphones</td></tr>
<tr><td colspan="2">Test Case 2: Boundary / Minimum Input</td></tr>
<tr><td>Recommend for: Chetan</td><td>No similar customers found.
Popular product: Laptop (bought by 3 customers)</td></tr>
<tr><td colspan="2">Test Case 3: Invalid Input</td></tr>
<tr><td>(a) Recommend for: Zara
(b) Recommend for: (empty input)</td><td>(a) Customer 'Zara' not found.
(b) Name cannot be empty.</td></tr>
<tr><td colspan="2">Test Case 4: Different Valid Scenario</td></tr>
<tr><td>Recommend for: Bina</td><td>Similarity scores:
Amit : 0.50
Divya : 0.17
Chetan : 0.00
Most similar customer: Amit
Recommended: Keyboard</td></tr>
</table>
</td></tr>
<tr><td>Bare Minimum Requirements</td><td>1. Write the solution in Python 3 (one .py file) and run it from the console.
2. Store purchases in a dictionary of lists and convert them to sets for comparison.
3. Write a function to calculate Jaccard similarity.
4. Show the similarity scores with two decimal places, highest first.
5. Test the program with all four test cases.</td></tr>
<tr><td>Instruction</td><td>1. Understand the problem statement and define the problem clearly.
2. Develop the solution using proper logic and programming concepts.
3. Implement all required features and pass the given test cases.
4. Test and debug the project to ensure correct output.
5. Prepare the documentation/report and upload the code and report to GitHub.
6. Complete and submit the project within the given deadline.</td></tr>
</table>

BACHELOR OF COMPUTER APPLICATION

<table>
<tr><td>PS No.</td><td>89</td></tr>
<tr><td>Project Title</td><td>Student Course Registration System</td></tr>
<tr><td>Problem Statement</td><td>Build a registration program using classes Student and Course. Register, drop and view courses. Reject full courses, duplicates, unknown courses and more than 3 courses per student. Save registrations to a file.
Language: Python 3 Concepts: OOP, Dictionaries, CRUD, File Handling
Note: Courses: CS101 Programming (capacity 2), MA101 Mathematics (capacity 30), PH101 Physics (capacity 30).
Note: Test cases run in order from an empty registrations.txt.</td></tr>
<tr><td>Test Cases / Case Statements (4 cases)</td><td>
<table>
<tr><th>INPUT</th><th>EXPECTED OUTPUT</th></tr>
<tr><td colspan="2">Test Case 1: Normal Input</td></tr>
<tr><td>Menu: Register
Student ID: S101
Name: Riya
Course: CS101</td><td>Registered: Riya (S101) in CS101 - Programming.
Seats left: 1</td></tr>
<tr><td colspan="2">Test Case 2: Boundary / Minimum Input</td></tr>
<tr><td>Menu: Register
Student ID: S102
Name: Arjun
Course: CS101</td><td>Registered: Arjun (S102) in CS101 - Programming.
Seats left: 0</td></tr>
<tr><td colspan="2">Test Case 3: Invalid Input</td></tr>
<tr><td>(a) Menu: Register, S103, Meera, CS101
(b) Menu: Register, S101, Riya, CS101
(c) Menu: Register, S104, Neel, EE999</td><td>(a) Course CS101 is full.
(b) Student S101 is already registered in CS101.
(c) Course EE999 does not exist.</td></tr>
<tr><td colspan="2">Test Case 4: Different Valid Scenario</td></tr>
<tr><td>(a) Menu: Drop course, S101, CS101
(b) Menu: View student, S101</td><td>(a) Riya (S101) dropped CS101. Seats left: 1
(b) Riya (S101) has no registered courses.</td></tr>
</table>
</td></tr>
<tr><td>Bare Minimum Requirements</td><td>1. Write the solution in Python 3 (one .py file) and run it from the console.
2. Create classes Student and Course with suitable attributes and methods.
3. Keep courses in a dictionary with capacity and enrolled students.
4. Provide Register, View student, Drop course and View course list operations (CRUD).
5. Test the program with all four test cases.</td></tr>
<tr><td>Instruction</td><td>1. Understand the problem statement and define the problem clearly.
2. Develop the solution using proper logic and programming concepts.
3. Implement all required features and pass the given test cases.
4. Test and debug the project to ensure correct output.
5. Prepare the documentation/report and upload the code and report to GitHub.
6. Complete and submit the project within the given deadline.</td></tr>
</table>

BACHELOR OF COMPUTER APPLICATION

<table>
<tr><td>PS No.</td><td>90</td></tr>
<tr><td>Project Title</td><td>Faculty Workload Management System</td></tr>
<tr><td>Problem Statement</td><td>Read workload.csv with Pandas. Show total weekly hours per faculty (limit 16), mark overloaded faculty, assign a new subject only if the limit is not exceeded, and find faculty with enough free hours.
Language: Python 3 Concepts: Pandas, Dictionaries, Functions, Scheduling
Note: Dr. Sharma,DAA,6
Note: Dr. Sharma,DBMS,5
Note: Dr. Verma,OOP,6
Note: Dr. Verma,Maths,6
Note: Dr. Verma,Lab,6
Note: Dr. Iyer,Networks,4
Note: Each test case starts from this data.
Sample data (3 columns, 0 rows):
<code>Faculty,Subject,Hours</code></td></tr>
<tr><td>Test Cases / Case Statements (4 cases)</td><td>
<table>
<tr><th>INPUT</th><th>EXPECTED OUTPUT</th></tr>
<tr><td colspan="2">Test Case 1: Normal Input</td></tr>
<tr><td>Menu: Show workload summary</td><td>Dr. Sharma : 11 hours (5 free)
Dr. Verma : 18 hours - OVERLOADED by 2 hours
Dr. Iyer : 4 hours (12 free)</td></tr>
<tr><td colspan="2">Test Case 2: Boundary / Minimum Input</td></tr>
<tr><td>Menu: Assign subject
Faculty: Dr. Sharma
Subject: Python Lab
Hours: 5</td><td>Assigned. Dr. Sharma now has 16 hours (limit reached).</td></tr>
<tr><td colspan="2">Test Case 3: Invalid Input</td></tr>
<tr><td>(a) Menu: Assign subject, Dr. Sharma, Statistics, 7 hours
(b) Menu: Assign subject, Dr. Iyer, Python Lab, -2 hours</td><td>(a) Cannot assign: Dr. Sharma would have 18 hours, exceeding the 16-hour limit.
(b) Hours must be a positive whole number.</td></tr>
<tr><td colspan="2">Test Case 4: Different Valid Scenario</td></tr>
<tr><td>Menu: Find faculty with free hours
Hours needed: 8</td><td>Faculty available for 8 hours:
Dr. Iyer : 12 hours free</td></tr>
</table>
</td></tr>
<tr><td>Bare Minimum Requirements</td><td>1. Write the solution in Python 3 (one .py file) and run it from the console.
2. Read workload.csv with Pandas and handle a missing file.
3. Calculate total weekly hours per faculty member and free hours (limit 16).
4. Mark faculty above 16 hours as overloaded.
5. Test the program with all four test cases.</td></tr>
<tr><td>Instruction</td><td>1. Understand the problem statement and define the problem clearly.
2. Develop the solution using proper logic and programming concepts.
3. Implement all required features and pass the given test cases.
4. Test and debug the project to ensure correct output.
5. Prepare the documentation/report and upload the code and report to GitHub.
6. Complete and submit the project within the given deadline.</td></tr>
</table>

BACHELOR OF COMPUTER APPLICATION

<table>
<tr><td>PS No.</td><td>91</td></tr>
<tr><td>Project Title</td><td>Classroom Timetable Generator</td></tr>
<tr><td>Problem Statement</td><td>Generate a weekly class timetable. Total periods must equal days x periods per day and a subject can have at most 2 periods a day. Fill slots by choosing the subject with most periods left (ties: the one entered first).
Language: Python 3 Concepts: Lists, Dictionaries, Constraints, Scheduling
Note: Day names used: Mon, Tue, Wed, Thu, Fri, Sat in order.</td></tr>
<tr><td>Test Cases / Case Statements (4 cases)</td><td>
<table>
<tr><th>INPUT</th><th>EXPECTED OUTPUT</th></tr>
<tr><td colspan="2">Test Case 1: Normal Input</td></tr>
<tr><td>Days: 3
Periods per day: 3
Subjects (periods): Maths 3, Physics 2, Programming 3, English 1</td><td>Mon: Maths, Programming, Maths
Tue: Physics, Programming, Maths
Wed: Physics, Programming, English</td></tr>
<tr><td colspan="2">Test Case 2: Boundary / Minimum Input</td></tr>
<tr><td>Days: 1
Periods per day: 2
Subjects (periods): Maths 2</td><td>Mon: Maths, Maths</td></tr>
<tr><td colspan="2">Test Case 3: Invalid Input</td></tr>
<tr><td>(a) Days: 3, Periods per day: 3, Subjects: Maths 3, Physics 2, Programming 3 (total 8)
(b) Days: 1, Periods per day: 3, Subjects: Maths 3</td><td>(a) Error: Total weekly periods (8) must equal available slots (9).
(b) Timetable cannot be generated: Maths needs 3 periods in 1 day, but the limit is 2 per day.</td></tr>
<tr><td colspan="2">Test Case 4: Different Valid Scenario</td></tr>
<tr><td>Days: 2
Periods per day: 2
Subjects (periods): Programming 2, English 2</td><td>Mon: Programming, English
Tue: Programming, English</td></tr>
</table>
</td></tr>
<tr><td>Bare Minimum Requirements</td><td>1. Write the solution in Python 3 (one .py file) and run it from the console.
2. Accept days, periods per day and subject-wise weekly periods with validation (positive whole numbers).
3. Check that the total periods equal the total available slots.
4. Use a dictionary to track remaining periods for each subject.
5. Print the timetable day by day and test with all four test cases.</td></tr>
<tr><td>Instruction</td><td>1. Understand the problem statement and define the problem clearly.
2. Develop the solution using proper logic and programming concepts.
3. Implement all required features and pass the given test cases.
4. Test and debug the project to ensure correct output.
5. Prepare the documentation/report and upload the code and report to GitHub.
6. Complete and submit the project within the given deadline.</td></tr>
</table>

BACHELOR OF COMPUTER APPLICATION

<table>
<tr><td>PS No.</td><td>92</td></tr>
<tr><td>Project Title</td><td>Examination Timetable Generator</td></tr>
<tr><td>Problem Statement</td><td>Generate an exam date sheet from subjects, a start date and holidays. One exam per day in the given order; skip Sundays and holidays. Use datetime to find dates and weekdays.
Language: Python 3 Concepts: Lists, Dictionaries, Scheduling, Constraints
Note: Reference: 12-10-2026 is a Monday; 18-10-2026 is a Sunday.</td></tr>
<tr><td>Test Cases / Case Statements (4 cases)</td><td>
<table>
<tr><th>INPUT</th><th>EXPECTED OUTPUT</th></tr>
<tr><td colspan="2">Test Case 1: Normal Input</td></tr>
<tr><td>Subjects: DAA, DBMS, OOP, Maths, CSA
Start date: 12-10-2026
Holidays: 14-10-2026</td><td>DAA : 12-10-2026 (Monday)
DBMS : 13-10-2026 (Tuesday)
OOP : 15-10-2026 (Thursday)
Maths : 16-10-2026 (Friday)
CSA : 17-10-2026 (Saturday)</td></tr>
<tr><td colspan="2">Test Case 2: Boundary / Minimum Input</td></tr>
<tr><td>Subjects: DAA
Start date: 18-10-2026
Holidays: none</td><td>DAA : 19-10-2026 (Monday)</td></tr>
<tr><td colspan="2">Test Case 3: Invalid Input</td></tr>
<tr><td>(a) Subjects: DAA, DBMS; Start date: 32-10-2026
(b) Subjects: (none)
(c) Subjects: DAA, DAA; Start date: 12-10-2026</td><td>(a) Invalid date. Please enter the date as DD-MM-YYYY.
(b) At least one subject is required.
(c) Duplicate subject: DAA.</td></tr>
<tr><td colspan="2">Test Case 4: Different Valid Scenario</td></tr>
<tr><td>Subjects: DAA, DBMS, OOP, Maths, CSA, DM, Java
Start date: 12-10-2026
Holidays: none</td><td>DAA : 12-10-2026 (Monday)
DBMS : 13-10-2026 (Tuesday)
OOP : 14-10-2026 (Wednesday)
Maths : 15-10-2026 (Thursday)
CSA : 16-10-2026 (Friday)
DM : 17-10-2026 (Saturday)
Java : 19-10-2026 (Monday)</td></tr>
</table>
</td></tr>
<tr><td>Bare Minimum Requirements</td><td>1. Write the solution in Python 3 (one .py file) and run it from the console.
2. Accept a list of subjects, a start date and optional holiday dates, and validate them (date format, not empty, no duplicates).
3. Use a list to hold subjects and dictionaries to hold each date-sheet entry.
4. Schedule exactly one exam per day in the given order.
5. Test the program with all four test cases.</td></tr>
<tr><td>Instruction</td><td>1. Understand the problem statement and define the problem clearly.
2. Develop the solution using proper logic and programming concepts.
3. Implement all required features and pass the given test cases.
4. Test and debug the project to ensure correct output.
5. Prepare the documentation/report and upload the code and report to GitHub.
6. Complete and submit the project within the given deadline.</td></tr>
</table>

BACHELOR OF COMPUTER APPLICATION

PS No.	**93**
Project Title	**Automated Marksheet Generator**

Problem Statement	Read marks.csv with Pandas. Calculate total (out of 300), percentage, grade (90+ A+, 80 A, 70 B, 60 C, 50 D, 40 E, below 40 F) and result (FAIL if any subject is below 40). Print a neat marksheet.
	Language: Python 3 **Concepts:** Pandas, Functions, File Handling, Formatting
	Sample data (5 columns, 4 rows):
	`RollNo,Name,Maths,Physics,Programming`
	`101,Anita,85,90,80`
	`102,Rahul,40,45,50`
	`103,Suman,35,70,60`
	`104,Vikas,100,100,100`

Test Cases / Case Statements (4 cases)	**INPUT** / **EXPECTED OUTPUT** **Test Case 1: Normal Input** File: marks.csv (sample above) Menu: Single marksheet Roll no: 101 → Roll No: 101 | Name: Anita Maths: 85 | Physics: 90 | Programming: 80 Total: 255/300 | Percentage: 85.00% Grade: A | Result: PASS **Test Case 2: Boundary / Minimum Input** Menu: Single marksheet Roll no: 102 → Roll No: 102 | Name: Rahul Maths: 40 | Physics: 45 | Programming: 50 Total: 135/300 | Percentage: 45.00% Grade: E | Result: PASS **Test Case 3: Invalid Input** (a) Menu: Single marksheet, Roll no: 999 (b) marks.csv contains an extra row 106,Kiran,105,60,70 → (a) Roll number 999 not found. (b) Warning: Row for roll 106 skipped - marks must be between 0 and 100. **Test Case 4: Different Valid Scenario** Menu: Single marksheet Roll no: 103 → Roll No: 103 | Name: Suman Maths: 35 | Physics: 70 | Programming: 60 Total: 165/300 | Percentage: 55.00% Grade: D | Result: FAIL (Maths below 40)
Bare Minimum Requirements	**1.** Write the solution in Python 3 (one .py file) and run it from the console. **2.** Read marks.csv with Pandas and handle a missing file. **3.** Validate that each mark is between 0 and 100 and skip invalid rows with a warning. **4.** Calculate total and percentage with two decimal places. **5.** Test the program with all four test cases.
Instruction	**1.** Understand the problem statement and define the problem clearly. **2.** Develop the solution using proper logic and programming concepts. **3.** Implement all required features and pass the given test cases. **4.** Test and debug the project to ensure correct output. **5.** Prepare the documentation/report and upload the code and report to GitHub. **6.** Complete and submit the project within the given deadline.

BACHELOR OF COMPUTER APPLICATION

<table>
<tr><td>PS No.</td><td>94</td></tr>
<tr><td>Project Title</td><td>Result Statistics Analyzer</td></tr>
<tr><td>Problem Statement</td><td>Read results.csv (RollNo, Marks). Show mean, median, mode, highest, lowest, standard deviation (np.std), pass percentage (pass mark 40) and the count of students in bands 0-39, 40-59, 60-79, 80-100 with a chart.
Language: Python 3 Concepts: Pandas, NumPy, Visualization, Statistics
Note: results.csv (sample marks): 45, 56, 67, 72, 72, 80, 85, 90, 38, 95 for roll numbers 1 to 10 in this order.</td></tr>
<tr><td>Test Cases / Case Statements (4 cases)</td><td>
<table>
<tr><th>INPUT</th><th>EXPECTED OUTPUT</th></tr>
<tr><td colspan="2">Test Case 1: Normal Input</td></tr>
<tr><td>File: results.csv (sample above)
Menu: Show statistics</td><td>Students: 10
Mean: 70.00
Median: 72.00
Mode: 72
Highest: 95 | Lowest: 38
Standard deviation: 17.92</td></tr>
<tr><td colspan="2">Test Case 2: Boundary / Minimum Input</td></tr>
<tr><td>File with one data row: 1,40
Menu: Show statistics</td><td>Students: 1
Mean: 40.00
Median: 40.00
Mode: 40
Highest: 40 | Lowest: 40
Standard deviation: 0.00
Pass percentage: 100.00%</td></tr>
<tr><td colspan="2">Test Case 3: Invalid Input</td></tr>
<tr><td>(a) Extra row in file: 11,120
(b) File: results.csv has only the header row</td><td>(a) Warning: Row 11 skipped - marks must be between 0 and 100.
(b) No valid marks found. Nothing to analyse.</td></tr>
<tr><td colspan="2">Test Case 4: Different Valid Scenario</td></tr>
<tr><td>File: results.csv (sample above)
Menu: Pass percentage and distribution</td><td>Passed: 9 | Failed: 1 | Pass percentage: 90.00%
0-39 : 1
40-59 : 2
60-79 : 3
80-100 : 4
(bar chart is displayed)</td></tr>
</table>
</td></tr>
<tr><td>Bare Minimum Requirements</td><td>1. Write the solution in Python 3 (one .py file) and run it from the console.
2. Read results.csv using Pandas and handle a missing file.
3. Skip missing marks and marks outside 0-100 with a warning.
4. Calculate mean, median, mode, highest, lowest and standard deviation using NumPy or Pandas.
5. Test the program with all four test cases.</td></tr>
<tr><td>Instruction</td><td>1. Understand the problem statement and define the problem clearly.
2. Develop the solution using proper logic and programming concepts.
3. Implement all required features and pass the given test cases.
4. Test and debug the project to ensure correct output.
5. Prepare the documentation/report and upload the code and report to GitHub.
6. Complete and submit the project within the given deadline.</td></tr>
</table>

BACHELOR OF COMPUTER APPLICATION

PS No.	**95**
Project Title	**Attendance Percentage Predictor**
Problem Statement	Use weekly attendance percentages to fit a straight line with np.polyfit and predict attendance for a future week (limit 0-100). At least 2 weeks of data are needed. Show 'Safe' if 75 or more, else 'Warning: below 75%'. **Language:** Python 3 **Concepts:** Pandas, NumPy, Regression Basics **Note:** Reference data sets used in the test cases are given inside each test case.
Test Cases / Case Statements (4 cases)	**INPUT** / **EXPECTED OUTPUT** **Test Case 1: Normal Input** Weeks 1-5, Attendance: 90, 88, 86, 84, 82 Predict for week: 10 Slope: -2.00 | Intercept: 92.00 Predicted attendance for week 10: 72.00% Warning: below 75% **Test Case 2: Boundary / Minimum Input** Weeks 1-2, Attendance: 90, 80 Predict for week: 3 Slope: -10.00 | Intercept: 100.00 Predicted attendance for week 3: 70.00% Warning: below 75% **Test Case 3: Invalid Input** (a) Week 1 only, Attendance: 90 (b) Weeks 1-3, Attendance: 90, 105, 80 (a) At least 2 weeks of data are required. (b) Attendance must be between 0 and 100. **Test Case 4: Different Valid Scenario** Weeks 1-5, Attendance: 70, 74, 78, 82, 86 Predict for week: 9 Slope: 4.00 | Intercept: 66.00 Predicted attendance for week 9: 100.00% (capped at 100) Safe
Bare Minimum Requirements	**1.** Write the solution in Python 3 (one .py file) and run it from the console. **2.** Load weekly attendance from a CSV file or from user input and validate it (0-100, at least 2 weeks). **3.** Use NumPy to calculate the slope and intercept of the best-fit line. **4.** Predict the attendance for a future week entered by the user and cap the result between 0 and 100. **5.** Test the program with all four test cases.
Instruction	**1.** Understand the problem statement and define the problem clearly. **2.** Develop the solution using proper logic and programming concepts. **3.** Implement all required features and pass the given test cases. **4.** Test and debug the project to ensure correct output. **5.** Prepare the documentation/report and upload the code and report to GitHub. **6.** Complete and submit the project within the given deadline.

BACHELOR OF COMPUTER APPLICATION

<table>
<tr><td>PS No.</td><td>96</td></tr>
<tr><td>Project Title</td><td>Student Feedback Analyzer</td></tr>
<tr><td>Problem Statement</td><td>Read feedback.csv (StudentID, Rating 1-5, Comment). Show average rating, rating counts, Positive (4-5), Neutral (3) and Negative (1-2) percentages and the top words in comments (remove stop words: and, but, the, is, a, of, too; ties alphabetical).
Language: Python 3 Concepts: Strings, Pandas, Frequency Analysis, Visualization
Note: S1,5,Great teaching and clear notes
Note: S2,4,Clear explanations and good examples
Note: S3,2,Too fast and notes unclear
Note: S5,3,Good but too fast
Sample data (3 columns, 2 rows):
<pre>StudentID,Rating,Comment
S4,5,Great examples
S6,4,Great notes</pre></td></tr>
<tr><td>Test Cases / Case Statements (4 cases)</td><td>
<table>
<tr><th>INPUT</th><th>EXPECTED OUTPUT</th></tr>
<tr><td colspan="2">Test Case 1: Normal Input</td></tr>
<tr><td>File: feedback.csv (sample above)
Menu: Summary and top 3 words</td><td>Valid feedback: 6
Average rating: 3.83
Ratings: 5 -> 2, 4 -> 2, 3 -> 1, 2 -> 1, 1 -> 0
Top 3 words: great (3), notes (3), clear (2)</td></tr>
<tr><td colspan="2">Test Case 2: Boundary / Minimum Input</td></tr>
<tr><td>File with one row: S1,5,Great
Menu: Summary and top 3 words</td><td>Valid feedback: 1
Average rating: 5.00
Ratings: 5 -> 1, 4 -> 0, 3 -> 0, 2 -> 0, 1 -> 0
Top 3 words: great (1)</td></tr>
<tr><td colspan="2">Test Case 3: Invalid Input</td></tr>
<tr><td>Sample file plus two extra rows:
S7,6,Nice
S8,abc,Nice</td><td>Warning: Row 7 skipped - rating must be a whole number from 1 to 5.
Warning: Row 8 skipped - rating must be a whole number from 1 to 5.
Valid feedback: 6
Average rating: 3.83</td></tr>
<tr><td colspan="2">Test Case 4: Different Valid Scenario</td></tr>
<tr><td>File: feedback.csv (sample above)
Menu: Sentiment</td><td>Positive: 4 (66.67%)
Neutral: 1 (16.67%)
Negative: 1 (16.67%)
(bar chart is displayed)</td></tr>
</table>
</td></tr>
<tr><td>Bare Minimum Requirements</td><td>1. Write the solution in Python 3 (one .py file) and run it from the console.
2. Read feedback.csv using Pandas and handle a missing file.
3. Validate ratings (whole number from 1 to 5) and skip invalid rows with a warning.
4. Calculate the average rating and the count of each rating.
5. Test the program with all four test cases.</td></tr>
<tr><td>Instruction</td><td>1. Understand the problem statement and define the problem clearly.
2. Develop the solution using proper logic and programming concepts.
3. Implement all required features and pass the given test cases.
4. Test and debug the project to ensure correct output.
5. Prepare the documentation/report and upload the code and report to GitHub.
6. Complete and submit the project within the given deadline.</td></tr>
</table>

BACHELOR OF COMPUTER APPLICATION

<table>
<tr><td>PS No.</td><td>97</td></tr>
<tr><td>Project Title</td><td>Weather Data Analyzer</td></tr>
<tr><td>Problem Statement</td><td>Build a program that loads daily weather data (date, temperature, humidity, rainfall) from a CSV file, cleans it, and analyses it to find hottest/coldest days, averages and rainfall trends, with charts.
Language: Python 3 Concepts: Pandas, NumPy, Matplotlib, Data Cleaning
Note: CSV columns: Date, Temp, Humidity, Rainfall. Test cases use the small CSV rows given in each INPUT.</td></tr>
<tr><td>Test Cases / Case Statements (4 cases)</td><td>
<table>
<tr><th>INPUT</th><th>EXPECTED OUTPUT</th></tr>
<tr><td colspan="2">Test Case 1: Normal Input</td></tr>
<tr><td>weather.csv:
2026-08-01,32,70,0
2026-08-02,34,65,0
2026-08-03,30,80,12
Menu option 1 (Summary)</td><td>Average Temp: 32.00 C | Max: 34 (2026-08-02) | Min: 30 (2026-08-03) | Total Rainfall: 12 mm</td></tr>
<tr><td colspan="2">Test Case 2: Boundary / Minimum Input</td></tr>
<tr><td>weather.csv: 1 row only
2026-08-01,31,60,0
Menu option 1 (Summary)</td><td>Average Temp: 31.00 C | Max: 31 | Min: 31 | Std Dev: 0.00</td></tr>
<tr><td colspan="2">Test Case 3: Invalid Input</td></tr>
<tr><td>File name: wether.csv (file does not exist)</td><td>Error: File 'wether.csv' not found. Please enter a valid file name.</td></tr>
<tr><td colspan="2">Test Case 4: Different Valid Scenario</td></tr>
<tr><td>weather.csv:
2026-08-01,32,70,0
2026-08-02,,65,0
2026-08-02,,65,0
2026-08-03,30,80,12
Menu option 2 (Clean & Plot)</td><td>Duplicates removed: 1 | Missing Temp filled with mean: 31.00
Cleaned rows: 3 | Line chart 'Temperature Trend' displayed</td></tr>
</table>
</td></tr>
<tr><td>Bare Minimum Requirements</td><td>1. Write the solution in Python 3 (one .py file) and run it from the console.
2. Load the CSV file using Pandas and show a short data summary.
3. Clean the data (handle missing values and duplicates).
4. Use NumPy to calculate mean, maximum, minimum and standard deviation.
5. Plot at least two Matplotlib charts (line and bar) with title and labels.</td></tr>
<tr><td>Instruction</td><td>1. Understand the problem statement and define the problem clearly.
2. Develop the solution using proper logic and programming concepts.
3. Implement all required features and pass the given test cases.
4. Test and debug the project to ensure correct output.
5. Prepare the documentation/report and upload the code and report to GitHub.
6. Complete and submit the project within the given deadline.</td></tr>
</table>

BACHELOR OF COMPUTER APPLICATION

<table>
<tr><td>PS No.</td><td>98</td></tr>
<tr><td>Project Title</td><td>ATM Management System</td></tr>
<tr><td>Problem Statement</td><td>Create a menu-driven ATM simulation where a user logs in with a PIN and can check balance, deposit, withdraw and view a mini statement. Account data and transactions are stored in a file.
Language: Python 3 Concepts: Conditions, Loops, Functions, Exceptions, File Handling
Note: Default PIN is 1234 and opening balance is 10000 (stored in atm.txt). Each test case starts with this default data.</td></tr>
<tr><td>Test Cases / Case Statements (4 cases)</td><td>
<table>
<tr><th>INPUT</th><th>EXPECTED OUTPUT</th></tr>
<tr><td colspan="2">Test Case 1: Normal Input</td></tr>
<tr><td>PIN: 1234
Menu option 2 (Deposit)
Amount: 5000</td><td>Login successful
Deposit successful. New Balance: 15000</td></tr>
<tr><td colspan="2">Test Case 2: Boundary / Minimum Input</td></tr>
<tr><td>PIN: 1234
Menu option 3 (Withdraw)
Amount: 10000 (equal to balance)</td><td>Withdrawal successful. New Balance: 0</td></tr>
<tr><td colspan="2">Test Case 3: Invalid Input</td></tr>
<tr><td>PIN: 1111, then 2222, then 3333</td><td>Invalid PIN. Attempts left: 2
Invalid PIN. Attempts left: 1
Account locked. Exiting...</td></tr>
<tr><td colspan="2">Test Case 4: Different Valid Scenario</td></tr>
<tr><td>PIN: 1234
(a) Option 3 (Withdraw): abc
(b) Option 3 (Withdraw): 2000
(c) Option 4 (Mini statement)</td><td>(a) Invalid amount. Please enter numbers only.
(b) Withdrawal successful. New Balance: 8000
(c) 1. Withdraw 2000 | Balance: 8000</td></tr>
</table>
</td></tr>
<tr><td>Bare Minimum Requirements</td><td>1. Write the solution in Python 3 (one .py file) and run it from the console.
2. Verify the PIN with a maximum of 3 attempts using a loop.
3. Implement balance check, deposit, withdraw and mini statement using functions.
4. Handle invalid input and file errors using try-except.
5. Save balance and transactions to a file after every operation.</td></tr>
<tr><td>Instruction</td><td>1. Understand the problem statement and define the problem clearly.
2. Develop the solution using proper logic and programming concepts.
3. Implement all required features and pass the given test cases.
4. Test and debug the project to ensure correct output.
5. Prepare the documentation/report and upload the code and report to GitHub.
6. Complete and submit the project within the given deadline.</td></tr>
</table>

BACHELOR OF COMPUTER APPLICATION

<table>
<tr><td>PS No.</td><td>99</td></tr>
<tr><td>Project Title</td><td>Library Management System</td></tr>
<tr><td>Problem Statement</td><td>Develop a console application to manage a library's books: add books, issue and return books, search by title/author and view all books. Data is saved to a file.
Language: Python 3 Concepts: Lists, Dictionaries, Functions, File Handling
Note: Books are stored in library.txt. The first book ID generated is B101.</td></tr>
<tr><td>Test Cases / Case Statements (4 cases)</td><td>
<table>
<tr><th>INPUT</th><th>EXPECTED OUTPUT</th></tr>
<tr><td colspan="2">Test Case 1: Normal Input</td></tr>
<tr><td>Menu option 1 (Add book)
Title: Python Basics
Author: Mark Lutz</td><td>Book added. ID: B101 | Python Basics | Mark Lutz | Available</td></tr>
<tr><td colspan="2">Test Case 2: Boundary / Minimum Input</td></tr>
<tr><td>Library has only book B101
Menu option 2 (Issue): ID B101, Member: Riya</td><td>Book B101 issued to Riya. Status: Issued
Available books: 0</td></tr>
<tr><td colspan="2">Test Case 3: Invalid Input</td></tr>
<tr><td>Menu option 2 (Issue)
Book ID: B999</td><td>Book ID B999 not found.</td></tr>
<tr><td colspan="2">Test Case 4: Different Valid Scenario</td></tr>
<tr><td>(a) Option 1: Data Science | Wes McKinney
(b) Option 4 (Search by author): McKinney
(c) Option 3 (Return): ID B101</td><td>(a) Book added. ID: B102 | Data Science | Wes McKinney | Available
(b) B102 | Data Science | Wes McKinney | Available
(c) Book B101 returned. Status: Available</td></tr>
</table>
</td></tr>
<tr><td>Bare Minimum Requirements</td><td>1. Write the solution in Python 3 (one .py file) and run it from the console.
2. Store each book as a dictionary inside a list.
3. Provide add, issue, return, search and display-all features using functions.
4. Do not allow issuing a book that is already issued.
5. Load data from the file at start and save it on exit.</td></tr>
<tr><td>Instruction</td><td>1. Understand the problem statement and define the problem clearly.
2. Develop the solution using proper logic and programming concepts.
3. Implement all required features and pass the given test cases.
4. Test and debug the project to ensure correct output.
5. Prepare the documentation/report and upload the code and report to GitHub.
6. Complete and submit the project within the given deadline.</td></tr>
</table>

BACHELOR OF COMPUTER APPLICATION

<table>
<tr><td>PS No.</td><td>100</td></tr>
<tr><td>Project Title</td><td>Student Performance Analyzer</td></tr>
<tr><td>Problem Statement</td><td>Analyse student marks across subjects from a dataset to calculate totals, percentages, grades and class statistics, and present the results with charts.
Language: Python 3 Concepts: Pandas, NumPy, Matplotlib, Statistics
Note: Marks are out of 100 in Maths, Physics, Chemistry. Grades: >=90 A, >=75 B, >=60 C, >=40 D, below 40 F.</td></tr>
<tr><td>Test Cases / Case Statements (4 cases)</td><td>
<table>
<tr><th>INPUT</th><th>EXPECTED OUTPUT</th></tr>
<tr><td colspan="2">Test Case 1: Normal Input</td></tr>
<tr><td>students.csv:
Aman,80,70,90
Riya,60,65,70
Karan,40,35,50
Menu option 1 (Result)</td><td>Aman | 240 | 80.00% | B
Riya | 195 | 65.00% | C
Karan | 125 | 41.67% | D</td></tr>
<tr><td colspan="2">Test Case 2: Boundary / Minimum Input</td></tr>
<tr><td>students.csv:
Neha,100,100,100
Ravi,0,0,0
Menu option 1 (Result)</td><td>Neha | 300 | 100.00% | A
Ravi | 0 | 0.00% | F</td></tr>
<tr><td colspan="2">Test Case 3: Invalid Input</td></tr>
<tr><td>students.csv:
Amit,85,abc,70</td><td>Invalid marks for Amit in Physics: 'abc'. Row skipped.</td></tr>
<tr><td colspan="2">Test Case 4: Different Valid Scenario</td></tr>
<tr><td>Same CSV as Test Case 1
Menu option 2 (Statistics)</td><td>Mean %: 62.22 | Median %: 65.00 | Std Dev: 15.77
Topper: Aman (80.00%)
Subject averages: Maths 60.00, Physics 56.67, Chemistry 70.00
Bar chart displayed</td></tr>
</table>
</td></tr>
<tr><td>Bare Minimum Requirements</td><td>1. Write the solution in Python 3 (one .py file) and run it from the console.
2. Load student data with Pandas and calculate total, percentage and grade.
3. Calculate mean, median and standard deviation using NumPy/Pandas.
4. Display the topper and the subject-wise average.
5. Plot a bar chart and a histogram with proper labels.</td></tr>
<tr><td>Instruction</td><td>1. Understand the problem statement and define the problem clearly.
2. Develop the solution using proper logic and programming concepts.
3. Implement all required features and pass the given test cases.
4. Test and debug the project to ensure correct output.
5. Prepare the documentation/report and upload the code and report to GitHub.
6. Complete and submit the project within the given deadline.</td></tr>
</table>